

Stronger Security for Blind Signatures with Attributes

Foteini Baldimtsi¹, Julia Kastner², Julian Loss³, and Omar Renawi⁴

¹ George Mason University and Mysten Labs
Fairfax, VA, USA
`foteini@gmu.edu`

² Centrum Wiskunde & Informatica
Amsterdam, Netherlands
`julia.kastner@cwi.nl`

³ Ruhr University Bochum
Bochum, Germany
`julian.loss@rub.de`

⁴ CISPA Helmholtz Center for Information Security and Saarland University
Saarbrücken, Germany
`omar.renawi@cispa.de`

Abstract. Anonymous Attribute-Based Credentials (ABCs) let users authenticate by proving only the attributes required by a policy. In many applications, *single-show* ABCs are particularly attractive because they can be instantiated from Blind Signatures with Attributes (BSAs) and showing reduces to a signature check, yielding low overhead. However, most existing Blind Signatures do not allow for attribute embedding. We provide a *generic transform* that lifts a broad class of two/three-move blind signatures to BSAs. We furthermore identify issues with prior definitions of blindness and secure showing of BSAs, particularly related to the selective reveal of attributes, and we upgrade these definitions. Our transform uses Merkle commitments and a parallel cut-and-choose to enforce attribute consistency and achieves concurrent one-more unforgeability, our stronger secure-showing notion, and the upgraded blindness. Finally, we prove that the well-known construction of *Anonymous Credentials Light* (ACL) satisfies the stronger notions without modification (in ROM+AGM). Taken together, our results consolidate the definitional landscape for single-show ABCs with attributes and provide a modular route to practical constructions.

1 Introduction

Blind signatures are a fundamental cryptographic primitive that allow a user U to obtain a signature from signer S on a hidden message and later prove validity to a verifier V without linking issuance and showing [20]. They are especially useful for building privacy-preserving protocols and, in particular, for private user authentication. A blind signature can almost directly instantiate a

lightweight Anonymous Credentials (AC) scheme, where each issued signature functions as a *single-show* token.

While the literature has been largely dominated by multi-show credential systems [20,13,46,10,14,12,7,5,17,24,8,48,32,11] that support repeated anonymous user authentications, these approaches typically rely on expensive pairings or computationally heavy zero-knowledge proofs during credential showing. By contrast, blind signature-based single-show ACs are lightweight: presenting a credential essentially reduces to simply verifying a signature while still supporting important applications. Such single-show ACs (often called *anonymous tokens*) have attracted significant attention in recent years, both by the research community [25,42,6,50,16,45] and the industry (e.g., Cloudflare [43], Apple [3], Google [30], Facebook [27]) while they are also undergoing standardization through the IETF Privacy Pass working group [36]. One crucial feature that most existing single-show ACs lack is the ability to embed *user attributes*, e.g., age, residency, or an eligibility bit into the issued credential. This is limiting because many deployments must enforce simple policy predicates at presentation time, such as pricing tiers, access eligibility, or jurisdictional restrictions, without an online check with the issuer and of course without linking to issuance. Embedding a small set of attributes in the issued token lets the verifier evaluate these predicates locally.

The only known exception is Anonymous Credentials Light (ACL) [4], which allows to embed and show attributes efficiently and is therefore considered one of the most popular single-show ACs. ACL [4] is constructed as a Blind Signature with Atttributes (BSA), an extension of blind signatures that allows embedding a commitment to user attributes inside the blindly signed message and enforces attribute consistency at showing. In this sense, BSAs naturally give rise to single-show ACs with attribute support, and can be viewed as a generalization of blind signature-based single-show credentials. Unfortunately, ACL has, so far, only been proven secure in the AGM.

Driven by recent deployments of anonymous-token systems, the central question we investigate whether attribute-friendly single-show ACs can be obtained generically from standard blind signatures and under standard assumptions, rather than requiring new, scheme-specific constructions and proofs each time. Concretely, we show the following:

- We present *Pachirisu*⁵: a generic transform which allows to transform a class of blind signature schemes, BS, into a BSA, and thus a single-show AC with attribute support, in the random oracle model. Our transform applies to many popular constructions such as the Schnorr and BLS blind signature schemes. In this manner, we are able to obtain reasonably efficient single-show AC with attribute support in the random oracle model from various standard assumptions including DLOG, RSA, factoring, the linear code equivalence problem, and GA-DLOG.

⁵ *Pachirisu* is closely related to PI-Cut-Choo [15], but it stores attributes in (Merkle) trees.

- As a contribution of independent interest, we revisit security notions from the literature related to the showing process of embedded attributes in BSAs. Existing notions consider only the oversimplified scenario in which *all* embedded attributes must be shown at presentation. As we discuss below, the correct way of how to formalize secure showing when only *a part of* the attribute vector is revealed, is still unsettled. In this work, we give the first concise definitions for this important and natural scenario.
- In the final part of our work, we apply our new security notions to ACL and prove security without modifying the original scheme.

We view **Pachirisu** as a baseline feasibility result toward obtaining attribute-friendly single-show ACs from standard assumptions with reasonable efficiency. Our work establishes an interesting modular link between blind signatures and blind signatures with attributes (BSAs) by adapting the well-known ‘boosting’ technique of Katz et al. [41] and Chairattana-Apirom et al. [15] to the attribute setting. Although concentered community efforts have improved on the efficiency of these early works significantly, we stress that schemes obtained via boosting were considered to be at the forefront of efficient blind signature constructions from standard assumptions as recently as three years ago at the time of writing. We believe that, analogous to how the blind-signature landscape has evolved through successive optimizations on these and other techniques, our work will serve as a starting point for a more systematic effort toward practical BSAs, and thus attribute-based ACs, from standard assumptions.

2 Technical Overview

As already explained, the purpose of our generic transform **Pachirisu** is to turn a blind signature scheme **BS** into a blind signature with attributes (BSA). By $\mathbf{P}[\mathbf{BS}]$ we denote the output scheme of our transformation.

2.1 Our Transformation

We introduce **Pachirisu** (denoted $\mathbf{P}[\cdot]$), a transformation that can be applied to a large class of *key-only-secure two- or three-move* blind signature schemes **BS** (i.e. schemes unforgeable against adversaries who only get access to the verification key, but do not get to see any signatures). The goal of our transform is to obtain a BSA that achieves selective secure showing as well as one-more unforgeability with a polynomial number of concurrent signing sessions.

Our transformation turns a scheme **BS** that merely supports blind signing of plain messages, into a scheme $\mathbf{P}[\mathbf{BS}]$ in which the user can embed a commitment to its attributes inside the signed message. Conceptually, the construction can be viewed as a variant of *blind-signature boosting*: we lift a plain blind signature into a blind signature with attributes while preserving blindness and strengthening unforgeability.

A crucial feature of our construction is ensuring that the signer actually issues signatures with respect to the “right” attributes. A natural idea would be

to require the user to provide a non-interactive zero-knowledge (NIZK) proof that the committed attributes were encoded honestly. However, because many efficient blind signatures (including most of the schemes we target) are proven secure in the Random Oracle Model, plugging in a NIZK is delicate. In the ROM, a NIZK proof of knowledge requires proving knowledge of the *random-oracle output* itself, creating circular dependencies where the simulator must both program the oracle and extract from proofs that depend on that programming. This interaction breaks standard simulation strategies and is incompatible with the way ROM-based blind signatures are analyzed.

Boosting. To avoid these issues, we instead rely on the (parallel-instance) cut-and-choose strategy first introduced in [41] and further developed in [15] with the goal of strengthening the security of blind signature schemes so that they become one-more unforgeable for a larger number of concurrent signing sessions. One of the key challenges addressed by these transforms is to overcome the ROM-related issues described above, they turn out to be an ideal starting point for $\mathbf{P}[\cdot]$.

We specifically build on the variant by [15] which only requires key-only security of the underlying scheme. We briefly recall the transform: The signer generates K public keys of the underlying (key-only secure) blind signature scheme. To sign a message, the signer and user run N parallel sessions for each of the public keys (where N is a number that is chosen dependent on how many signing sessions have already been opened with the signer). To ensure honest behaviour by the user, the user has to commit to all of their randomness at the beginning of each session. Later on, before receiving the last message from the signer, the user is asked to open all but one (random) commitment among the parallel sessions corresponding to each of the K public keys. This allows the user to show that the first user messages were computed honestly. Since the commitments were chosen at random, it is unlikely that the user gets away with cheating (i.e., using randomness different from that in the commitment) in *all* of the unopened sessions. In fact, through a simple concentration argument, it is possible to show that for one of the public keys $\mathbf{pk}$, the adversary will always have to use pre-committed randomness in order to close the final part of the signing protocol. This allows a reduction from key only security of the scheme corresponding to $\mathbf{pk}$. Blindness is preserved because the signer only closes the session for which it did not ask to open the commitment. The final signature consists of K subsignatures $(\sigma_1, \dots, \sigma_K)$.

Adding Attributes. As the focus of our work is building blind signatures with attributes for one-show anonymous credentials, it is important that we also achieve strong one-more unforgeability, so that the same attribute vector could not be shown more often than it was signed. It turns out that this is not achieved by original parallel-instance cut-and-choose transform. Namely, for two signature tuples $(\sigma_1, \dots, \sigma_K)$ and $(\sigma'_1, \dots, \sigma'_K)$ on the same message M , $(\sigma'_1, \sigma_2, \dots, \sigma_K)$ is also a valid signature on M .

Our main insight is that one can modify the construction of [15] so that the message for subsignature σ_i contains the subsignature σ_{i-1} . We therefore run the cut-and-choose sessions for the different public keys sequentially. As a result, a strong forgery (i.e., two signatures on the same message) can be translated into a forgery on two different messages with respect to the first index where the strong forgeries begin to diverge. Some additional properties (see below) of the underlying scheme BS are needed to prevent a strong forgery in which only the outermost signature indices differ.

This explanation of course sweeps some technical details under the rug. First, to achieve blindness, the user needs to use a commitment to its message as the message for the underlying scheme in each signing session, so that when it reveals its randomness and computation of the first user message, it does not reveal the underlying message that it is trying to get signed. On the other hand, the user has to provide a commitment to their attributes inside the message. This we achieve using Merkle-Tree commitments with fresh randomness for each public key and each parallel session. Merkle-Tree commitments can be opened partially which comes in handy when proving our new stronger secure showing property for our construction.

Compatible Schemes. We identify a class of schemes that are compatible with our construction. We dub this class *three-move simulateable blind signature schemes*. These schemes have the following (simplified) properties:

- They have three or less moves. If they have two moves, we consider them to have three moves but with an empty first message from the signer.
- During issuance, the user makes one random-oracle query that is repeated during verification of the resulting signature. This query is linked⁶ to the signing session, user randomness, and message to be signed.
- There exists a signer simulation algorithm that produces a simulated first signer message, and a state. Using the state, the message to be signed, and the user’s random coins, random oracle responses can be simulated, and there exists a simulator algorithm that uses this information to produce a matching second signer message.

As key-only unforgeability is sufficient for our security reductions, we can use a wide range of schemes captured by the above definition without having to worry about ROS-style attacks. Candidate schemes include many Schnorr-style blind signatures, for example Schnorr [49] (from DLOG), Abe-Okamoto [2] (from DLOG), Abe’s blind signature [1] (from DLOG), Tessaro-Zhu [51] (from DLOG), Okamoto-Schnorr [44] (from DLOG), CSI-Otter [40] (from GA-DLOG instantiated with CSI), Tanuki [31] (from GA-DLOG instantiated with the CSI or LESS group action), and the schemes based on linear function families described in [33] (from DLOG, factoring, or RSA). It can also be used as in [15] for two-move schemes such as blind BLS [9] (from CDH in pairing groups) or blind RSA [19]

⁶ The exact meaning of this is going to be specified in more detail in section 4.

(from RSA) by setting the first signer message to be an empty string and making only very minor modifications to the security proofs.

2.2 The Right Security Framework for BSAs

The first security definition for BSAs was given in [4] where one-more unforgeability and secure showing of attributes were covered by one definition. In a recent work, Kastner et al. [37] provide a new, modular security definition which separates the security of the showing procedure from one-more unforgeability. In doing so, however, their work also uncovers a subtlety with the original showing security definition of Baldimtsi and Lysyanskaya. In their security experiment, the adversary interacts with the signer and obtains a set of ℓ signatures $\sigma_1, \dots, \sigma_\ell$ with corresponding attribute vectors $\mathbf{L}'_1, \dots, \mathbf{L}'_\ell$. The adversary is considered successful in breaking the secure showing property if, at the end of the experiment, it can produce an alternative multiset of vectors $\mathbf{L}_1, \dots, \mathbf{L}_\ell$ which differs from the original one in at least one component, yet still verifies against $\sigma_1, \dots, \sigma_\ell$. (We consider multisets as the same attribute vector may appear multiple times among $\mathbf{L}'_1, \dots, \mathbf{L}'_\ell$.) While this intuitively ensures that the adversary cannot alter its attributes post-facto, closer scrutiny reveals that this definition does not cover the natural scenario in which the user only *does a selective opening of its attributes* in the showing procedure (partial reveal). Although the option of partial attribute reveal is provided by the syntax of the showing procedure, the security experiment corresponding to the showing definition only allows for the full set of attributes to be shown!

Secure Showing with Selective Reveal In Section 6.1, we formalize a stronger secure showing definition for blind signatures with attributes which remedies the aforementioned shortcomings by supporting selective attribute reveal. We note that our new definition is the natural one needed in order to use single-use BSAs in applications (since most of them do require partial showing for attributes!).

Our new definition, and later the proofs that our generic transform and ACL satisfy it, comes with several unexpected subtleties both at a definitional and technical level. In our newly proposed notion, the adversary can again query the challenger of the security experiment for signatures corresponding to attribute vectors $\mathbf{L}'_1, \dots, \mathbf{L}'_k$. Similar to the original definition, the adversary has to provide, at the end of the experiment, a set of vectors $\mathbf{L}_1, \dots, \mathbf{L}_\ell$ which each verify against at least one of the signatures σ_i . However, different from before, $\mathbf{L}_i$ can also be *partial* vectors, that is some entries can be replaced by a special empty symbol $\perp$ (for honest users, these partial vectors would be subvectors of the previously signed vectors $\mathbf{L}'_i$, i.e. they would agree on the non- $\perp$ entries).

A New Winning Condition. At the heart of our modified experiment lies a new winning condition which takes into account the above discussion. Intuitively, this condition has to check whether the set of opened partial vectors cannot be

explained by a set of honest users opening their signatures. For such an honest set of users, there would be an explanation for each of the opened vectors $\mathbf{L}_i$ by a signing session with a vector $\mathbf{L}'_j$ such that $\mathbf{L}_i$ is a subvector of $\mathbf{L}'_j$ and no signing session is used to explain more than one opened signature. More formally, the game checks if there is an *injective* mapping from the opened (partial) vectors $\mathbf{L}_1, \dots, \mathbf{L}_\ell$ to the signed vectors $\mathbf{L}'_1, \dots, \mathbf{L}'_k$ (multiply signed vectors appear multiple times) such that the opened vectors are subvectors of the corresponding signed vectors. If no such mapping exists, the adversary has won the game, as it must have altered at least one of the signed vectors post-facto. Alternatively, this means for any attempt at explaining the opened partial vectors through the signatures issued, there is at least one signature with opened vector $\mathbf{L}_i$ that cannot be explained by a signing session.

For example, consider an adversary that queried the challenger for one signature corresponding to $\mathbf{L}'_1 = (a, b)$ and two signatures corresponding to $\mathbf{L}'_2 = \mathbf{L}'_3 = (a, c)$. If it outputs three signatures $\sigma_1, \sigma_2, \sigma_3$ with (verifying) corresponding partial openings to attributes $(a, c), (\perp, c), (\perp, c)$, it wins this game. Namely, since both (a, c) and $(\cdot, c)$ are subvectors only to $\mathbf{L}'_2 = \mathbf{L}'_3$, but not $\mathbf{L}'_1$, it is impossible to assign the three openings as subvectors injectively to the (multi-)set of previously signed attribute vectors $\{\mathbf{L}'_1, \mathbf{L}'_2, \mathbf{L}'_3\}$. On the other hand, an adversary that opens the signatures to $(a, \perp), (\perp, c), (\perp, c)$ would not win the game, as the $(a, \perp)$ now constitutes a subvector to any of $\mathbf{L}'_1, \mathbf{L}'_2$ and $\mathbf{L}'_3$, and the opened vectors can be explained by $(a, \perp) \mapsto \mathbf{L}'_1, (\perp, c) \mapsto \mathbf{L}'_2, (\perp, c) \mapsto \mathbf{L}'_3$.

At a first glance, it may seem unclear whether this new secure showing definition is satisfiable *at all* as it seemingly asks the challenger to check an exponentially complex set of constraints related to the winning condition of the security experiment. In Section 6.1 we explain how to overcome this obstacle by abstracting the secure showing experiment as a bipartite graph G and using the Hopcroft-Karp algorithm [35] to find a maximum cardinality matching.

Upgrading the Blindness Definition. The standard blindness notion in the literature concerns *issuance* only: it hides which signing session produced which signature, but it says nothing about privacy *after showing*. If a user later reveals *all* attributes, they may be trivially identifiable when their attribute vector is unique, so pre-showing blindness is essentially the best guarantee in that case. In contrast, under *partial* reveal an honest user should still enjoy anonymity within the cohort of users who revealed the *same* attributes (e.g., same age or residency).

Motivated by this, we propose a natural extension to the blindness notion that explicitly includes registration and showing. It guarantees unlinkability between issuance sessions and showings among all presentations that reveal identical (fully or partially opened) attributes. The definition does not assume that attributes are hidden at registration, accommodating applications where some or all attributes must be disclosed to the issuer during enrollment.

2.3 Security Proofs for Our Transformation

One-More Unforgeability. The proof largely follows that of the CDH-based scheme in [15], but we use the properties of the underlying generic scheme instead of concrete scheme-specific simulation strategies.

The proof strategy is to provide a reduction that breaks key-only security of the underlying scheme BS. The reduction is thus tasked with providing signatures for the underlying scheme BS without having access to a signing oracle. The property we use for this is, that if given the random coins used to generate the user message, as well as the capability to program the random oracle in some points, the reduction can answer signing queries using the simulators. For Schnorr-style schemes, this is essentially the simulator of the underlying Fiat-Shamir proof, whereas for RSA and BLS, this is done by programming the random oracle with a point for which the reduction knows the signature. The reduction then needs to apply the appropriate blinding factors to its solution. Therefore, we need to make sure that there is at least one public key for which the reduction can always extract the user’s randomness so that it knows all the blinding factors. As the adversary is tasked to commit to all of its randomness, we show that with a high probability, the adversary cannot “cheat” where by cheating we mean that there is a session, where it manages to successfully reveal all of its randomness, but there is no matching commitment for the unrevealed sessions. This can be achieved through an appropriate choice of the number K of keys and the number of cut-and-choose sessions N .

Secure Showing The proof of our secure-showing property mirrors the one-more unforgeability proof. There are two ways a user can break secure showing:

- It “cheats” such that the signer sees attribute vector $\mathbf{L}$ revealed during signing, but the unrevealed sessions are using a different attribute vector $\mathbf{L}'$. Such cheating requires guessing all cut-and-choose indices correctly for that session. This can be ruled out using the same argument as before.
- It opens a commitment to a different attribute (sub)vector. This can be ruled out using the binding property of the underlying Merkle-Tree commitment scheme.
- It generates a novel signature without the help of the signer. This can be ruled out using the key-only security like before.

Blindness For blindness, we use the underlying scheme’s blindness to gradually swap which message is used during signing across sessions. To do this, we first decouple the opened messages from their commitments, as well as decouple the Merkle-tree commitments from the attribute vectors. We achieve this with standard random-oracle techniques, since all commitments are computed via random oracles. This enables swapping messages in a hybrid argument based on the blindness of BS. For the opening phase, we are able to simulate the openings of the Merkle-Tree-based attribute commitments using the random oracle.

2.4 An efficient scheme: ACL

We furthermore prove that the popular Anonymous Credentials Light scheme fulfills our new secure showing definition, as well as and the upgraded blindness definition. In Appendix G.5 we also show how a simple modification can be used to detect double spending.

To show secure showing, we rely on similar techniques in the algebraic group model as [37] used for OMUF. Recall that in ACL, the attributes are embedded as a Pedersen commitment in a component of the signature, and this same component forms a Diffie-Hellman tuple with components from the signer messages. Therefore, each signature has a (computationally hidden) link to at most one signing session, and this allows us to categorize attacks on Secure Showing in the following way:

- The adversary provides a signature that is not linked to a signing session. This can be ruled out using similar techniques as for the OMUF-proof provided in [37], in particular we prove a variant of what is often dubbed the “restrictive-blinding lemma” [1,4,39,37].
- The adversary provides two signatures linked to the same session. This can also be ruled out using similar techniques as in [37] and essentially boils down to a 2-1-ROS problem which is information-theoretically hard.
- Lastly, the adversary could open a commitment to different attributes. Using extraction techniques for the proof of knowledge of an opening in the AGM, we extract the full opening and show that this breaks the binding property of the underlying Pedersen commitment.

The blindness proof is very similar to that in [4], but taking into account that the adversary may also request the user(s) to show some of their attributes. We simulate the showing of these attributes using simulation of the proof of knowledge in the ROM.

For double-spending security, we provide a simple modification of the ACL scheme which is based on a similar protocol proposed by Abe [1] for the blind signature scheme ACL is based on. This same scheme was also implemented for ACL in [34] but without defining or proving its security. The key idea is to add the first flow of a sigma-protocol to be used in signature showing to the signature in such a way that it cannot be removed/exchanged, namely by appending it in the string that is hashed at signature generation time. Then, the showing protocol also involves proving knowledge of the user’s attributes using this first flow contained in the signature. We implement this using the Fiat-Shamir (FS) heuristic. We note that both the user and the signer include a current timestamp to derive their FS challenge. This is important in order to ensure that if the user shows the same signature twice with different timestamps, the challenge from the hash function will be different. As the user has to always use the same first flow contained in the signature, the two different transcripts reveal his identity.

3 Preliminaries and Definitions

Notation. For a natural number n , we denote by $[n]$ the set of numbers $\{1, \dots, n\}$ and by $[n]_0$ the set $\{0\} \cup [n]$. Throughout the paper, we fix a group $\mathbb{G}$ of prime order p with generator g and assume that all algorithms have access to this group's parameters even when not given as an explicit input. We use $:=$ for deterministic assignments and $\leftarrow_{\S}$ to denote that a value is sampled at random from a set or the output of a randomized algorithm. More explicitly, we use the notation $b \leftarrow_{\S} \mathcal{A}(a)$ to denote a randomized algorithm with input a and output b . We write $\mathcal{A}^{\mathcal{O}(\cdot)}$ if $\mathcal{A}$ is equipped with oracle access to an algorithm $\mathcal{O}$.

$\mathbf{L}$ is a vector and L_i the i_{th} entry of $\mathbf{L}$. We say a vector $\mathbf{L}'$ is a *partial* vector if for some indices i , $L'_i = \perp$ where $\perp$ is a special empty symbol. For a vector $\mathbf{L}$ of length n and a (partial) vector $\mathbf{L}'$, we say $\mathbf{L}'$ is a *subvector* of $\mathbf{L}$, denoted by $\mathbf{L}' \subseteq \mathbf{L}$, if for each index $i \in [n]$ it holds that either $L'_i = L_i$ or $L'_i = \perp$. A *multiset* is a generalization of a set where each element can appear multiple times. We call the number of times an element x appears in a multiset the *multiplicity* of x . For multisets M, M' we say that M' is a *subset* of M , denoted by $M' \subseteq M$, if each element in M' is also contained in M where the multiplicity of each element contained in both sets is lower or equal for M' than for M .

In the following sections, we assume that there exist public parameters $\mathbf{pp}$ that are known to all algorithms even when not explicitly given as an input.

3.1 Multi-index Vector Commitment

In this work, we use a commitment-notion called Multi-index Vector Commitment abstracting the properties of Merkle-Tree Commitments that we need with algorithms $\text{MiVC} := (\mathcal{C}, \mathcal{O}, \mathcal{CV})$. $\mathcal{C}$ allows to commit to a vector $\mathbf{L}$ using randomness $\mathbf{R}$. $\mathcal{O}$ allows to partially open the commitment at indices $\mathbf{I}_{\text{op}}$. $\mathcal{CV}$ allows to verify an opening of a commitment at the set of indices. The security properties are straightforward. We provide a formal definition in appendix A.5.

Merkle Tree-based Multi-index Vector Commitment

Construction 1. We describe Multi-index Vector Commitment schemes $\text{MT} := (\mathcal{C}, \mathcal{O}, \mathcal{CV})$ that are based on Merkle trees. Let $\text{H}_{\text{MT}}: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a hash function treated as a random oracle.

- $\mathcal{C}(\mathbf{L}, \mathbf{R})$. The algorithm takes as input a message vector $\mathbf{L} = (L_1, \dots, L_\nu)$ and a randomness vector $\mathbf{R} = (R_1, \dots, R_\nu)$. First, it ensures that no $L_i = \perp$ and aborts otherwise. It computes the sequence of leaves $S = (s_1, \dots, s_\nu)$ where:

$$s_i := \begin{cases} \text{H}_{\text{MT}}(L_i, R_i) & \text{if } R_i \neq \perp \\ L_i & \text{otherwise} \end{cases}$$

It then constructs a binary Merkle tree over the sequence S and outputs the root as the commitment C .

- $\mathcal{O}(C, \mathbf{L}, \mathbf{R}, \mathbf{I}_{op})$. Parse $(L_1, \dots, L_\nu) := \mathbf{L}, (R_1, \dots, R_\nu) := \mathbf{R}$. For all $i \in \mathbf{I}_{op}$, let $R'_i := R_i$ and $L'_i := L_i$, and for all $j \in [\nu] \setminus \mathbf{I}_{op}$, let $R'_j := \perp$ and $L'_j := \text{H}_{\text{MT}}(L_j, R_j)$. Let $\mathbf{R}' := (R'_1, \dots, R'_\nu)$ and $\mathbf{L}' := (L'_1, \dots, L'_\nu)$, and return $(\mathbf{L}', \mathbf{R}')$.
- $\mathcal{CV}(C, \mathbf{L}', \mathbf{R}', \mathbf{I}_{op})$. Parse $(R'_1, \dots, R'_\nu) := \mathbf{R}'$, and output 0 if $\exists i \in \mathbf{I}_{op}$ with $R'_i = \perp$. Outputs 1 if $\mathcal{C}(\mathbf{L}', \mathbf{R}') = C$, otherwise it outputs 0.

Our scheme is Correct (Lemma 5), Hiding (Lemma 6), and Binding (Lemma 7) as proven in appendix A.9.

3.2 Blind Signature with Attributes

As a starting point, we use the definition presented in [37].

Definition 1. A blind signature with attributes is a tuple of algorithms $\text{BSA} := (\mathcal{G}, \mathcal{RG}, \mathcal{S}, \mathcal{U}, \mathcal{V}, \mathcal{SH})$, with the following properties:

- The key generation algorithm $\mathcal{G}$ takes as input public parameters pp and outputs a pair of keys (pk, sk) for the signer. In the rest of the paper we assume that pk implicitly contains a copy of pp and sk implicitly contains a copy of pk .
- The registration protocol $\mathcal{RG}$ between the user and the signer consists of two algorithms $\mathcal{RG}_{\mathcal{U}}$ and $\mathcal{RG}_{\mathcal{S}}$:
 - The user's registration algorithm $\mathcal{RG}_{\mathcal{U}}$ takes as input a vector of attributes $\mathbf{L}$, and outputs a commitment C , user's registration state $\text{rst}_{\mathcal{U}}$, and a proof π . We assume the state $\text{rst}_{\mathcal{U}}$ contains the commitment C which can be accessed as $\text{rst}_{\mathcal{U}}.C$. It further contains information that the user needs to participate in signing and showing interactions.
 - The signer's registration algorithm $\mathcal{RG}_{\mathcal{S}}$ takes as input a commitment C , and a proof π , and a signer's registration state $\text{rst}'_{\mathcal{S}}$. It outputs an updated signer's registration state $\text{rst}_{\mathcal{S}}$ or $\perp$ (ERROR) to the signer, and it returns 1 (COMPLETE) or 0 (ERROR) to the user. The signer uses $\text{rst}_{\mathcal{S}}$ later to identify registered commitments.
- The interactive signing/issuance algorithm $\mathcal{S}$ takes as input a secret key sk , a registration state $\text{rst}_{\mathcal{S}}$, and a commitment C , and outputs 1 (COMPLETE) or $\perp$ (ERROR).
- The interactive user algorithm $\mathcal{U}$ takes as input the signer's public key pk , a registration state $\text{rst}_{\mathcal{U}}$ ⁷, and a message m , and outputs a signature σ , and an opening state $\text{ost}_{\mathcal{U}}$. We assume the signature σ contains a blinded version of the commitment $\text{rst}_{\mathcal{U}}.C$ and $\text{ost}_{\mathcal{U}}$ contains an opening for $\text{rst}_{\mathcal{U}}.C$.
- The interactive show algorithm $\mathcal{SH}$ is split into two algorithms $\mathcal{SH}_{\mathcal{U}}$ and $\mathcal{SH}_{\mathcal{V}}$.
 - The show generation algorithm $\mathcal{SH}_{\mathcal{U}}$ takes as input a public key pk , the user's registration and opening states $\text{rst}_{\mathcal{U}}, \text{ost}_{\mathcal{U}}$, a message m , a signature σ , and an index set I_{op} . It outputs a partial vector $\mathbf{L}'$ and a proof $\bar{\pi}$.

⁷ Recall that $\text{rst}_{\mathcal{U}}$ encodes a commitment accessible by $\text{rst}_{\mathcal{U}}.C$

- The deterministic show verification algorithm $\mathcal{SH}_V$ takes as input a public key $\mathbf{pk}$, a message m , a signature σ , a (partial) attribute vector $\mathbf{L}'$, an index set I_{op} corresponding to the non- $\perp$ indices in $\mathbf{L}'$, and a proof $\bar{\pi}$. It outputs 1 (ACCEPT) or 0 (REJECT).
- The deterministic signature verification algorithm $\mathcal{V}$ takes as input a public key $\mathbf{pk}$, a message m , and a signature σ . It outputs 1 (ACCEPT) or 0 (REJECT).

Remark 1. Blind Signatures with Attributes are equipped with two verification algorithms, $\mathcal{V}$ and $\mathcal{SH}_V$. While $\mathcal{V}$ checks a signature's validity, i.e., whether the signature was issued by an honest signer, $\mathcal{SH}_V$ checks whether a *show* tuple consists of a valid signature and a valid proof of knowledge of a vector of attributes, for which the signature was granted.

Remark 2. In our registration algorithm, we do not require the user attributes to be hidden in the commitment, i.e. the commitment can be the vector $\mathbf{L}$ of the attributes, and the proof can be $\perp$. Depending on the protocol or the application, the user may be required to reveal some or all of their attributes at registration. In fact, in our transformation $\mathbf{P}[\mathbf{BS}]$, the user is required to reveal all of its attributes, whereas in $\mathbf{ACL}$, all attributes may remain hidden at registration.

Definition 2 (Correctness of Blind Signatures with Attributes). *We say that a blind signature with attributes scheme $\mathbf{BSA} = (\mathcal{G}, \mathcal{RG}, \mathcal{S}, \mathcal{U}, \mathcal{V}, \mathcal{SH})$ is perfectly correct if the following holds: for all public parameters $\mathbf{pp}$, all vectors of attributes $\mathbf{L}$, all signer registration states $\mathbf{rst}'_S$, all index sets $I_{op} \subset [\nu]$, and all messages m ,*

1. *Generate a pair of keys $(\mathbf{pk}, \mathbf{sk}) \leftarrow_{\$} \mathcal{G}(\mathbf{pp})$.*
2. *Run the registration protocol honestly by executing $(\mathbf{rst}_U, \pi) \leftarrow_{\$} \mathcal{RG}_U(\mathbf{L})$ followed by $\mathbf{rst}_S \leftarrow \mathcal{RG}_S(\mathbf{rst}_U.C, \pi, \mathbf{rst}'_S)$.*
3. *Run the signing protocol honestly by the signer $\mathcal{S}(\mathbf{sk}, \mathbf{rst}_S, \mathbf{rst}_U.C)$ and the user $\mathcal{U}(\mathbf{pk}, \mathbf{rst}_U, m)$ resulting in the signer outputting 1 and the user obtaining a signature σ and an opening state $\mathbf{ost}_U$.*
4. *Run the user's showing procedure honestly by executing $(\mathbf{L}', \bar{\pi}) \leftarrow_{\$} \mathcal{SH}_U(\mathbf{pk}, \mathbf{rst}_U, m, \sigma, \mathbf{ost}_U, I_{op})$.*
5. *Both verifications $\mathcal{V}(\mathbf{pk}, m, \sigma)$ and $\mathcal{SH}_V(\mathbf{pk}, m, \sigma, \mathbf{L}', I_{op}, \bar{\pi})$ output 1.*

We recall the definition of strong One-More Unforgeability, where an adversary wins if it is able to output $\ell + 1$ valid message-signature pairs after completing only ℓ signing sessions. In the setting of blind signatures with attributes, this game is augmented with a registration oracle. The registration oracle $\mathcal{RG}_S$ allows the adversary to register users with attributes of its choice. The game keeps the internal state the signer would keep regarding registered users which includes a list of the registered attribute commitments. It responds with 1/0 if the signer side registration algorithm outputs $\mathbf{rst}_S/\perp$, respectively. The signing oracle $\mathcal{S}(\mathbf{sk}, \cdot)$ simulates the signing algorithm for the adversary. In schemes with multiple rounds we denote the signing algorithm to be split into

multiple sub-algorithms. Then, the adversary is provided with an oracle for each sub-algorithm, and the experiment maintains a state internally. If the scheme consists of multiple rounds, the adversary may query the signing oracles concurrently and in an interleaved manner (e.g. make many calls to $\mathcal{S}_1$ before making the call with the same session identifier to $\mathcal{S}_2$).

Definition 3 (Strong One-More Unforgeability (OMUF)). For a blind signature with attributes $\text{BSA} = (\mathcal{G}, \mathcal{RG}, \mathcal{U}, \mathcal{S}, \mathcal{V}, \mathcal{SH})$, public parameters pp , a positive integer $\ell \in \mathbb{Z}^+$, and an adversary $\mathcal{A}$, we define Game ℓ -OMUF as follows:

1. Generate key pair $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}(\text{pp})$.
2. Invoke $\mathcal{A}^{\mathcal{RG}_S(\cdot), \mathcal{S}(\text{sk}, \cdot)}(\text{pk})$. $\mathcal{A}$ may query its oracles $\mathcal{RG}_S(\cdot)$ and $\mathcal{S}(\text{sk}, \cdot)$ arbitrarily and in an interleaved fashion as long as it completes at most ℓ sessions with $\mathcal{S}$.
3. $\mathcal{A}$ outputs $\ell + 1$ distinct tuples $(\sigma_1, m_1), \dots, (\sigma_{\ell+1}, m_{\ell+1})$.
4. Output 1 if for all $i \in [\ell + 1]$, $\mathcal{V}(\text{pk}, m_i, \sigma_i) = 1$.

We define $\mathcal{A}$'s advantage in winning the game ℓ -OMUF against a blind signature scheme with attributes BSA as

$$\text{Adv}^\ell\text{-OMUF}_{\text{BSA}}(\mathcal{A}) := \Pr[\ell\text{-OMUF}_{\text{BSA}}^{\mathcal{A}} = 1].$$

We say that BSA is (t, ε, ℓ) -OMUF-secure if for all adversaries $\mathcal{A}$ that run in time at most t , $\text{Adv}^\ell\text{-OMUF}_{\text{BSA}}(\mathcal{A}) \leq \varepsilon$.

4 Three-Move Simulatable Blind Signature Schemes

In this section, we describe the class of blind signature schemes underlying our generic construction.

Definition 4 (Three-Move Simulatable Blind Signature Schemes). For a security parameter λ , let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a hash function treated as a random oracle and let pp be a public parameter. We consider blind signature schemes $\text{BS}^H = (\mathcal{G}^H, \mathcal{S}, \text{STM}_{\mathcal{S}}, \mathcal{U}^H, \mathcal{V}^H)$ that have the following structure:

- **Key Generation** $\mathcal{G}$ takes as input public parameters pp and outputs a key pair (pk, sk) .
- **Signer Algorithms** $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ The Signer algorithms are split into two:
 - $\mathcal{S}_1^H(\text{sk})$ The probabilistic first stage signer algorithm takes as input the secret key sk and outputs a first signer message msgS1 and the signer's randomness rnd_S .
 - $\mathcal{S}_2^H(\text{sk}, \text{rnd}_S, \text{msgU})$ The deterministic second stage signer algorithm takes as input the secret key sk , the signer's randomness rnd_S , and the user message msgU , and outputs a response msgS2 .
- **User Algorithms** $\mathcal{U}^H = (\mathcal{U}_1^H, \mathcal{U}_2^H)$. The user algorithm is split into two:
 - $\mathcal{U}_1^H(\text{pk}, m, \text{msgS1}; \text{Urnd})$ The probabilistic first user stage takes as input the public key pk , the message m and a first signer message msgS1 . We write the entire user randomness Urnd explicitly here. It outputs a user message msgU .

- $\mathcal{U}_2^H(\text{pk}, m, \text{Urnd}, \text{msgS2})$ The deterministic second user stage takes as input the public key pk , the message m , the first stage user randomness Urnd , and the second signer message msgS2 . It outputs a signature σ .
- **Verification** $\mathcal{V}^H(\text{pk}, m, \sigma)$ The deterministic verification algorithm takes a public key pk , a message m , and a signature σ as input, and it outputs 1 (ACCEPT) or 0 (REJECT).
- **Signer Simulation Algorithms** $\text{SIM}_S^H = (\text{SIM}_{S_1}^H, \text{SIM}_{S_2}^H, \text{SIM}_H)$. They are defined as follows:
 - $\text{SIM}_{S_1}(\text{pk})$ outputs a signer message msgS1 and a state simSt .
 - $\text{SIM}_{S_2}(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$ takes as input a signer's first message msgS1 , a simulation state simSt , a message m , and user randomness Urnd . It outputs a signer message msgS2 , such that msgS1 and msgS2 are identically distributed to a real execution of the S_1, S_2 with $\text{msgU} = \mathcal{U}_1^{H'}(\text{pk}, \text{msgS1}, m; \text{Urnd})$ where H' returns the hash response provided by SIM_H .
 - $\text{SIM}_H(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$ takes as input a signer's first message msgS1 , a simulation state simSt along with a message m and a user randomness Urnd . It outputs a corresponding hash response c . In case of a two-move scheme, we assume that SIM_H takes as input only the public key and the message m and outputs a hash response c and a simulation state simSt .

Throughout this work, we often drop the superscript H as long as it is clear from the context.

Our generic construction requires the underlying blind signature scheme BS to satisfy the following properties.

Property 1 (Unique Hash Query). In an honest execution of BS , the user algorithm $\mathcal{U}_1^H(\text{pk}, m, \text{msgS1}; \text{Urnd})$ makes a single hash query to the random oracle H . We denote this query as $\text{hq}(\text{pk}, m, \text{msgS1}; \text{Urnd})$. Similarly, for each valid signature-message pair (σ, m) , the verification procedure $\mathcal{V}^H(\text{pk}, m, \sigma)$ makes exactly one query to H . If the signature σ was generated honestly, the query $\mathcal{V}$ makes is identical to the query $\text{hq}(\text{pk}, m, \text{msgS1}; \text{Urnd})$ made by $\mathcal{U}_1$ beforehand. This query is uniquely determined by the signer message msgS1 , the user coins Urnd , and the message m .

Property 2 (Signature-hash Linking). Let $\mathcal{A}$ be an adversary running in time at most t_{SHL} . $\mathcal{A}$ is invoked with a public key pk as input and has oracle access to the signing oracles $\mathcal{S}_1(\text{sk}, \cdot)$ and $\mathcal{S}_2(\text{sk}, \cdot)$ in addition to the random oracle H . Once $\mathcal{A}$ terminates, it outputs a message-signature pair (m, σ) , and it wins if $\mathcal{V}(\text{pk}, m, \sigma) = 1$ and the single random oracle query Q made by $\mathcal{V}$ to H was not seen before. It must hold

$$\Pr \left[\begin{array}{l} \mathcal{V}^H(\text{pk}, m, \sigma) = 1 \\ \wedge T[Q] = \perp \end{array} \middle| \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow_{\S} \mathcal{G}(\lambda) \\ (m, \sigma) \leftarrow_{\S} \mathcal{A}^{\mathcal{S}_1, \mathcal{S}_2, H}(\text{pk}) \end{array} \right] \leq \varepsilon_{\text{SHL}}.$$

Property 3 (Unforgeability for $\mathcal{U}_1^H$). Let $\mathcal{A}$ be an adversary running in time at most t_{UUF} and outputting $(\text{msgU}, \text{pk}, m, \text{msgS1}, \text{Urnd})$, and let $\mathbf{T}$ be a snapshot of the hash table of the oracle $\mathbf{H}$ by the time $\mathcal{A}$ submits its output. Let $Q := \text{hq}(\text{pk}, m, \text{msgS1}, \text{Urnd})$ be the corresponding unique hash query made by $\mathcal{U}_1^H(\text{pk}, m, \text{msgS1}; \text{Urnd})$. Then it holds

$$\Pr \left[\begin{array}{l} \text{msgU}' = \text{msgU} \\ \wedge \mathbf{T}[Q] = \perp \end{array} \middle| \begin{array}{l} (\text{msgU}, \text{pk}, m, \text{msgS1}, \text{Urnd}) \leftarrow_{\$} \mathcal{A}() \\ Q := \text{hq}(\text{pk}, m, \text{msgS1}, \text{Urnd}) \\ \text{msgU}' := \mathcal{U}_1^H(\text{pk}, m, \text{msgS1}; \text{Urnd}) \end{array} \right] \leq \varepsilon_{\text{UUF}}.$$

Property 4 (Unique $\mathcal{U}_1$ Output). Let $\mathcal{A}$ be an adversary that runs in time at most t_{UO} . $\mathcal{A}$ takes as input a public key pk and a signer message msgS1 , and it outputs two distinct pairs (m_1, Urnd_1) and (m_2, Urnd_2) , it holds

$$\Pr \left[\begin{array}{l} (m_1, \text{Urnd}_1) \neq (m_2, \text{Urnd}_2) \\ \text{msgU}_1 = \text{msgU}_2 \end{array} \middle| \begin{array}{l} (m_1, \text{Urnd}_1, m_2, \text{Urnd}_2) \leftarrow_{\$} \mathcal{A}(\text{pk}, \text{msgS1}) \\ \text{msgU}_1 := \mathcal{U}_1^H(\text{pk}, m_1, \text{msgS1}; \text{Urnd}_1) \\ \text{msgU}_2 := \mathcal{U}_1^H(\text{pk}, m_2, \text{msgS1}; \text{Urnd}_2) \end{array} \right] \leq \varepsilon_{\text{UO}}.$$

Property 5 (Message Hashing). The unique hash query $\text{hq}(\text{pk}, m, \text{msgS1}; \text{Urnd})$ made by $\mathcal{V}^H(\text{pk}, m, \sigma)$ to $\mathbf{H}$ contains the message m in plain, i.e. the query is of the form $(\dots, m, \dots)$. Consequently, due to Property 1, this applies to queries initiated from $\mathcal{U}_1$.

Property 6 (Unique Link from Hash Query to Signing Session). For any adversary $\mathcal{A}$ that runs in time at most t_{UL} , any $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}$, and any (not necessarily distinct) $\text{msgS1}_1 \leftarrow_{\$} \mathcal{S}_1(\text{sk}), \text{msgS1}_2 \leftarrow_{\$} \mathcal{S}_1(\text{sk})$, it holds

$$\Pr \left[\begin{array}{l} (\text{msgS1} = \text{msgS1}' = \perp) \vee \\ (\text{msgS1}; \text{Urnd}) \neq \\ (\text{msgS1}'; \text{Urnd}') \\ \wedge \text{hq}(\text{pk}, m, \text{msgS1}; \text{Urnd}) \\ = \text{hq}(\text{pk}, m, \text{msgS1}'; \text{Urnd}') \end{array} \middle| \begin{array}{l} (m, \text{Urnd}, \text{Urnd}') \leftarrow_{\$} \\ \mathcal{A}(\text{pk}, \text{msgS1}, \text{msgS1}') \end{array} \right] \leq \varepsilon_{\text{UL}}.$$

Property 7 (Double Signature Property). We require that the blind signature scheme has (t, ε) -double signature resistance as defined below.

We define the double signature game for an adversary $\mathcal{A}$ as follows:

- **Setup.** The game samples a key pair $(\text{pk}, \text{sk}) \leftarrow_{\$} \text{BS}.\mathcal{G}$
- **Online Phase.** The adversary gets access to the following oracles:
 - **Signer Simulation 1** Runs $\mathcal{SIM}_{\mathcal{S}_1}(\text{pk})$ to produce $\text{msgS1}, \text{simSt}$ and outputs msgS1 . Saves $(\text{msgS1}, \text{simSt})$.
 - **Random Oracle Simulation** takes as input $\text{msgS1}, \text{Urnd}, m$. Retrieves simSt corresponding to msgS1 (aborts if Signer Simulation 1 has never output msgS1 before), and runs $c := \mathcal{SIM}_{\mathbf{H}}(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$. Returns c .

- **Signer Simulation 2** Takes as input $\text{msgS1}, \text{Urnd}, m$. Retrieves simSt and c corresponding to these values if Signer Simulation 1 and Random Oracle Simulation have been called before. Runs $\text{SIM}_{\mathcal{S}_2}$ to output a response.
- **Output Determination:** The adversary outputs two signatures σ_1 and σ_2 on the same message m . It wins the game if $\mathcal{V}^H(\text{pk}, m, \sigma_1) = \mathcal{V}^H(\text{pk}, m, \sigma_2) = 1$ and the singular hash query that is made during verification of both signatures is the same.

The advantage of an adversary in this double signature game is the probability that the game outputs 1. We say the scheme has $(t_{\text{DS}}, \varepsilon_{\text{DS}})$ -double-signature-resistance if for any adversary that runs in time t_{DS} , the advantage is at most ε_{DS} .

Many existing blind signature schemes fit our definition 4 and satisfy the properties we defined above, if we just apply a slight modification.

- Three-move blind signature schemes such as Abe-Okamoto [2], Blind Schnorr [49], Abe [1], Tessaro and Zhu [51], and $\text{BS}_{[f]}$ [23]. The modifications required for these schemes is to sample the user’s randomness outside the user’s algorithm and to pass it as input to $\mathcal{U}_1$.
- Two-move blind signature schemes such as blind BLS [9] and blind RSA [19]. To adapt our definition to these schemes, we can define $\mathcal{S}_1$ and $\text{SIM}_{\mathcal{S}_1}$ to output an empty message $\perp$.

We give concrete examples showing how these schemes fit our definition in appendix D (Blind Schnorr) and appendix E (Abe-Okamoto). The rest of the schemes fit analogously.

We use the standard definitions for correctness and blindness of blind signature schemes and recall them in appendix A.3 for completeness.

5 A Generic Compiler for Blind Signatures with Attributes

In this section, we present our compiler *Pachirisu* that transforms any three-move simulatable blind signature scheme BS into a blind signature with attributes scheme $\text{P}[\text{BS}]$. *Pachirisu* utilizes the cut-and-choose technique [41,15] to sign a commitment of the user’s attribute using the underlying scheme BS . The user can later present the signature with an opening for the signed commitment, allowing them to present their attributes selectively alongside the signature.

In contrast to [41,15] which run K instances of the signing protocol concurrently, *Pachirisu* performs nested signing. While concurrent execution is more efficient, it falls short of achieving Strong One-More Unforgeability due to mix and match attacks. We resolve this by introducing dependencies between the different sub-signatures in *Pachirisu* and therefore achieve Strong One-More Unforgeability. For a set of keys $\{\text{sk}_1, \dots, \text{sk}_k\}$, *Pachirisu* runs the cut-and-choose

protocol sequentially starting from key $\mathbf{sk}_1$. After each interaction, it uses the message-subsignature pair of the previous interaction (under key $\mathbf{sk}_{i-1}$) as the message to be signed in the current interaction (under key $\mathbf{sk}_i$).

Each $\mathbf{P}[\mathbf{BS}]$ signing session starts with an initialization round. The overview of such a session is depicted in fig. 1. In this round, the signer sends the current cut-and-choose parameter N and commits to its cut-and-choose indices.

This is followed by executing the following subroutine described in fig. 2 K times, one time for each of the underlying keys.

For each round, an underlying message m_i is generated which will be kept secret from the signer. In the first round, this message m_1 is just a hash of the actual message M to be signed. In all subsequent rounds, the message m_i is a hash of the previous message m_{i-1} along with components of the previous sub-signature. This link between the sub-signatures for the different keys ensures strong one-more unforgeability as it prevents a malicious user from re-arranging the sub-signatures to generate multiple signatures on the same message.

In the i -th run of the subroutine, the signer and user essentially run N subsessions of the signing protocol of $\mathbf{BS}$ with key pair $(\mathbf{sk}_i, \mathbf{pk}_i)$ in parallel, but the user is expected to show that it behaved honestly by revealing its random coins for all but one subsession before the signer closes it.

To achieve this, the user is first required to commit to the randomness $\text{Urnd}_{i,j}$ for $j \in [N]$ it will use in each subsession and the signer computes and sends N first signer messages msgS1 for the underlying scheme after receiving the user's randomness commitments $\text{com}_{i,j}$.

Upon receiving the signer's messages, the user generates the messages $\mathcal{Y}_{i,j}$ that will be signed by the underlying scheme. To preserve blindness in the cut-and-choose process, the signed messages $\mathcal{Y}_{i,j}$ contain commitments on the message m_i with fresh Merkle-Tree commitments $\zeta_{i,j}$ to all of the user's attributes for each of the cut-and-choose subsessions.

The user runs the user algorithm $\mathbf{BS.U1}$ for $\mathbf{pk}_i$ with the message $\mathcal{Y}_{i,j}$, the first signer message $\text{msgS1}_{i,j}$, and the user randomness $\text{Urnd}_{i,j}$ it had previously committed to, to generate the user messages $\text{msgU}_{i,j}$.

After the user sends the user messages $\text{msgU}_{i,j}$ for all cut-and-choose subsessions, the signer opens the commitment to the current subsession's cut-and-choose index J_i .

The user now has to open the user randomness commitments and also provide the openings for the Merkle-Trees used for all subsessions, except the J_i -th, to show that they behaved honestly in the signing sessions and that the correct user attributes were committed to.

Only if the commitments were opened correctly, the signer sends the final signer message msgS2 for the index J_i , from which the user derives the sub-signature σ_i . The i -th round of the subroutine outputs the public information σ_i , the commitment randomness η_{i,J_i} for the commitment to m_i , and the Merkle-Tree commitment ζ_{i,J_i} . The user keeps the private state $\mathbf{R}_{i,J_i}$ which is the randomness used to generate ζ_{i,J_i} to use later in the showing procedure.

If the concluded subsession is not the final subsession $i = K$, then the message m_i along with $(\sigma_i, \eta_i, \zeta_i)$ are hashed together as explained earlier to generate the message m_{i+1} to be signed by the next subsession $i + 1$.

After this subroutine has run for all K key pairs, the user assembles the final signature out of the public components generated in all K subsessions.

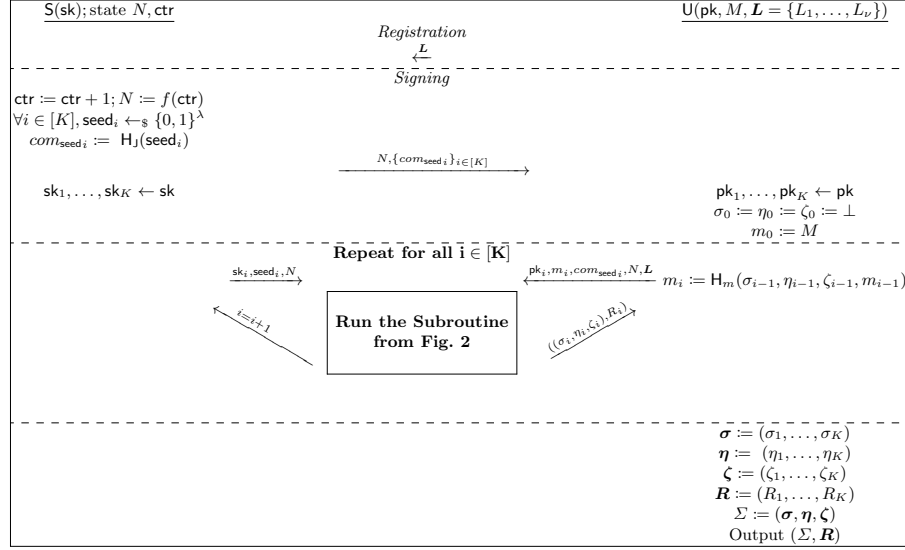


Fig. 1: The transform performs nested signing by applying the signing subroutine from 2 under keys $i = 1, \dots, K$, such that the message in the i -th signing step is a hash of the signature and message from the previous signing step $i - 1$.

Theorem 1 (ℓ -OMUF). *Let $H, H_T : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_{rnd} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be hash functions treated as random oracles. Let BS^H be an arbitrary three-move simulatable blind signature scheme with the properties described in Definition 4, and let $\mathbf{P}[BS^H]$ be the scheme resulting from applying our transform on BS^H . Assume there exists $\vartheta > 0$ and $f : \mathbb{N} \rightarrow \mathbb{N}$ with*

$$f(\text{ctr}) = \lceil 3 \cdot \vartheta \cdot \ln(\ell + 1) \cdot \text{ctr} \rceil.$$

Then for all adversaries $\mathcal{A}$ making at most $Q_H, Q_{rnd}, Q_T, Q_{mt}$ queries to the oracles H, H_{rnd}, H_T and H_{MT} , respectively, and Q_{S_2} and Q_{S_3} queries to $\mathbf{P}[BS^H].S_2$ and $\mathbf{P}[BS^H].S_3$, respectively, and for all public parameters pp with a security parameter λ , and all non-negative integers ℓ , there exists $\delta > 0$ with $(1 - \delta) \cdot \vartheta > 1$, and a reduction $\mathcal{R}_1$ such that if BS is (t, ε) -0-OMUF, then $\mathbf{P}[BS]$ is (t', ε') - ℓ -OMUF with $t' \approx t$ and

$$\varepsilon' \leq K \cdot (Q_H - \ell) \cdot \varepsilon + \epsilon_{cheat} + \epsilon_{ro} + \epsilon_{bs}.$$

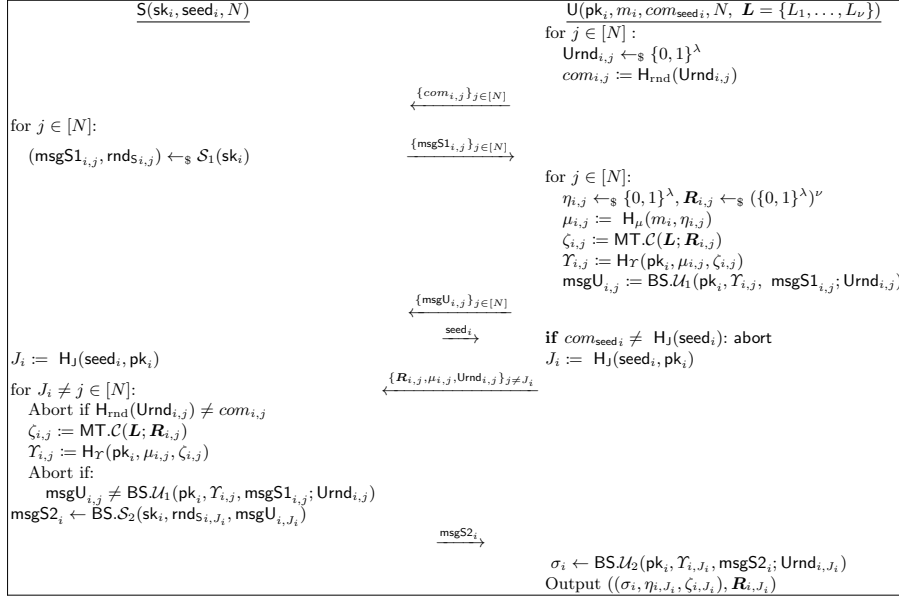


Fig. 2: This is the subroutine used in Figure 1, which shows the execution of the i -th subsession between the signer (the owner of pk_i) and the user. MT is the commitment scheme described in Section 3.1, which relies internally on a hash function H_{MT} for hashing. $\text{H}, \text{H}_{\text{rnd}}, \text{H}_{\text{MT}}, \text{H}_J, \text{H}_{\gamma}, \text{H}_{\mu} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ are random oracles. $f : \mathbb{N} \rightarrow \mathbb{N}$ is an arbitrary function that determines how fast the value N increases.

where:

$$\begin{aligned}
\epsilon_{\text{cheat}} &:= \frac{K \cdot \ell}{Q_{\text{H}}} + \exp(-\delta \cdot K), \\
\epsilon_{\text{ro}} &:= \frac{Q_{\gamma}^2 + Q_{\text{rnd}}^2 + Q_{\text{mt}}^2}{2^{\lambda+1}} + \frac{N \cdot Q_{\mathcal{S}_3} \cdot Q_{\gamma}}{2^{\lambda}} + \frac{N \cdot Q_{\mathcal{S}_2} \cdot Q_{\text{rnd}}}{2^{\lambda}}, \\
\epsilon_{\text{bs}} &:= \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \ell \cdot K \cdot (N - 1) \cdot (\varepsilon_{\text{UUF}} + \varepsilon_{\text{UO}}) \\
&\quad + K \cdot \varepsilon_{\text{UL}} + K \cdot (Q_{\text{H}} - \ell) \cdot Q_{\text{H}} \cdot \varepsilon_{\text{DS}}.
\end{aligned}$$

Proof Overview. We prove this lemma by providing a reduction $\mathcal{R}_1$ that, given an adversary $\mathcal{A}$ winning Game $\ell\text{-OMUF}_{\mathbf{P}[\text{BS}]}^{\mathcal{A}}$ for any BS scheme following the definition in Section 4 and any positive integer ℓ , it wins Game $0\text{-OMUF}_{\text{BS}}^{\mathcal{R}_1}$. $\mathcal{R}_1$ runs Game $0\text{-OMUF}_{\text{BS}}^{\mathcal{R}_1}$ with a challenger and aims to produce a forgery $(\mathcal{T}, \sigma)$ that is valid under the challenger's public key pk and its random oracle H_{ch} , i.e., $\text{BS}.\mathcal{V}^{\text{H}_{\text{ch}}}(\text{pk}, \mathcal{T}, \sigma) = 1$.

The reduction $\mathcal{R}_1$ embeds the challenger's public key pk as one of the K public keys and generates the remaining public keys honestly. Crucially, since $\mathcal{R}_1$ is running Game 0-OMUF with the challenger, it may not make any signing

$\mathcal{V}(\text{pk}, M, \Sigma)$	$\mathcal{SH}_U(\text{pk}, \text{rst}_U, \text{ost}_U, M, \Sigma, I_{op})$
1 : $(\sigma, \eta, \zeta) := \Sigma$	1 : $(\mathbf{R}_1, \dots, \mathbf{R}_K) := \text{ost}_U.\mathbf{R}$
2 : $(\sigma_1, \dots, \sigma_K) := \sigma$	2 : for $i \in [K]$:
3 : $(\eta_1, \dots, \eta_K) := \eta$	3 : $(\mathbf{L}_i^*, \mathbf{R}_i^*) := \text{MT}.\mathcal{O}(\text{rst}_U.C, \text{ost}_U.\mathbf{L}, \mathbf{R}_i, I_{op})$
4 : $(\zeta_1, \dots, \zeta_K) := \zeta$	4 : $\mathbf{L}^* := (\mathbf{L}_1^*, \dots, \mathbf{L}_K^*)$
5 : $(\text{pk}_1, \dots, \text{pk}_K) := \text{pk}$	5 : $\pi := (\mathbf{R}_1^*, \dots, \mathbf{R}_K^*)$
6 : $\sigma_0 := \eta_0 := \zeta_0 := \perp$	6 : return $(\mathbf{L}^*, \pi)$
7 : $m_0 := M$	$\mathcal{SH}_V(\text{pk}, M, \Sigma, \mathbf{L}^*, I_{op}, \pi)$
8 : for $i \in [K]$:	1 : $(\sigma, \eta, \zeta) := \Sigma$
9 : $m_i := \text{H}_m(\sigma_{i-1}, \eta_{i-1}, \zeta_{i-1}, m_{i-1})$	2 : $(\zeta_1, \dots, \zeta_K) := \zeta$
10 : $\mu_i := \text{H}_\mu(m_i, \eta_i)$	3 : $(\mathbf{L}_1^*, \dots, \mathbf{L}_K^*) := \mathbf{L}^*$
11 : $\gamma_i := \text{H}_T(\text{pk}_i, \mu_i, \zeta_i)$	4 : $(\mathbf{R}_1^*, \dots, \mathbf{R}_K^*) := \pi$
12 : if $\text{BS}.\mathcal{V}(\text{pk}_i, \gamma_i, \sigma_i) = 0$	5 : for $i \in [K]$:
13 : return 0	6 : if $\text{MT}.\mathcal{CV}(\zeta_i, (\mathbf{L}_i^*, \mathbf{R}_i^*), I_{op}) = 0$: return 0
14 : return 1	7 : if $\text{P}[\text{BS}].\mathcal{V}(\text{pk}, M, \Sigma) = 0$: return 0
	8 : return 1

Fig. 3: For a three-move simulatable blind signature scheme BS, this figure shows the signature verification procedure $\text{P}[\text{BS}].\mathcal{V}$, the showing generation algorithm $\text{P}[\text{BS}].\mathcal{SH}_U$, and the showing verification algorithm $\text{P}[\text{BS}].\mathcal{SH}_V$ of the blind signature with attributes scheme $\text{P}[\text{BS}]$ resulting from applying our transform Pachirisu on BS.

query, while $\mathcal{R}_1$ must answer $\mathcal{A}$'s signing queries ($\mathcal{A}$ may complete up to ℓ signing sessions with $\mathcal{R}_1$), including those queries targeting the challenger's key pk without knowing the corresponding secret key.

To answer signing queries by the adversary $\mathcal{A}$, $\mathcal{R}_1$ uses the secret keys of the keys that it generated itself. To answer the parts of signing queries corresponding to the challenger's key, the reduction uses the random oracle and signing simulation $\text{BS}.\mathcal{SIM}$ of the underlying scheme. To use these algorithms, it needs to extract the messages and user randomness used to generate the random oracle queries to H , as well as the user messages msg_U . The reduction achieves this by observing the random oracle queries that the adversary makes when computing the commitments $\text{com}_{i,j}$. This works as long as $\mathcal{A}$ behaves somewhat honestly, i.e. for the challenger's public key, the adversary always makes the corresponding random oracle queries to compute its commitments. The case that would be the most problematic for the reduction is when the adversary "cheats" without getting caught, that is, it has not queried the random oracle to produce the commitments at one of the cut-and-choose indices, and this index is exactly the one that is used to produce the signature later on. Using the observation from [15], we use the Chernoff bound to show that it is likely that there is at least one public key where $\mathcal{A}$ does not cheat. If the reduction correctly guesses this public key, it can then use the hash queries to extract all the information it needs for simulating hash and signing responses.

Once $\mathcal{A}$ terminates and outputs $\ell + 1$ forgeries for $\mathbf{P}[\text{BS}]$, $\mathcal{R}_1$ can extract a forgery for BS from $\mathcal{A}$'s output. This is possible since a $\mathbf{P}[\text{BS}]$ signature is in fact a nested BS signature, i.e., a forgery (M, Σ) for a public key $\mathbf{pk} := \{\mathbf{pk}_1, \dots, \mathbf{pk}_K\}$ is computed as

$$\Sigma := \text{BS.S}(\text{sk}_k, H_m(\text{BS.S}(\text{sk}_{k-1}, H_m(\dots \text{BS.S}(\text{sk}_1, M))))),$$

where BS.S denotes the execution of the interactive signing protocol of BS between the signer and the user. By choosing $\mathbf{pk}$ such that the challenger's key $\mathbf{pk} \in \mathbf{pk}$, $\mathcal{R}_1$ effectively receives a valid message-subsignature pair (γ, σ) for $\text{BS.V}(\mathbf{pk}, \gamma, \sigma)$ if $\mathcal{A}$ outputs a valid $\mathbf{P}[\text{BS}]$ signature that is valid under $\mathbf{pk}$. However, such a forgery will be accepted by the challenger only if the forgery is based on a hash query generated by the challenger's oracle (we call this *linking*); therefore, $\mathcal{R}_1$ chooses a random oracle query index $\kappa \in [Q_H]$ and answers the κ -th query by forwarding it to the challenger's oracle H_{ch} . In a *lucky* simulation, $\mathcal{A}$ does not make a signing query linked to this query, while one of the forgeries is linked to it, effectively providing $\mathcal{R}_1$ with a winning forgery.

We proceed via a sequence of games to rule out scenarios in which the adversary cheats by e.g. finding a preimage of a random oracle or guessing the cut-and-choose index J_i for all keys $i \in [K]$, reaching a game where the adversary behaves honestly and $\mathcal{R}_1$ can efficiently extract the user's random coins and the signed messages, which allows $\mathcal{R}_1$ to simulate BSsigning queries using BS.SIM .

The full proof is deferred to appendix C.1.

6 Stronger Security Definitions

6.1 Selective Secure Showing

The secure showing property as defined in [37], does not cover all possible attacks that one might consider. In particular, the adversary is required to always reveal all attributes. However, as we discussed in the introduction, in the real world, users typically do not reveal all their attributes but instead they selectively disclose only those necessary to access a specific service. Thus, it is imperative that the security notion also captures attacks where adversaries might attempt to alter the attributes contained in their signatures before revealing them.

We model this criterion through the existence of an injective mapping from showed signatures to signing sessions, i.e. each shown partial vector is explained by one signing session's committed attribute vector, and no signing session is being used to "explain" two or more opened signatures. The adversary wins if no such mapping exists.

One concern raised in [37] was that the winning criterion might be inefficient to check because the challenger would have to check that there exists no mapping from the "showed" partial vectors to the full vectors that the adversary requested signatures for. In particular, we cannot trust the adversary to output

a “witness” for winning the game (i.e. a bad mapping that leaves at least one vector unmapped), as a better mapping might exist.

We resolve this through the observation that this mapping can be expressed as a matching in a bipartite graph and describe a graph-based approach for efficiently checking the winning condition below.

Definition 5 (Secure Showing). For a blind signature with attributes $\text{BSA} = (\mathcal{G}, \mathcal{RG}, \mathcal{U}, \mathcal{S}, \mathcal{V}, \mathcal{SH})$, a positive integer $\ell \in \mathbb{Z}^+$, and an adversary $\mathcal{A}$, we define the game $\ell\text{-SH}$ (with unique messages/with arbitrary messages) (requiring full opening/with selective reveal) as follows.

1. Generate key pair $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}(\text{pp})$.
2. Invoke $\mathcal{A}^{\mathcal{RG}_S(\cdot), \mathcal{S}(\text{sk}, \cdot)}$. The adversary $\mathcal{A}$ may query its oracles arbitrarily and in an interleaved fashion as long as it completes at most ℓ sessions with $\mathcal{S}$. For $i \in [\ell]$, let $\text{rst}_{\mathcal{U}}^i.C$ denote the registered commitment corresponding to the i th session with $\mathcal{S}(\text{sk}, \cdot)$. We define $\text{rst}_{\mathcal{U}}^i.C = \perp$ if there is no session i (this occurs if $\mathcal{A}$ makes fewer than i signing queries).
3. $\mathcal{A}$ outputs up to ℓ tuples of the form $(m_i, \sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \pi_i)$ as well as ℓ pairs $(\mathbf{L}'_i, \text{rst}_{\mathcal{U}}^i.C)$.
4. Output 1 if all of the following hold and 0 otherwise:
 - a. For all $i \in [\ell]$, either $\text{rst}_{\mathcal{U}}^i.C$ contains a valid opening of $\text{rst}_{\mathcal{U}}^i.C$ to $\mathbf{L}'_i$ or $\text{rst}_{\mathcal{U}}^i.C = \perp$ (recall that $\text{rst}_{\mathcal{U}}^i.C = \perp$ when the adversary chooses to open less than i signing sessions).
 - b. For all $(m_i, \sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \pi_i)$ in $\mathcal{A}$'s output, it holds that $\mathcal{SH}_{\mathcal{V}}(\text{pk}, m_i, \sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \pi_i) = 1$ and $\mathcal{V}(\text{pk}, m_i, \sigma_i) = 1$. Oracles $\mathcal{RG}(\cdot)$ and $\mathcal{S}(\text{sk}, \cdot)$ share state and we require that session i with $\mathcal{S}(\text{sk}, \cdot)$ uniquely identifies some prior session j with $\mathcal{RG}(\cdot)$.
 - c. The tuples $(m_1, \sigma_1), \dots, (m_{\ell}, \sigma_{\ell})$ are pairwise-distinct.
 - d. One of the following mapping requirements hold:
 - i. **EITHER Requiring Full Opening:**
 - A. For all $(m_i, \sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \text{ost}_{\mathcal{U}}^i, \pi_i)$ in $\mathcal{A}$'s output, it holds that $\text{ost}_{\mathcal{U}}^i$ contains a valid opening of $\sigma_i.C$ to $\mathbf{L}_i$.
 - B. All $\mathbf{L}_i, \mathbf{L}'_i$ contain an entry for each attribute, i.e., they do not contain the $\perp$ symbol anywhere and all $I_{\text{op}, i} = [\nu]$, i.e. all attributes are opened.
 - C. The multiset of all attribute vectors $\{\mathbf{L}_i\} \not\subseteq \{\mathbf{L}'_i : \text{rst}_{\mathcal{U}}^i.C \neq \perp\}$.
 - ii. **OR With Selective Showing:** There exists no injective mapping

$$\mu : \{\mathbf{L}_i\}_{i=1}^{\ell} \rightarrow \{\mathbf{L}'_i\}_{i=1}^{\ell}$$

such that for all i , it holds that $\mathbf{L}_i \subseteq \mu(\mathbf{L}_i)$.

We define $\mathcal{A}$'s advantage in winning the game $\ell\text{-SH}$ against a blind signature scheme with attributes BSA as

$$\text{Adv}_{\text{BSA}}^{\ell\text{-SH}}(\mathcal{A}) := \Pr[\ell\text{-SH}_{\text{BSA}}^{\mathcal{A}} = 1].$$

We say that BSA is (t, ε, ℓ) -SH secure if for all adversaries that run in time at most t , $\text{Adv}_{\text{BSA}}^{\ell\text{-SH}}(\mathcal{A}) \leq \varepsilon$.

We give a brief discussion of our new security notion.

On the Efficiency of the Game. It is possible to check the winning condition of the above game in time $O(\ell^{2.5})$ by using the Hopcroft-Karp[35] algorithm to find a maximum cardinality matching. To apply the algorithm, we briefly explain how to view the sets of vectors $\{\mathbf{L}_i\}_{i=1}^\ell$ and $\{\mathbf{L}'_i\}_{i=1}^\ell$ as a graph: Each vector from the multiset $\{\mathbf{L}_i\}_{i=1}^\ell$ corresponds to a vertex v_i and each vector from the multiset $\{\mathbf{L}'_i\}_{i=1}^\ell$ corresponds to a vertex w_i in the graph, There exists an edge between v_i and w_j if and only if $\mathbf{L}_i \subset \mathbf{L}'_j$.

It is easy to see that this graph is bipartite as there are only edges between v_i 's and w_j 's but not within the set of the v_i 's or within the set of the w_j 's.

The running time of the Hopcroft-Karp algorithm is $n^{2.5}$ where n is the number of vertices - in our case the number of vertices is 2ℓ , thus the running time of the challenger lies in $O(\ell^{2.5})$.

On the Relation of Definition 5 and that of [37]. The definition from [37] is achieved by choosing the criteria item 4.c and item 4.d.i above.

It is easy to see that exchanging item 4.d.i by item 4.d.ii, one gets a strictly stronger definition. This is because item 4.d.i.C directly implies item 4.d.ii for the case that all attributes are opened (which is ensured by item 4.d.i.B). The checks in items 4.d.i.A and 4.d.i.B fall away in item 4.d.ii.

On the Relation to Non-Transferability and Unforgeability. Our Strong Secure Showing property might resemble the notions of *non-transferability* of single use (attribute-less) tokens [26], and *unforgeability* defined for multi-use ABCs [18,32]. Under the unforgeability definition, the adversary would win if it presented a token w.r.t. to a partial vector that is not a subvector of a vector it previously obtained a signature for, and under non-transferability, the adversary would win if it presented a token that was not issued to it (where users are identified through key pairs). Under Definition 5, the adversary would also win if it presents partial vectors that are subvectors of vectors it requested before but with the wrong multiplicity. The unforgeability notion for multi-use ABCs does not capture this scenario as it allows the user to show the same token several times.

6.2 Blindness with Partial Attributes

In previous blindness definitions, only signature verification $\mathcal{V}$ is taken into account which does not give any privacy or untraceability guarantees to the user once they show their signature and reveal a subset of their attributes using $\mathcal{SH}_U$. Thus, in theory, a blind signature scheme that reveals the user's randomness during $\mathcal{SH}_U$ and thus makes the signatures linkable to signing sessions, could fulfill blindness. We address this issue with the following blindness definition which allows the malicious signer to specify also users' attribute vectors and to ask for openings of the signatures in case both signing sessions succeed, on a subset of attributes where the specified attribute vectors coincide. This new blindness notion guarantees to users that they are anonymous within a set of users that

share the same attributes at the coordinates where they opened their attribute commitment.

Definition 6 (Blindness with Partial Showing). For a blind signature with attributes $\text{BSA} = (\mathcal{G}, \mathcal{RG}, \mathcal{U}, \mathcal{S}, \mathcal{V}, \mathcal{SH})$ and an adversary $\mathcal{A}$ we define the game **BLIND** as follows:

1. Invoke the first signer stage $\mathcal{A}$ that outputs a public key pk , two messages m_0, m_1 , and two attribute vectors $\mathbf{L}_0, \mathbf{L}_1$.
2. Sample a bit $b \leftarrow \{0, 1\}$ uniformly at random.
3. Invoke $\mathcal{A}^{\mathcal{RG}_U(\mathbf{L}_0), \mathcal{RG}_U(\mathbf{L}_1), \mathcal{U}^0(\text{pk}, m_b, \text{rst}_U^0), \mathcal{U}^1(\text{pk}, m_{1-b}, \text{rst}_U^1)}$, where $\mathcal{A}$ may query its oracles in an interleaved fashion as long as it queries each oracle once. The oracles behave as follows:
 - a. $\mathcal{RG}_U(\mathbf{L}_0), \mathcal{RG}_U(\mathbf{L}_1)$ register the attribute vectors and locally save rst_U^0 (corresponding to $\mathbf{L}_0$) and rst_U^1 (corresponding to $\mathbf{L}_1$).
 - b. $\mathcal{U}^0(\text{pk}, m_b, \text{rst}_U^0), \mathcal{U}^1(\text{pk}, m_{1-b}, \text{rst}_U^1)$: If the corresponding registration has been called, the oracles run the user-side of the signing protocol. $\mathcal{U}^0$ internally uses the registration state rst_U^0 corresponding to $\mathbf{L}_0$, and $\mathcal{U}^1$ internally uses registration state rst_U^1 corresponding to $\mathbf{L}_1$. When a user algorithm terminates, the game locally stores the tuple (m_b, σ_b) resp. (m_{1-b}, σ_{1-b}) as well as the opening states $\text{ost}_U^0, \text{ost}_U^1$.
4. Once both $\mathcal{U}^0$ and $\mathcal{U}^1$ have terminated, if both sessions resulted in a signature, output the two message-signature pairs (m_0, σ_0) and (m_1, σ_1) to the adversary. Otherwise, output $(\perp, \perp)$ to the adversary.
5. $\mathcal{A}$ may output two index sets I_0 and I_1 . If for all $i \in I_0 \cup I_1$ it holds that $L_{0,i} = L_{1,i}$, and the adversary was given two message-signature pairs in the previous phase, run $(\mathbf{L}'_0, \pi_0) \leftarrow_{\mathcal{S}} \mathcal{SH}_U(\text{pk}, \text{rst}_U^b, m_0, \sigma_0, \text{ost}_U^0, I_0)$ and $(\mathbf{L}'_1, \pi_1) \leftarrow_{\mathcal{S}} \mathcal{SH}_U(\text{pk}, \text{rst}_U^{1-b}, m_1, \sigma_1, \text{ost}_U^1, I_1)$, and output $(\mathbf{L}_0, \pi_0)$ and $(\mathbf{L}_1, \pi_1)$ to the adversary. Otherwise, output $(\perp, \perp)$ again to the adversary.
6. The adversary outputs a bit b' .

We define the advantage of $\mathcal{A}$ in the game **BLIND** as

$$\text{Adv}_{\text{BSA}}^{\text{BLIND}}(\mathcal{A}) := 2 \cdot \left| \Pr[b = b'] - \frac{1}{2} \right|.$$

We say that BSA is (t, ε) -blind if for all adversaries that run in time at most t , $\text{Adv}_{\text{BSA}}^{\text{BLIND}}(\mathcal{A}) \leq \varepsilon$.

It is easy to see that the new notion of blindness implies the old notions used in [4,37] as any adversary that wins in the old notions can be used to also win in this new notion by merely skipping the showing step (the reduction can request a showing of the empty attribute vector).

7 Proving Secure Showing for Our Transformation

Theorem 2. Let BS be a blind signature scheme satisfying the properties of definition 4. Let $\mathbf{P}[\text{BS}]$ be the scheme resulting from applying our transform on BS . If $\mathbf{P}[\text{BS}]$ is (t, ε) - ℓ -OMUF, then $\mathbf{P}[\text{BS}]$ is (t', ε') - ℓ -SH with $t' \approx t$ and

$$\varepsilon' \leq K \cdot \text{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_2) + \varepsilon.$$

Simplifying Notation. A message-signature pair (M, Σ) consists of K message-subsignature pairs $(\Upsilon_1, \sigma_1), \dots, (\Upsilon_K, \sigma_K)$. Similarly, each signing session ctr consists of K signing subsessions $(\text{ctr}, 1), \dots, (\text{ctr}, K)$. We say that (Υ_i, σ_i) is linked to a signing subsession (ctr, i) if there is a user message $\text{msgU}_{i,j} := \text{BS}.\mathcal{U}_1(\text{pk}_i, \Upsilon_{i,j}, \text{msgS1}_{i,j}; \text{Urnd}_{i,j})$, $\mathcal{A}$ makes an $\mathcal{S}_3$ query for the subsession (ctr, i) with $\text{msgU}_{i,j}$ in the input, and the hash query made by $\text{BS}.\mathcal{V}(\text{pk}_i, \Upsilon_i, \sigma_i)$ is the same one made by $\text{BS}.\mathcal{U}_1$ while generating $\text{msgU}_{i,j}$, i.e. $Q = \text{hq}(\text{pk}_i, \Upsilon_i, \text{msgS1}_{i,j}, \text{Urnd}_{i,j})$ (see Property 1).

Proof Overview. Recall that winning Game ℓ -SH requires the adversary $\mathcal{A}$ to output ℓ pairs of the form $(\text{rst}_{\mathcal{U}}^i, \mathbf{L}'_i)$ along with ℓ tuples of the form $(M_i, \Sigma_i, \mathbf{L}_i, I_{op,i}, \pi_i)$ after completing at most ℓ signing sessions with the signing oracle $\mathbf{P}[\text{BS}].\mathcal{S}$, while all the following conditions hold:

1. For all $i \in [\ell]$, either $\text{rst}_{\mathcal{U}}^i$ contains a valid opening of $\text{rst}_{\mathcal{U}}^i.C$ to $\mathbf{L}'_i$ or $\text{rst}_{\mathcal{U}}^i.C = \perp$ (recall that $\text{rst}_{\mathcal{U}}^i.C = \perp$ when the adversary chooses to open less than i signing sessions).
2. For all $(M_i, \Sigma_i, \mathbf{L}_i, I_{op,i}, \pi_i)$ in $\mathcal{A}$'s output, it holds that $\mathcal{SHV}(\text{pk}, M_i, \sigma_i, \mathbf{L}_i, I_{op,i}, \pi_i) = 1$ and $\mathcal{V}(\text{pk}, M_i, \sigma_i) = 1$.
3. The tuples $(M_1, \Sigma_1), \dots, (M_\ell, \Sigma_\ell)$ are pairwise-distinct.
4. There exists no injective mapping

$$\mu : \{\mathbf{L}_i\}_{i=1}^\ell \rightarrow \{\mathbf{L}'_i\}_{i=1}^\ell$$

such that for all i , it holds that $\mathbf{L}_i \subseteq \mu(\mathbf{L}_i)$.

Essentially, this means that $\mathcal{A}$ managed to output a valid message-signature pair (M_i, Σ_i) for a possibly partial vector of attributes $\mathbf{L}_i$ with a proof π_i proving knowledge of missing attributes from $\mathbf{L}_i$, and $\mathbf{L}_i$ does not match any vector $\mathbf{L}'_j$ used in some signing session j , denoted $\mathbf{L}_i \not\subseteq \mathbf{L}'_j$ for all $j \in [\ell]$. We assume without loss of generality there is a single tuple $(M_{\text{ctr}^*}, \Sigma_{\text{ctr}^*}, \mathbf{L}_{\text{ctr}^*}, I_{op,\text{ctr}^*}, \pi_{\text{ctr}^*})$ for $\text{ctr}^* \in [\ell]$ with $\mathbf{L}_{\text{ctr}^*}$ mismatching all registered (and used during signing) attribute vectors $\mathbf{L}'_j$. We distinguish two cases regarding the message-signature pair $(M_{\text{ctr}^*}, \Sigma_{\text{ctr}^*})$ of the mismatching tuple $(M_{\text{ctr}^*}, \Sigma_{\text{ctr}^*}, \mathbf{L}_{\text{ctr}^*}, I_{op,\text{ctr}^*}, \pi_{\text{ctr}^*})$.

- Every subsignature $(\Upsilon_i^{(\text{ctr}^*)}, \sigma_i^{(\text{ctr}^*)})$ for $i \in [K]$ is linked to some signing subsession. We rule out this case via a reduction to the binding property of the underlying commitment scheme since this would require $\mathcal{A}$ to know one commitment with two openings. While the first opening is directly received from the adversary in the mismatching tuple, i.e. $(\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*})$, the second must be extracted from the random oracles hash tables. If the signature is linked to a signing session $\text{ctr}^{**} \in [\ell]$, the second opening consists of the attribute vector $\mathbf{L}'_{\text{ctr}^{**}}$ used in that session, and the randomness $\mathbf{R}_{\text{ctr}^{**}, J_i}$ $\mathcal{A}$ used to commit to the vector $\mathbf{L}'_{\text{ctr}^{**}}$ for the subsession (ctr^{**}, i) in the interaction corresponding to the cut-and-choose index J_i .
- The other case is the existence of a subsignature that is not linked to any signing subsession. This means, however, that the adversary outputs $\ell \cdot K$

message-subsignature pairs, while using less than $\ell \cdot K$ signing sessions with the signer. Using the same strategy we used in OMUF, we reduce this case to the 0-OMUF security of the underlying scheme BS. (As a matter of fact, we use the same reduction from the proof of theorem 1).

Proof. Assume there is an adversary $\mathcal{A}$ winning Game ℓ -SH by outputting ℓ pairs $(\text{rst}_U^{\text{ctr}}, \mathbf{L}'_{\text{ctr}})$ along with ℓ tuples $(M_{\text{ctr}}, \Sigma_{\text{ctr}}, \mathbf{L}_{\text{ctr}}, I_{\text{op}, \text{ctr}}, \pi_{\text{ctr}})$ for $\text{ctr} \in [\ell]$ satisfying the constraints defined in Game ℓ -SH.

Recall that in our transform, the user does not send a commitment of its attributes to the signer in the registration phase, instead it sends the attribute vector in plain $\mathbf{L}$. Therefore, the winning condition regarding the pair $(\text{rst}_U^{\text{ctr}}, \mathbf{L}'_{\text{ctr}})$ requires $\mathcal{A}$ to only output the vectors $\mathbf{L}'_{\text{ctr}}$ if $\mathcal{A}$ opens the ctr -th signing session.

Akin to the one-more unforgeability proof, we want to show that the adversary cannot cheat for all keys pk_i for $i \in [K]$. That is, there must exist some public key pk_i , for which $\mathcal{A}$ does not cheat at all. We note that the simulation of both games ℓ -OMUF and ℓ -SH is identical, since the simulator must simulate at most ℓ signing sessions for $\mathcal{A}$, in addition to the signer registration oracle and the random oracles. Thus, we use the same game hops we used in the OMUF proof to rule out the same bad events ruled out in the OMUF proof (appendix C.1). Since most of these games rule out basic bad events (such as finding hash collisions or computing hash preimages), we defer these game hops here and provide them in appendix C.2. We end up in $\mathbf{G}_9$.

Per this game, we know there is a uniformly chosen $\iota \in [K]$ such that $\mathcal{A}$ does not cheat for pk_ι . Each signature-message pair (M_j, Σ_j) $\mathcal{A}$ outputs consists of K message-subsignature pairs (γ_i, σ_i) for $i \in [K]$. Additionally, recall that $\mathcal{A}$ closes at most ℓ signing sessions and each session consists of K subsessions, a subsession per key pk_i . Now, we want to rule out that all subsignatures corresponding to the ι -th key pk_ι are linked to some signing subsession corresponding to that key. That is, we want a game ensuring that there exists a message-subsignature pair $(\gamma_\iota^{(\text{ctr}^*)}, \sigma_\iota^{(\text{ctr}^*)})$ that is not linked to any signing session (ctr, ι) .

Game $\mathbf{G}_{10}$. This game is identical to $\mathbf{G}_9$, except that it aborts and outputs 0 if Event **BAD**₁ occurs, which we define as follows:

BAD₁: $\forall \text{ctr} \exists \text{ctr}'$ such that the message-subsignature pair $(\gamma_\iota^{(\text{ctr})}, \sigma_\iota^{(\text{ctr})})$ is linked to the signing subsession (ctr', ι) .

Claim 1.

$$\text{Adv}^{\mathbf{G}_{10}}(\mathcal{A}) \geq \text{Adv}^{\mathbf{G}_9}(\mathcal{A}) - \text{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_2)$$

Proof. Assume $\mathcal{A}$ provokes Event **BAD**₁, hence each message-subsignature pair is linked to some signing subsession. We provide a reduction $\mathcal{R}_2$ that uses $\mathcal{A}$ to break the binding property of the commitment scheme MT (lemma 7).

$\mathcal{R}_2$ simulates $\mathbf{G}_{10}$ for $\mathcal{A}$ perfectly. Once $\mathcal{A}$ terminates, it outputs $\mathbf{L}'_j$ and $(M_i, \Sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \pi_i)$ for all $i, j \in [\ell]$. Per our assumption, $\exists \text{ctr}^* \in [\ell] : \mathbf{L}_{\text{ctr}^*} \not\subseteq \mathbf{L}'_j$ for all $j \in [\ell]$.

Let (ctr^{**}, ι) for $\text{ctr}^{**} \in [\ell]$ be the signing subsession linked to $(\gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$, and $(\zeta_1^{(\text{ctr}^*)}, \dots, \zeta_K^{(\text{ctr}^*)}) := \zeta_{\text{ctr}^*}$. $\mathcal{R}_2$ gets the commitment and the first opening $(\zeta_{\iota}^{(\text{ctr}^*)}, (\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*}))$ directly from $\mathcal{A}$'s output. Now, $\mathcal{R}_2$ has to find another opening, which it can extract from the tables of the random oracles from the entries corresponding to the subsession (ctr^{**}, ι) .

Let $\text{msgU}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ be the user message corresponding to the index J_{ι} for the sub-session (ctr^{**}, ι) , i.e., $\text{msgU}_{\iota, J_{\iota}}^{(\text{ctr}^{**})} := \text{BS.U}_1(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}; \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$. Due to Games $\mathbf{G}_6$ and $\mathbf{G}_8$, $\mathcal{R}_2$ can extract the message $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ used to generate $\text{msgU}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$. Additionally, due to Game $\mathbf{G}_7$, $\mathcal{R}_2$ can look up the message $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ in the table of $\mathbf{H}_{\mathcal{R}}$ to extract the query $(\text{pk}_{\iota}, \mu_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ used to generate $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$. Similarly, $\mathcal{R}_2$ looks up the table of $\mathbf{H}_{\text{MT}}$ to reverse the tree root $\zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ level by level until it recovers the leaves $\mathbf{L}$ and $\mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$, which is possible due to Game $\mathbf{G}_2$. The tuple $(\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ constitutes the second opening for $\zeta_{\iota}^{(\text{ctr}^*)}$.

Finally, $\mathcal{A}$ outputs $(\zeta_{\iota}^{(\text{ctr}^*)}, (\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*})), (\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ as an attack against $\text{vcBINDING}_{\text{MT}}^{\mathcal{R}_2}$.

Next, we show that both openings are valid for $\zeta_{\iota}^{(\text{ctr}^*)}$. Obviously, $(\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*})$ is a valid opening since $\mathcal{SH}_{\mathcal{V}}(\text{pk}, M_i, \sigma_i, \mathbf{L}_i, I_{\text{op}, i}, \pi_i) = 1$ for $i = \text{ctr}^*$. (Recall that $\mathcal{SH}_{\mathcal{V}}$ (fig. 3) checks the opening validity of each $\zeta_i^{(\text{ctr}^*)}$).

The second opening $(\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ is valid because the commitment extracted from the table of $\mathbf{H}_{\mathcal{R}}$ is identical to the commitment in the signature, i.e. $\zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})} = \zeta_{\iota}^{(\text{ctr}^*)}$ since the subsignature (ctr^*, ι) and the signing sub-session (ctr^{**}, ι) are linked. To see this, we analyze the implication of the linking between the message-subsignature pair $(\gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$ and the signing sub-session (ctr^{**}, ι) . Recall that the linking between the subsignature and the sub-session implies that $\text{BS.V}(\text{pk}_{\iota}, \gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$ and $\text{BS.U}_1(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}; \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ make the same query $Q = \text{hq}(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$. However, this is only possible if the message being verified and the message passed to the user's algorithm BS.U_1 are identical, i.e. $\gamma_{\iota}^{(\text{ctr}^*)} = \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ since the message is part of the hash (see Game $\mathbf{G}_4$), unless $\mathcal{A}$ can cause a collision for $\mathbf{H}_{\mathcal{R}}$, which is ruled out in Game $\mathbf{G}_2$. Again, if the messages are identical, this means that the query (pk, η, ζ) sent to the random oracle $\mathbf{H}_{\mathcal{R}}$ to generate $\gamma_{\iota}^{(\text{ctr}^*)}$ and $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ is identical, unless $\mathcal{A}$ provokes a collision for $\mathbf{H}_{\mathcal{R}}$ (Game $\mathbf{G}_2$). Consequently, it holds that $\zeta_{\iota}^{(\text{ctr}^*)} = \zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$.

Finally, the openings $\mathcal{R}_2$ outputs are distinct since $\mathbf{L}_{\text{ctr}^*} \not\subseteq \mathbf{L}'_{\text{ctr}^{**}}$.

By this game, $\mathcal{A}$ outputs a message-subsignature pair that is not linked to any signing sub-session. This is equivalent to $\mathcal{A}$ outputting a BS signature without relying on the signing oracle. We reduce this to 0-OMUF using the same reduction from the proof of theorem 1. Note that all games from that proof are adopted for this proof, hence none of the abort conditions makes the reduction abort, which concludes the proof. Since the only difference compared to the proof of

theorem 1 is the additional game $\mathbf{G}_{10}$, we compute $\mathcal{A}$ ' advantage as

$$\mathbf{Adv}_{\mathbf{P}_{[\text{BS}]}}^{\ell\text{-SH}}(\mathcal{A}) \leq K \cdot \mathbf{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_2) + \mathbf{Adv}_{\mathbf{P}_{[\text{BS}]}}^{\ell\text{-OMUF}}(\mathcal{R}_1).$$

Acknowledgments

Foteini Baldimtsi was funded through NSF #2143287. Julia Kastner is supported by the Dutch Research Agenda (NWA) project HAPKIDO (Project No. NWA.1215.18.002), which is financed by the Dutch Research Council (NWO). Omar Renawi and Julian Loss were funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 507237585, and by the European Union, ERC-2023-STG, Project ID: 101116713. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

References

1. Abe, M.: A secure three-move blind signature scheme for polynomially many signatures. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 136–151. Springer, Berlin, Heidelberg (May 2001). https://doi.org/10.1007/3-540-44987-6_9
2. Abe, M., Okamoto, T.: Provably secure partially blind signatures. In: Bellare, M. (ed.) CRYPTO 2000. LNCS, vol. 1880, pp. 271–286. Springer, Berlin, Heidelberg (Aug 2000). https://doi.org/10.1007/3-540-44598-6_17
3. Apple: icloud private relay overview. https://www.apple.com/icloud/docs/iCloud_Private_Relay_Overview_Dec2021.pdf (2021), accessed: 2026-04-29
4. Baldimtsi, F., Lysyanskaya, A.: Anonymous credentials light. In: Sadeghi, A.R., Gligor, V.D., Yung, M. (eds.) ACM CCS 2013. pp. 1087–1098. ACM Press (Nov 2013). <https://doi.org/10.1145/2508859.2516687>
5. Belenkiy, M., Camenisch, J., Chase, M., Kohlweiss, M., Lysyanskaya, A., Shacham, H.: Randomizable proofs and delegatable anonymous credentials. In: Halevi, S. (ed.) CRYPTO 2009. LNCS, vol. 5677, pp. 108–125. Springer, Berlin, Heidelberg (Aug 2009). https://doi.org/10.1007/978-3-642-03356-8_7
6. Benhamouda, F., Lepoint, T., Orrù, M., Raykova, M.: Publicly verifiable anonymous tokens with private metadata bit. Cryptology ePrint Archive, Report 2022/004 (2022), <https://eprint.iacr.org/2022/004>
7. Bichsel, P., Camenisch, J., Groß, T., Shoup, V.: Anonymous credentials on a standard java card. In: Al-Shaer, E., Jha, S., Keromytis, A.D. (eds.) ACM CCS 2009. pp. 600–610. ACM Press (Nov 2009). <https://doi.org/10.1145/1653662.1653734>
8. Blömer, J., Bobolz, J., Diemert, D., Eidens, F.: Updatable anonymous credentials and applications to incentive systems. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 1671–1685. ACM Press (Nov 2019). <https://doi.org/10.1145/3319535.3354223>
9. Boldyreva, A.: Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In: Desmedt, Y. (ed.) PKC 2003. LNCS, vol. 2567, pp. 31–46. Springer, Berlin, Heidelberg (Jan 2003). https://doi.org/10.1007/3-540-36288-6_3

10. Boneh, D., Boyen, X., Shacham, H.: Short group signatures. In: Franklin, M. (ed.) CRYPTO 2004. LNCS, vol. 3152, pp. 41–55. Springer, Berlin, Heidelberg (Aug 2004). https://doi.org/10.1007/978-3-540-28628-8_3
11. Bootle, J., Lyubashevsky, V., Nguyen, N.K., Sorniotti, A.: A framework for practical anonymous credentials from lattices. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part II. LNCS, vol. 14082, pp. 384–417. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38545-2_13
12. Camenisch, J., Hohenberger, S., Kohlweiss, M., Lysyanskaya, A., Meyerovich, M.: How to win the clonewars: Efficient periodic n-times anonymous authentication. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 201–210. ACM Press (Oct / Nov 2006). <https://doi.org/10.1145/1180405.1180431>
13. Camenisch, J., Lysyanskaya, A.: An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 93–118. Springer, Berlin, Heidelberg (May 2001). https://doi.org/10.1007/3-540-44987-6_7
14. Camenisch, J., Lysyanskaya, A.: Signature schemes and anonymous credentials from bilinear maps. In: Franklin, M. (ed.) CRYPTO 2004. LNCS, vol. 3152, pp. 56–72. Springer, Berlin, Heidelberg (Aug 2004). https://doi.org/10.1007/978-3-540-28628-8_4
15. Chairattana-Apirom, R., Hanzlik, L., Loss, J., Lysyanskaya, A., Wagner, B.: Piccut-choo and friends: Compact blind signatures via parallel instance cut-and-choose and more. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part III. LNCS, vol. 13509, pp. 3–31. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15982-4_1
16. Chase, M., Durak, F.B., Vaudenay, S.: Anonymous tokens with stronger metadata bit hiding from algebraic MACs. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part II. LNCS, vol. 14082, pp. 418–449. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38545-2_14
17. Chase, M., Kohlweiss, M., Lysyanskaya, A., Meiklejohn, S.: Malleable signatures: New definitions and delegatable anonymous credentials. In: Datta, A., Fournet, C. (eds.) CSF 2014 Computer Security Foundations Symposium. pp. 199–213. IEEE Computer Society Press (2014). <https://doi.org/10.1109/CSF.2014.22>
18. Chase, M., Meiklejohn, S., Zaverucha, G.: Algebraic MACs and keyed-verification anonymous credentials. In: Ahn, G.J., Yung, M., Li, N. (eds.) ACM CCS 2014. pp. 1205–1216. ACM Press (Nov 2014). <https://doi.org/10.1145/2660267.2660328>
19. Chaum, D.: Blind signatures for untraceable payments. In: Chaum, D., Rivest, R.L., Sherman, A.T. (eds.) CRYPTO’82. pp. 199–203. Plenum Press, New York, USA (1982). https://doi.org/10.1007/978-1-4757-0602-4_18
20. Chaum, D.: Blind signature system. In: Chaum, D. (ed.) CRYPTO’83. p. 153. Plenum Press, New York, USA (1983). https://doi.org/10.1007/978-1-4684-4730-9_14
21. Cramer, R.: Modular Design of Secure yet Practical Cryptographic Protocols. Ph.D. thesis (Jan 1997)
22. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Desmedt, Y. (ed.) CRYPTO’94. LNCS, vol. 839, pp. 174–187. Springer, Berlin, Heidelberg (Aug 1994). https://doi.org/10.1007/3-540-48658-5_19
23. Crites, E.C., Komlo, C., Maller, M., Tessaro, S., Zhu, C.: Snowblind: A threshold blind signature in pairing-free groups. In: Handschuh, H., Lysyanskaya, A. (eds.)

- CRYPTO 2023, Part I. LNCS, vol. 14081, pp. 710–742. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38557-5_23
24. Crites, E.C., Lysyanskaya, A.: Delegatable anonymous credentials from mercurial signatures. In: Matsui, M. (ed.) CT-RSA 2019. LNCS, vol. 11405, pp. 535–555. Springer, Cham (Mar 2019). https://doi.org/10.1007/978-3-030-12612-4_27
 25. Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy pass: Bypassing internet challenges anonymously. PoPETs **2018**(3), 164–180 (Jul 2018). <https://doi.org/10.1515/popets-2018-0026>
 26. Durak, F.B., Marco, L., Talayhan, A., Vaudenay, S.: Non-transferable anonymous tokens by secret binding. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 2460–2474. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3670338>
 27. Facebook: Facebook private stats. <https://research.fb.com/privatestats> (2024), accessed: 2026-01-29
 28. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO’86. LNCS, vol. 263, pp. 186–194. Springer, Berlin, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
 29. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part II. LNCS, vol. 10992, pp. 33–62. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96881-0_2
 30. Google: Private state tokens. <https://developers.google.com/privacy-sandbox/protections/private-state-tokens>, accessed: 2026-01-29
 31. Hanzlik, L., Lai, Y.F., Mula, M., Paracucchi, E., Slamanig, D., Tang, G.: Tanuki: New frameworks for (concurrently secure) blind signatures from post-quantum groups actions. In: Asiacrypt 2025.
 32. Hanzlik, L., Slamanig, D.: With a little help from my friends: Constructing practical anonymous credentials. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021. pp. 2004–2023. ACM Press (Nov 2021). <https://doi.org/10.1145/3460120.3484582>
 33. Hauck, E., Kiltz, E., Loss, J.: A modular treatment of blind signatures from identification schemes. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 345–375. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17659-4_12
 34. Hinterwalder, G., Zenger, C.T., Baldimtsi, F., Lysyanskaya, A., Paar, C., Burleson, W.P.: Efficient E-cash in practice: NFC-based payments for public transportation systems. In: De Cristofaro, E., Wright, M.K. (eds.) PETS 2013. LNCS, vol. 7981, pp. 40–59. Springer, Berlin, Heidelberg (Jul 2013). https://doi.org/10.1007/978-3-642-39077-7_3
 35. Hopcroft, J.E., Karp, R.M.: An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. SIAM Journal on Computing **2**(4), 225–231 (1973). <https://doi.org/10.1137/0202019>, <https://doi.org/10.1137/0202019>
 36. IETF: Private pass. <https://datatracker.ietf.org/wg/privacypass/about/> (2020), accessed: 2026-01-29
 37. Kastner, J., Loss, J., Renawi, O.: Concurrent security of anonymous credentials light, revisited. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 45–59. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623184>
 38. Kastner, J., Loss, J., Renawi, O.: Concurrent security of anonymous credentials light, revisited. Cryptology ePrint Archive, Report 2023/707 (2023), <https://eprint.iacr.org/2023/707>

39. Kastner, J., Loss, J., Xu, J.: On pairing-free blind signature schemes in the algebraic group model. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part II. LNCS, vol. 13178, pp. 468–497. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_16
40. Katsumata, S., Lai, Y.F., LeGrow, J.T., Qin, L.: CSI-Otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part III. LNCS, vol. 14083, pp. 729–761. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_24
41. Katz, J., Loss, J., Rosenberg, M.: Boosting the security of blind signature schemes. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 468–492. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_16
42. Kreuter, B., Lepoint, T., Orrù, M., Raykova, M.: Anonymous tokens with private metadata bit. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part I. LNCS, vol. 12170, pp. 308–336. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56784-2_11
43. Meunier, T., Rubin, C.D., Faz-Hernández, A.: Privacy pass: upgrading to the latest protocol version. <https://blog.cloudflare.com/privacy-pass-standard> (2024), accessed: 2026-01-29
44. Okamoto, T.: Provably secure and practical identification schemes and corresponding signature schemes. In: Brickell, E.F. (ed.) CRYPTO’92. LNCS, vol. 740, pp. 31–53. Springer, Berlin, Heidelberg (Aug 1993). https://doi.org/10.1007/3-540-48071-4_3
45. Orrù, M., Tessaro, S., Zaverucha, G., Zhu, C.: Oblivious issuance of proofs. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IX. LNCS, vol. 14928, pp. 254–287. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68400-5_8
46. Paquin, C., Zaverucha, G.: U-prove technology overview v1.1. In: Microsoft Technical Report, <https://www.microsoft.com/en-us/research/publication/uprove-cryptographic-specification-v1-1-revision-3/> (2013)
47. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) CRYPTO’91. LNCS, vol. 576, pp. 129–140. Springer, Berlin, Heidelberg (Aug 1992). https://doi.org/10.1007/3-540-46766-1_9
48. Sanders, O.: Efficient redactable signature and application to anonymous credentials. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part II. LNCS, vol. 12111, pp. 628–656. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45388-6_22
49. Schnorr, C.P.: Security of blind discrete log signatures against interactive attacks. In: Qing, S., Okamoto, T., Zhou, J. (eds.) ICICS 01. LNCS, vol. 2229, pp. 1–12. Springer, Berlin, Heidelberg (Nov 2001). https://doi.org/10.1007/3-540-45600-7_1
50. Silde, T., Strand, M.: Anonymous tokens with public metadata and applications to private contact tracing. In: Eyal, I., Garay, J.A. (eds.) FC 2022. LNCS, vol. 13411, pp. 179–199. Springer, Cham (May 2022). https://doi.org/10.1007/978-3-031-18283-9_9
51. Tessaro, S., Zhu, C.: Short pairing-free blind signatures with exponential security. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 782–811. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_27

Supplementary Material

A Additional Preliminaries

A.1 Groups and Hardness Assumptions

Definition 7 (AGM/Algebraic Algorithms [29]). We say that an algorithm is algebraic with respect to a group $\mathbb{G}$ if it takes as input a set of group elements $g_1, \dots, g_n$ (w.l.o.g. we assume that one of them is the generator g and in case of interactive algorithms group elements in the output of any potential oracles are added to the set) along with a bit string input s and for every group element y that it outputs it also outputs an explanation vector (sometimes also called the representation) z such that

$$y = \prod_{i=1}^n g_i^{z_i}.$$

In the algebraic group model (AGM) it is assumed that every algorithm is algebraic.

Definition 8 (Discrete Logarithm Problem (DLP)). We define the discrete logarithm game $\text{DLP}_{\mathbb{G}}$ for the group $\mathbb{G}$ with generator g and prime order p as follows:

1. Sample $x \leftarrow_{\$} \mathbb{Z}_p$. Invoke the adversary on input g, g^x .
2. The adversary outputs a candidate solution $x' \in \mathbb{Z}_p$, the game outputs 1 if $g^{x'} = g^x$, 0 otherwise.

We define the advantage of an adversary $\mathcal{A}$ in the game $\text{DLP}_{\mathbb{G}}$ as

$$\text{Adv}_{\mathbb{G}}^{\text{DLP}}(\mathcal{A}) := \Pr[\text{DLP}_{\mathbb{G}}^{\mathcal{A}} = 1]$$

and say that DLP is (t, ε) -hard in the group $\mathbb{G}$ if for any adversary $\mathcal{A}$ that runs in time at most t , it holds that $\text{Adv}_{\mathbb{G}}^{\text{DLP}}(\mathcal{A}) \leq \varepsilon$

Definition 9 (Decisional Diffie-Hellman Problem (DDH)). We define the Decisional Diffie-Hellman game $\text{DDH}_{\mathbb{G}}$ for the group $\mathbb{G}$ with generator g and prime order p as follows:

1. Sample $x, y, z \leftarrow_{\$} \mathbb{Z}_p$ and a bit $b \leftarrow_{\$} \{0, 1\}$.
2. Invoke the adversary on input $g, X = g^x, Y = g^y, Z = g^{xy+bz}$.
3. The adversary outputs a candidate solution $b' \in \mathbb{Z}_p$.

We define the advantage of an adversary $\mathcal{A}$ in the game $\text{DDH}_{\mathbb{G}}$ as

$$\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) := 2 \cdot \left| \Pr[b = b'] - \frac{1}{2} \right|$$

and say that DDH is (t, ε) -hard in the group $\mathbb{G}$ if for any adversary $\mathcal{A}$ that runs in time at most t , it holds that $\text{Adv}_{\mathbb{G}}^{\text{DDH}}(\mathcal{A}) \leq \varepsilon$

A.2 Chernoff Bound

We recall a helpful lemma which we use later in this work.

Lemma 1 (Chernoff Bound). *Let X be the sum of independent $\{0, 1\}$ -random variables. Let $s \in \mathbb{N}$ with $s > \mathbb{E}[X]$. It holds that*

$$\Pr[X \geq s] \leq \exp(3 \cdot \mathbb{E}[X] - s).$$

For the proof, see Lemma E.1. in [15].

A.3 Definitions for Three-Move Blind Signatures

We recall the standard definitions for blind signatures.

Definition 10 (Correctness). *We say that a blind signature scheme $\text{BS} = (\mathcal{G}, \mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2), \mathcal{U} = (\mathcal{U}_1, \mathcal{U}_2, \mathcal{V}))$ is perfectly correct if the following holds: for all public parameters pp and all messages m ,*

1. *Generate a pair of keys $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}(\text{pp})$.*
2. *Run the signing protocol honestly by the signer $\mathcal{S}(\text{sk})$ and the user $\mathcal{U}(\text{pk}, m)$ resulting in the user obtaining a signature σ*
3. *Verification $\mathcal{V}(\text{pk}, m, \sigma)$ outputs 1.*

Definition 11 (Blindness). *For a blind signature $\text{BS} = (\mathcal{G}, \mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2), \mathcal{U} = (\mathcal{U}_1, \mathcal{U}_2, \mathcal{V}))$ and an adversary $\mathcal{A}$ we define the game **BLIND** as follows:*

1. *Invoke the first signer stage $\mathcal{A}$ that outputs a public key pk , two messages m_0, m_1 .*
2. *Sample a bit $b \leftarrow \{0, 1\}$ uniformly at random.*
3. *Invoke $\mathcal{A}^{\mathcal{U}^0(\text{pk}, m_b), \mathcal{U}^1(\text{pk}, m_{1-b})}$ where $\mathcal{A}$ may query its oracles in an interleaved fashion as long as it queries each oracle once. The oracles behave as follows:*
 - a. $\mathcal{U}^0(\text{pk}, m_b), \mathcal{U}^1(\text{pk}, m_{1-b})$: *The oracles run the user-side of the signing protocol. When a user algorithm terminates, the game locally stores the tuple (m_b, σ_b) resp. (m_{1-b}, σ_{1-b})*
4. *Once both $\mathcal{U}^0$ and $\mathcal{U}^1$ have terminated, if both sessions resulted in a signature, output the two message-signature pairs (m_0, σ_0) and (m_1, σ_1) to the adversary. Otherwise, output $(\perp, \perp)$ to the adversary.*
5. *The adversary outputs a bit b' .*

*We define the advantage of $\mathcal{A}$ in the game **BLIND** as*

$$\text{Adv}_{\text{BSA}}^{\text{BLIND}}(\mathcal{A}) := 2 \cdot \left| \Pr[b = b'] - \frac{1}{2} \right|.$$

We say that BSA is (t, ε) -blind if for all adversaries that run in time at most t , $\text{Adv}_{\text{BSA}}^{\text{BLIND}}(\mathcal{A}) \leq \varepsilon$.

Definition 12 ((strong) OMUF). For a blind signature with attributes $\text{BS} = (\mathcal{G}, \mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2), \mathcal{U} = (\mathcal{U}_1, \mathcal{U}_2), \mathcal{V})$, public parameters pp , a positive integer $\ell \in \mathbb{Z}^+$, and an adversary $\mathcal{A}$, we define Game (strong) ℓ -OMUF as follows:

1. Generate key pair $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}(\text{pp})$.
2. Invoke $\mathcal{A}^{\mathcal{S}(\text{sk}, \cdot)}(\text{pk})$. $\mathcal{A}$ may query its oracle $\mathcal{S}(\text{sk}, \cdot)$ arbitrarily and in an interleaved fashion as long as it completes at most ℓ sessions with $\mathcal{S}$.
3. $\mathcal{A}$ outputs $\ell+1$ pairwise-distinct tuples $(\sigma_1, m_1), \dots, (\sigma_{\ell+1}, m_{\ell+1})$ (in the case of strong OMUF). In the case of weak or unique-message OMUF we require additionally that $m_i \neq m_j$ for $i \neq j$.
4. Output 1 if for all $i \in [\ell+1]$, $\mathcal{V}(\text{pk}, m_i, \sigma_i) = 1$.

We define $\mathcal{A}$'s advantage in winning the game ℓ -OMUF against a blind signature scheme with attributes BSA as

$$\text{Adv}^{\ell\text{-OMUF}}_{\text{BSA}}(\mathcal{A}) := \Pr[\ell\text{-OMUF}^{\mathcal{A}}_{\text{BSA}} = 1].$$

We say that BSA is (t, ε, ℓ) -(strong-)OMUF-secure if for all adversaries $\mathcal{A}$ that run in time at most t , $\text{Adv}^{\ell\text{-OMUF}}_{\text{BSA}}(\mathcal{A}) \leq \varepsilon$.

A.4 Commitments

Definition 13 (Commitment Scheme). Commitment schemes $\text{CS} = (\mathcal{PG}, \mathcal{C}, \mathcal{V})$ consist of the following algorithms:

- The parameter generation algorithm $\mathcal{PG}$ takes as input public parameters pp and outputs commitment parameters pp_{CS} .
- The commit algorithm $\mathcal{C}$ takes as input commitment parameters pp_{CS} and a value v outputs a commitment C and an opening r .
- The verification algorithm $\mathcal{V}$ takes as input commitment parameters pp_{CS} , a commitment C , a value v and an opening r outputs either 1 (ACCEPT) or 0 (REJECT).

Definition 14 (Hiding). We define Game HIDING for a commitment scheme CS and an adversary $\mathcal{A}$ as follows:

1. Run $\text{CS}.\mathcal{PG}(\text{pp})$ to obtain pp_{CS} and output pp_{CS} to the adversary.
2. The adversary outputs two values v_0, v_1 .
3. Sample a bit b uniformly at random.
4. Compute $(C, r) \leftarrow_{\$} \text{CS}.\mathcal{C}(\text{pp}_{\text{CS}}, v_b)$ and output C to the adversary.
5. The adversary outputs a bit b' .

We define the advantage of the adversary $\mathcal{A}$ as

$$\text{Adv}^{\text{HIDING}}_{\text{CS}}(\mathcal{A}) := 2 \cdot \left| \Pr[b = b'] - \frac{1}{2} \right|$$

We say that CS is (ε, t) -hiding if for any adversary $\mathcal{A}$ that runs in time at most t , it holds that $\text{Adv}^{\text{HIDING}}_{\text{CS}}(\mathcal{A}) \leq \varepsilon$. We say that CS is perfectly hiding if for any adversary the advantage is 0.

Definition 15 (Binding). We define the BINDING game for a commitment scheme CS and an adversary $\mathcal{A}$ as follows:

1. Run CS.PG to obtain pp_{CS} and output pp_{CS} to the adversary.
2. The adversary outputs a commitment C , two values v_1, v_2 and two openings r_1, r_2 .
3. Output 1 if $v_1 \neq v_2$ and $\text{CV}(\text{pp}_{\text{CS}}, v_1, r_1) = \text{CV}(\text{pp}_{\text{CS}}, v_2, r_2) = 1$, 0 otherwise.

We define the advantage of the adversary $\mathcal{A}$ as

$$\text{Adv}_{\text{CS}}^{\text{BINDING}}(\mathcal{A}) := \Pr \left[\text{BINDING}_{\text{CS}}^{\mathcal{A}} = 1 \right]$$

and we say that CS is (t, ε) -binding if for any adversary $\mathcal{A}$ that runs in time at most t it holds that $\text{Adv}_{\text{CS}}^{\text{BINDING}}(\mathcal{A}) \leq \varepsilon$. We say that CS is perfectly binding if for any adversary the advantage is 0.

A.5 Multi-index Vector Commitment

Definition 16 (Multi-index Vector Commitment). We define a Multi-index Vector Commitment scheme $\text{MiVC} := (\mathcal{C}, \mathcal{O}, \text{CV})$ as follows.

- The commit algorithm $\mathcal{C}$ takes as input a randomness $\mathbf{R}$ and a messages vector $\mathbf{L}$, and it outputs a commitment C .
- The opening algorithm $\mathcal{O}$ takes as input a commitment C , a messages vector $\mathbf{L}$, a randomness $\mathbf{R}$, and an opening index vector $\mathbf{I}_{\text{op}}$, and it outputs an opening $(\mathbf{L}', \mathbf{R}')$.
- The verification algorithm CV takes as input a commitment C , an opening $(\mathbf{L}', \mathbf{R}')$, and an opening index vector $\mathbf{I}_{\text{op}}$, and it outputs either 1 (ACCEPT) or 0 (REJECT).

Definition 17 (Correctness). We say that a Multi-index Vector Commitment scheme MiVC is correct if for all message vectors $\mathbf{L}$, all randomness vectors $\mathbf{R}$, and all index sets $\mathbf{I}_{\text{op}} \subseteq [\nu]$:

$$\Pr \left[\text{CV}(C, \mathbf{L}', \mathbf{R}', \mathbf{I}_{\text{op}}) = 1 \mid \begin{array}{l} C \leftarrow \mathcal{C}(\mathbf{L}, \mathbf{R}) \\ (\mathbf{L}', \mathbf{R}') \leftarrow \mathcal{O}(C, \mathbf{L}, \mathbf{R}, \mathbf{I}_{\text{op}}) \end{array} \right] = 1.$$

Definition 18 (Hiding). We define Game HIDING for a Multi-index Vector Commitment scheme $\text{MiVC} := (\mathcal{C}, \mathcal{O}, \text{CV})$ and an adversary $\mathcal{A}$ as follows:

1. The adversary outputs two message vectors $\mathbf{L}^{(0)}, \mathbf{L}^{(1)}$ and a target set of opening indices $\mathbf{I}_{\text{op}}$.
2. If there exists $j \in \mathbf{I}_{\text{op}}$ such that $L_j^{(0)} \neq L_j^{(1)}$, abort and output 0.
3. Sample a bit b uniformly at random and a randomness vector $\mathbf{R}$.
4. Compute $C \leftarrow \mathcal{C}(\mathbf{L}^{(b)}, \mathbf{R})$ and the opening $(\mathbf{L}', \mathbf{R}') \leftarrow \mathcal{O}(C, \mathbf{L}^{(b)}, \mathbf{R}, \mathbf{I}_{\text{op}})$.
5. Output $(C, \mathbf{L}', \mathbf{R}')$ to the adversary.
6. The adversary outputs a bit b' .

We define the advantage of the adversary $\mathcal{A}$ as

$$\mathbf{Adv}_{\text{MiVC}}^{\text{HIDING}}(\mathcal{A}) := 2 \cdot \left| \Pr[b = b'] - \frac{1}{2} \right|$$

We say that MiVC is (ε, t) -hiding if for any adversary $\mathcal{A}$ that runs in time at most t , it holds that $\mathbf{Adv}_{\text{MiVC}}^{\text{HIDING}}(\mathcal{A}) \leq \varepsilon$.

Definition 19 (Position Binding). We define Game vcBINDING for a Multi-index Vector Commitment scheme $\text{MiVC} := (\mathcal{C}, \mathcal{O}, \mathcal{CV})$ and an adversary $\mathcal{A}$ as follows:

1. The adversary outputs a commitment C , two opening pairs $(\mathbf{L}^{(0)}, \mathbf{R}^{(0)})$ and $(\mathbf{L}^{(1)}, \mathbf{R}^{(1)})$, an opening index vector $\mathbf{I}_{\text{op}}$, and a target index $i \in \mathbf{I}_{\text{op}}$.
2. Output 1 if:
 - a. $L_i^{(0)} \neq L_i^{(1)}$,
 - b. $\mathcal{CV}(C, \mathbf{L}^{(0)}, \mathbf{R}^{(0)}, \mathbf{I}_{\text{op}}) = 1$, and
 - c. $\mathcal{CV}(C, \mathbf{L}^{(1)}, \mathbf{R}^{(1)}, \mathbf{I}_{\text{op}}) = 1$.
Otherwise, output 0.

We define the advantage of the adversary $\mathcal{A}$ as

$$\mathbf{Adv}_{\text{MiVC}}^{\text{vcBINDING}}(\mathcal{A}) := \Pr \left[\text{vcBINDING}_{\text{MiVC}}^{\mathcal{A}} = 1 \right]$$

We say that MiVC is (t, ε) -position binding if for any adversary $\mathcal{A}$ that runs in time at most t it holds that $\mathbf{Adv}_{\text{MiVC}}^{\text{vcBINDING}}(\mathcal{A}) \leq \varepsilon$.

A.6 Properties of Pedersen Commitments

Construction 2 (Generalized Pedersen Commitment [47]). We describe the generalized Pedersen Commitment Scheme GPC over a group $\mathbb{G}$ described by group parameters $\text{pp}_{\mathbb{G}}$ (which take the role of the public parameters pp) for vectors $\mathbf{v}$ of length ν below:

- $\mathcal{PG}(\text{pp}_{\mathbb{G}})$: sample $\nu + 1$ group elements $h_0, \dots, h_n \leftarrow_{\$} \mathbb{G}$. Output those group elements as the commitment parameters $\text{pp}_{\text{GPC}} = (h_0, \dots, h_n)$.
- $\mathcal{C}(\text{pp}_{\text{GPC}}, \mathbf{v})$: Given commitment parameters as above and a vector $\mathbf{v} \in \mathbb{Z}_p^\nu$ with ν entries, sample an exponent $r \leftarrow_{\$} \mathbb{Z}_p$ and output $C = h_0^r \prod_{i=1}^{\nu} h_i^{v_i}$ and r .
- $\mathcal{CV}(\text{pp}_{\text{GPC}}, v, C, r)$: Given the commitment parameters as above, a value v , a commitment C , and an opening r , output 1 iff $C = h_0^r \prod_{i=1}^{\nu} h_i^{v_i}$.

Construction 3 (PoK for Opening of a Generalized Pedersen Commitment). We describe a protocol for proving knowledge of a full opening of GPC as described in construction 2, i.e. for the relation $\mathcal{R}_{\mathbf{h}} = \{(C, (r, \mathbf{v})) : h_0^r \cdot \prod_{i=1}^{\nu} h_i^{v_i} = C\}$,

- The prover algorithm $\mathcal{P}$ is split into two algorithms $\mathcal{P}_1$ and $\mathcal{P}_2$.

- $\mathcal{P}_1(\mathbf{h}, C, (r, \mathbf{v}))$: Sample $r_0, \dots, r_n \leftarrow_{\$} \mathbb{Z}_p$. Compute $R = \prod_{i=0}^n h_i^{r_i}$. Output the first flow R and keep the local state $r_0, \dots, r_n$.
- $\mathcal{P}_2(r_0, \dots, r_n, c, \mathbf{v})$: Compute $s_0 = r_0 - c \cdot r$, for $i = 1, \dots, n$ compute $s_i = r_i - c \cdot v_i$. Output $s_0, \dots, s_n$.
- The *verifier algorithm* $\mathcal{V}$ is split into two algorithms $\mathcal{V}_1$ and $\mathcal{V}_2$:
 - $\mathcal{V}_1(\mathbf{h}, C, R)$: Sample a random challenge $c \leftarrow_{\$} \mathbb{Z}_p$, output c , keep local state $\mathbf{h}, C, R$.
 - $\mathcal{V}_2(\mathbf{h}, C, R, c, \mathbf{s})$: Output 1 if $R = C^c \cdot \prod_{i=0}^n h_i^{s_i}$, 0 otherwise.

Lemma 2 (Pedersen Commitments are perfectly hiding). *The commitment scheme GPC from construction 2 is perfectly hiding.*

Proof. Information-theoretically, for each vector of commitment parameters $\mathbf{h} \in \mathbb{G}^{\nu+1}$, each commitment $C \in \mathbb{G}$ and each value vector $\mathbf{v} \in \mathbb{Z}_p^\nu$, there exists exactly one opening $r = \text{dlog}_{h_0} \frac{C}{\prod_{i=1}^\nu h_i^{v_i}}$. The commitment thus perfectly hides the vector $\mathbf{v}$ and even an unbounded adversary cannot do better than guessing.

Lemma 3 (Pedersen Commitments are binding under DL). *If the DLP is (t, ε) -hard in the group $\mathbb{G}$, GPC from construction 2 is (t', ε') -binding with $t' \approx t$ and $\varepsilon' \leq \nu \cdot \varepsilon$.*

Proof. We provide a reduction $\mathcal{B}$ against DLP that uses a binding adversary $\mathcal{A}$ to break DLP. The reduction takes as input the discrete logarithm challenge g, Y . It samples $j \leftarrow_{\$} [\nu]$ and sets $h_j := Y$. For each $i \neq j \in [\nu]_0$ it samples $a_i \leftarrow_{\$} \mathbb{Z}_p$ and sets $h_i := g^{a_i}$. It outputs the commitment parameters $\text{pp}_{\text{GPC}} = (h_0, \dots, h_\nu)$ to the adversary $\mathcal{A}$. When the adversary outputs a commitment C and two vectors $\mathbf{v}_1, \mathbf{v}_2$ along with openings r_1, r_2 , the reduction aborts if $v_{1,j} = v_{2,j}$. As $\mathbf{v}_1 \neq \mathbf{v}_2$, there must be at least one index i^* where $v_{1,i^*} \neq v_{2,i^*}$ and $j = i^*$ happens with probability $\frac{1}{\nu}$. Thus, the reduction aborts with probability at most $1 - \frac{1}{\nu}$. If the reduction did not abort in the previous step, it computes $\text{dlog} Y = \frac{r_1 \cdot a_0 + \sum_{i \neq j} a_i \cdot v_{1,i} - (r_2 \cdot a_0 + \sum_{i \neq j} a_i \cdot v_{2,i})}{v_{2,j} - v_{1,j}}$ and outputs the solution.

A.7 Sigma-Protocols and Proofs

We recall definition of Σ -protocols and some of their security properties, see also for example [21]

Construction 4 (Σ -Protocol). A Σ -Protocol for a relation $\mathcal{R}$ consists of the following algorithms $\mathcal{P} = (\mathcal{P}_1, \mathcal{P}_2)$, $\mathcal{V} = (\mathcal{V}_1, \mathcal{V}_2)$.

- $\mathcal{P}_1(x, w)$ Outputs a message R and saves an internal state $s_{\mathcal{P}}$
- $\mathcal{V}_1(x, R)$ Takes as input the statement and the prover's first message and outputs a challenge $c \leftarrow_{\$} \mathcal{C}$.
- $\mathcal{P}_2(x, w, s_{\mathcal{P}}, c)$ Outputs a response S .
- $\mathcal{V}_2(x, R, c, S)$ Outputs a bit b indicating acceptance or rejection.

Definition 20 (Σ -Protocol Correctness). A Σ -Protocol is correct if for all $(x, w) \in \mathcal{R}$, all $(R, s_P) \leftarrow_{\$} \mathcal{P}_1(x, w)$, all $c \in \mathcal{C}$, and all $S \leftarrow_{\$} \mathcal{P}_2(x, w, s_P, c)$, it holds that $\mathcal{V}_2(x, R, c, S) = 1$

Definition 21. (HVZK) A Σ -Protocol is honest-verifier zero-knowledge (HVZK) if there exists a simulator $\mathcal{SIM}$ such that for all $x \in \mathcal{L}$ and all $c \in \mathcal{C}$, the distributions of $(R, c, S) \leftarrow_{\$} \mathcal{SIM}(x, c)$ are indistinguishable from $R' \leftarrow_{\$} \mathcal{P}_1(x, w), c, S \leftarrow_{\$} \mathcal{P}_2(x, w, s_P, c)$ conditioned on c being used as the challenge.

Definition 22 (k-Special-Soundness). A Σ -Protocol is k-special sound if there exists a deterministic extractor $\mathcal{E}$ such that given k valid transcripts $\{(R, c_i, S_i)\}_{i \in [k]}$ for statement x with pairwise distinct challenges c_i , outputs a witness w such that $(x, w) \in \mathcal{R}$.

Definition 23 (ϵ -Soundness). A Σ -Protocol is ϵ -sound if for any $x \notin \mathcal{L}$, any adversary $\mathcal{P}^*$ it holds that the probability that $\mathcal{V}_2(x, R, c, S) = 1$ is at most ϵ . Where $(R, s_P) \leftarrow_{\$} \mathcal{P}_1^*(x)$, $c \leftarrow_{\$} \mathcal{V}_1(x, R)$, and $S \leftarrow_{\$} \mathcal{P}_2(x, w, s_P, c)$ and the probability is taken over the random coins of the $\mathcal{P}^*$ and $\mathcal{V}_1(x, R)$.

Construction 5 (Fiat Shamir Transform [28]). The Fiat-Shamir transform of a Σ -protocol $\Sigma = (\mathcal{P} = (\mathcal{P}_1, \mathcal{P}_2), \mathcal{V} = (\mathcal{V}_1, \mathcal{V}_2))$ with challenge space $\mathcal{C}$ and a hash function $H: \{0, 1\}^* \rightarrow \mathcal{C}$ and some auxiliary information that might be used in surrounding protocols is described below.

$\mathcal{P}(x, w)$ Compute $(R, s_P) \leftarrow_{\$} \mathcal{P}_1(x, w)$. Compute $c := H(x, R, aux)$ for potential auxiliary information aux . Compute $S \leftarrow_{\$} \mathcal{P}_2(x, w, s_P, c)$. Output $\pi = (R, S)$
 $\mathcal{V}(x, \pi = (R, S), aux)$ Compute $c = H(x, R, aux)$, output $\mathcal{V}_2(x, R, c, S)$.

Lemma 4. Let Σ_1 be a Σ -protocol for language $\mathcal{L}_1$ with relation $\mathcal{R}_1$ and Σ_2 be a Σ -protocol for language $\mathcal{L}_2$ with relation $\mathcal{R}_2$ where Σ_1 and Σ_2 have the same challenge space $\mathcal{C}$ and both fulfill k -Special-Soundness and HVZK as above. Then, the following protocol is a Σ -protocol for the language $\mathcal{L}_1 \times \mathcal{L}_2$ with relation $\mathcal{R} = \{((x_1, x_2), (w_1, w_2)) : (x_1, w_1) \in \mathcal{R}_1, (x_2, w_2) \in \mathcal{R}_2\}$ which fulfills k -Special-Soundness and HVZK.

$\mathcal{P}_1$ compute $(R_1, s_1) \leftarrow_{\$} \Sigma_1.\mathcal{P}_1(x_1, w_1), (R_2, s_2) \leftarrow_{\$} \Sigma_2.\mathcal{P}_2$
 $\mathcal{V}_1$ sample $c \leftarrow_{\$} \mathcal{C}$
 $\mathcal{P}_2$ compute $S_1 \leftarrow_{\$} \Sigma_1.\mathcal{P}_2(x_1, w_1, s_1, c), S_2 \leftarrow_{\$} \Sigma_2.\mathcal{P}_2(x_2, w_2, s_2, c)$
 $\mathcal{V}_2(x_1, x_2, R_1, R_2, c, S_1, S_2)$ run $\Sigma_1.\mathcal{V}_2(x_1, R_1, c, S_1)$ and $\Sigma_2.\mathcal{V}_2(x_2, R_2, c, S_2)$ and output 1 iff both output 1.

Proof. For HVZK, given c, x_1, x_2 run $(R_1, S_1) \leftarrow_{\$} \mathcal{SIM}_1(x_1, c)$ and $(R_2, S_2) \leftarrow_{\$} \mathcal{SIM}_2(x_2, c)$ where $\mathcal{SIM}_1$ and $\mathcal{SIM}_2$ are the HVZK simulators for Σ_1 and Σ_2 , respectively.

For k -special soundness, given k transcripts with the same first flow R_1, R_2 and k different challenges c_i and corresponding responses $S_{1,i}, S_{2,i}$, run $\mathcal{E}_1$ on $\{(R_1, c_i, S_{1,i}) : i \in [k]\}$ and $\mathcal{E}_2$ on $\{(R_2, c_i, S_{2,i}) : i \in [k]\}$ to obtain the witnesses w_1 and w_2 respectively.

A.8 Identification Scheme from ACL (ID_{ACL})

In the following, we describe the identification scheme ID_{ACL} , which was proposed by [37]. The scheme is defined as tuple $(\mathcal{G}, \mathcal{P}, \mathcal{V})$, where

- $\mathcal{G}(\text{pp})$: Parse pp to obtain $\mathbb{G}, p, g$, sample $h \leftarrow_{\$} \mathbb{G}$ and $x, v_{0,1}, v_{0,2} \leftarrow_{\$} \mathbb{Z}_p$, compute $y = g^x, z := g^{v_{0,1}} \cdot h^{v_{0,2}}$, set $\text{sk} = x$ and $\text{pk} = (h, z, y)$, and output (pk, sk) .
- The prover algorithms $\mathcal{P} = (\mathcal{P}_1, \mathcal{P}_2)$ are defined as follows:
 - $\mathcal{P}_1(\text{sk})$: Sample $d, s_1, s_2, u, \text{rnd} \leftarrow_{\$} \mathbb{Z}_p$, compute $z_1 := g^{\text{rnd}}, z_2 := z/z_1, a := g^u, b_1 := g^{s_1} \cdot z_1^d, b_2 := h^{s_2} \cdot z_2^d$, set $R := (a, b_1, b_2, \text{rnd}), st_p := (d, s_1, s_2, u)$ and output (R, st_p) .
 - $\mathcal{P}_2(\text{sk}, st_p, R, \varepsilon)$: Parse st_p as (d, s_1, s_2, u) , compute $c := \varepsilon - d, r := u - c \cdot \text{sk}$, set and output $S := (c, d, r, s_1, s_2)$.
- The verifier algorithms $\mathcal{V} = (\mathcal{V}_1, \mathcal{V}_2)$ are defined as follows:
 - $\mathcal{V}_1(\text{pk}, R)$: Parse pk , and $R = (a, b_1, b_2, \text{rnd})$. Output $\perp$ if $\text{rnd} = 0$. Sample $\varepsilon \leftarrow_{\$} \mathbb{Z}_p$, set $st_v := (a, b_1, b_2, z_1 := g^{\text{rnd}}, z_2 := z/z_1)$, and output (ε, st_v) .
 - $\mathcal{V}_2(\text{pk}, st_v, R, \varepsilon, S)$: Parse S and st_v , and output 1 if the following condition holds: $\varepsilon = c + d \wedge a = g^r \cdot y^c \wedge b_1 = g^{s_1} \cdot z_1^d \wedge b_2 = h^{s_2} \cdot z_2^d$. Otherwise, output 0.

A.9 Merkle Tree-based Multi-index Vector Commitment

Correctness of Merkle Tree-based Multi-index Vector Commitment

Lemma 5 (Correctness). *Our construction in 3.1 is correct.*

Proof. We need to show that $\mathcal{C}(\mathbf{L}; \mathbf{R}) = \mathcal{C}(\mathbf{L}'; \mathbf{R}')$.

First, the verification checks that for all $i \in \mathbf{I}_{\text{op}}$, $R'_i \neq \perp$. Since the opening algorithm sets $R'_i = R_i$ for all $i \in \mathbf{I}_{\text{op}}$, and valid randomness implies $R_i \neq \perp$, this check always passes.

We note that the commitment function attempts to reconstruct the leaves set $S = \{s_1, \dots, s_\nu\}$, and constructs the Merkle tree of these leaves. Therefore, it is sufficient to show that the execution of $\mathcal{C}(\mathbf{L}; \mathbf{R}) = \mathcal{C}(\mathbf{L}'; \mathbf{R}')$ results in the same leaves set S .

For all $i \in [\nu]$, we note that $s_i = \text{H}_{\text{MT}}(L_i; R_i)$ in the execution of $\mathcal{C}(\mathbf{L}; \mathbf{R})$. But we distinguish two cases in the execution of $\mathcal{C}(\mathbf{L}'; \mathbf{R}')$:

1. $i \in \mathbf{I}_{\text{op}}$. In this case it holds that $L'_i = L_i$ and $R_i = R_i$. Consequently, we have $s_i = \text{H}_{\text{MT}}(L'_i; R'_i)$.
2. $i \notin \mathbf{I}_{\text{op}}$. In this case, we have $R'_i = \perp$ and $L'_i = \text{H}_{\text{MT}}(L_i; R_i)$ (see $\mathcal{O}$). Again, we have $s_i = L'_i = \text{H}_{\text{MT}}(L_i; R_i)$.

In both cases, the leaves set S is identical in the execution of $\mathcal{C}(\mathbf{L}; \mathbf{R})$ and $\mathcal{C}(\mathbf{L}'; \mathbf{R}')$, hence it holds $\mathcal{C}(\mathbf{L}'; \mathbf{R}') = C$, hence $\mathcal{CV}(C, \mathbf{L}', \mathbf{R}', \mathbf{I}_{\text{op}}) = 1$.

Hiding of Merkle Tree-based Multi-index Vector Commitment

Lemma 6 (Hiding). *In the random oracle model, the commitment scheme MT described in Section 3.1 with a random oracle H_{MT} is Hiding. Concretely, for all positive integers ν and all adversaries $\mathcal{A}$ making at most Q_{MT} queries to H_{MT} , it holds*

$$\text{Adv}_{MT}^{\text{HIDING}}(\mathcal{A}) \leq \frac{2 \cdot Q_{MT}}{2^\lambda}.$$

Proof. Let $\mathcal{A}$ be an adversary running Game $\text{HIDING}_{MT}^{\mathcal{A}}$. $\mathcal{A}$ submits two vectors $\mathbf{L}^{(0)}$ and $\mathbf{L}^{(1)}$ of length ν along with a vector $\mathbf{I}_{op}$ of indices. Then $\mathcal{A}$ is given $C \leftarrow_{\$} \mathcal{C}(\mathbf{L}^{(b)}, \mathbf{R})$ along with an opening $(\mathbf{L}', \mathbf{R}') \leftarrow_{\$} \mathcal{O}(C, \mathbf{L}^{(b)}, \mathbf{R}, \mathbf{I}_{op})$ for uniformly random $b \leftarrow_{\$} \{0, 1\}$ and $\mathbf{R} \leftarrow_{\$} (\{0, 1\}^\lambda)^\nu$ and it outputs b' .

We use a simple hybrid argument.

Game $\mathbf{G}_0$. This is Game $\text{HIDING}_{MT}^{\mathcal{A}}$.

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$, except that it aborts if $\mathcal{A}$ queries H_{MT} on $(L_i^{(b)}, R_i)$ for some $i \notin \mathbf{I}_{op}$.

Claim 2.

$$|\Pr[\mathbf{G}_1 = 1] - \Pr[\mathbf{G}_0 = 1]| \leq Q_{MT} \cdot 2^{-\lambda}.$$

Proof. Since R_i is chosen uniformly at random and is hidden from $\mathcal{A}$, the probability of $\mathcal{A}$ hitting the query $(L_i^{(b)}, R_i)$ is at most $2^{-\lambda}$ for a fixed query. Taking the union bound over all Q_{MT} queries $\mathcal{A}$ makes to H_{MT} , the probability that $\mathbf{G}_1$ aborts is at most $Q_{MT} \cdot 2^{-\lambda}$.

Now we analyze the advantage of $\mathcal{A}$ in winning $\mathbf{G}_1$.

We distinguish two types of vector positions:

Case 1. Opened positions $i \in \mathbf{I}_{op}$: These positions are visible in the opening $(\mathbf{L}', \mathbf{R}')$ $\mathcal{A}$ receives. However, since the winning conditions enforce both vectors to be identical at the opened positions, these positions do not help the adversary in guessing b .

Case 2. Unopened positions $i \notin \mathbf{I}_{op}$: For each position, $\mathcal{A}$ receives $L'_i = H_{MT}(L_i^{(b)}, R_i)$ and $R'_i = \perp$. In this case, the value L'_i is uniformly random in $\mathcal{A}$'s view, as long as it does not query H_{MT} on $(L_i^{(b)}, R_i)$. Due to the abort condition in $\mathbf{G}_1$, $\mathcal{A}$ does not query $H_{MT}(L_i^{(b)}, R_i)$. Consequently, as L'_i is uniformly random and independent of b in $\mathcal{A}$'s view, it outputs the correct b with probability $\frac{1}{2}$.

Overall, we have:

$$\text{Adv}_{MT}^{\text{HIDING}}(\mathcal{A}) = 2 \cdot \left| \Pr[\mathbf{G}_0 = 1] - \frac{1}{2} \right| \leq 2 \cdot \left(\left| \frac{1}{2} - \frac{1}{2} \right| + Q_{MT} \cdot 2^{-\lambda} \right) = \frac{2 \cdot Q_{MT}}{2^\lambda}.$$

Binding of Merkle Tree-based Multi-index Vector Commitment

Lemma 7 (Binding). *In the random oracle model, the commitment scheme MT described in Section 3.1 with a random oracle H_{MT} is position binding. Let Q_{MT} be an upper bound for the number of queries $\mathcal{A}$ makes to H_{MT} . Then it holds*

$$\text{Adv}_{MT}^{\text{BINDING}}(\mathcal{A}) \leq \frac{Q_{MT} \cdot (Q_{MT} - 1)}{2^{\lambda+1}}.$$

Proof. Let $\mathcal{A}$ be an adversary running game vcBINDING_{MT}^A that outputs a commitment C , a target index $i \in [\nu]$, and two pairs $(\mathbf{L}^{(0)}, \mathbf{R}^{(0)})$ and $(\mathbf{L}^{(1)}, \mathbf{R}^{(1)})$ such that $L_i^{(0)} \neq L_i^{(1)}$, $\mathcal{CV}(C, \mathbf{L}^{(0)}, \mathbf{R}^{(0)}, \mathbf{I}_{op}) = \mathcal{CV}(C, \mathbf{L}^{(1)}, \mathbf{R}^{(1)}, \mathbf{I}_{op}) = 1$.

Since both openings open C correctly, it holds $C = C_0 = C_1$, where $C_0 := \mathcal{C}(\mathbf{L}^{(0)}; \mathbf{R}^{(0)})$ and $C_1 := \mathcal{C}(\mathbf{L}^{(1)}; \mathbf{R}^{(1)})$. However, the roots C_0 and C_1 are identical iff the leaves are identical or a collision occurs inside the tree.

We proceed via a series of games.

Game $\mathbf{G}_0$. This is Game vcBINDING_{MT}^A .

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$, except that it aborts if $\mathcal{A}$ finds a collisions (Q_1, Q_2) for H_{MT} , i.e. $Q_1 \neq Q_2$ with $H_{MT}(Q_1) = H_{MT}(Q_2)$.

Claim 3.

$$\text{Adv}^{\mathbf{G}_1}(\mathcal{A}) \geq \text{Adv}^{\mathbf{G}_0}(\mathcal{A}) - \frac{Q_{MT} \cdot (Q_{MT} - 1)}{2^{\lambda+1}}.$$

Proof. The probability that any specific pair (Q_1, Q_2) collide is $\frac{1}{2^\lambda}$. Since $\mathcal{A}$ makes up to Q_{MT} queries, the number of possible distinct pairs is $Q_{MT} \cdot (Q_{MT} - 1)/2$, taking the union bound yields that a collision occurs with probability at most $\frac{Q_{MT} \cdot (Q_{MT} - 1)}{2^{\lambda+1}}$.

Let MT_0 and MT_1 be the Merkle trees constructed from the openings $(\mathbf{L}^{(0)}, \mathbf{R}^{(0)})$ and $(\mathbf{L}^{(1)}, \mathbf{R}^{(1)})$, respectively. Let $MT_b[j]$ denote the set of the nodes at the j -th level of the tree MT_b for $j \in [\log(\nu)]_0$. The tree levels are indexed top to bottom (the root is at $j = 0$ and the leaves are at $j = \log(\nu)$.) In the following game, we want to rule out that the trees are different on some level j , yet they are identical on the level above.

Let $j \in [\log(\nu)]$ be a fixed index satisfying $MT_0[j] \neq MT_1[j]$ but $MT_0[j-1] = MT_1[j-1]$. Let $S_{b,j} := (s_{b,1}, \dots, s_{b,t})$ be the nodes of $MT_b[j]$ for $t = 2^j$. Assume without loss of generality $S_{0,j}$ and $S_{1,j}$ differ in a single pair $(s_{b,2r-1}, s_{b,2r})$ for $r \in [t/2]$.

Game $\mathbf{G}_2$. The game is identical to $\mathbf{G}_1$, except that it aborts and outputs 0 if $MT_0[j] \neq MT_1[j]$ but $MT_0[j-1] = MT_1[j-1]$.

Claim 4.

$$|\Pr[\mathbf{G}_2 = 1] - \Pr[\mathbf{G}_1 = 1]| = 0.$$

Proof. Since $MT_0[j-1]$ and $MT_1[j-1]$ are identical, it follows per construction that $H_{MT}(s_{0,2r-1}, s_{0,2r}) = H_{MT}(s_{1,2r-1}, s_{1,2r})$. This, however, does not occur; otherwise, $\mathbf{G}_1$ would have aborted.

The only remaining strategy for $\mathcal{A}$ is to ensure that the leaves of both trees are identical $MT_0[\log(\nu)] = MT_1[\log(\nu)]$. However, since we know that $L_i^{(0)} \neq L_i^{(1)}$ and $R_i^{(0)} \neq \perp \wedge R_i^{(1)} \neq \perp$, it must hold due to Game $\mathbf{G}_1$ that $H_{MT}(L_i^{(0)}; R_i^{(0)}) \neq H_{MT}(L_i^{(1)}; R_i^{(1)})$.

Overall, we have $\text{Adv}_{MT}^{\text{vcBINDING}}(\mathcal{A}) \leq \text{Adv}^{\mathbf{G}_1} + \frac{Q_{MT} \cdot (Q_{MT}-1)}{2^{\lambda+1}} = \frac{Q_{MT} \cdot (Q_{MT}-1)}{2^{\lambda+1}}$.

B Our Transformation: Pachirisu

Construction 6 (Pachirisu). Let $\text{BS}^H := (\mathcal{G}^H, \mathcal{S}, \mathcal{SIM}_{\mathcal{S}}, \mathcal{U}^H, \mathcal{V}^H)$ be a three-move simulatable blind signature scheme with respect to a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, where λ is a security parameter specified by public parameters pp . Let $H_J, H_m, H_{\text{rnd}}, H_\mu, H_\gamma : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be hash functions treated as random oracles, and let $\text{MT} := (\mathcal{C}, \mathcal{O}, \mathcal{CV})$ be the commitment scheme described in Construction 1. For a positive integer $K \geq 1$, a function $f : \mathbb{N} \rightarrow \mathbb{N}$, we define BS as follows:

- $\mathcal{G}(\text{pp})$: Generate K pairs of keys $(\text{pk}_i, \text{sk}_i) \leftarrow_{\$} \text{BS}.\mathcal{G}(\lambda)$ for $i \in [K]$, and return $\{(\text{pk}_i, \text{sk}_i)\}_{i \in [K]}$.
- $\mathcal{RG}_{\mathcal{U}}(\mathbf{L})$: Set $C := \mathbf{L}$, $\pi := \perp$, and $\text{rst}_{\mathcal{U}} := (C, C)$, and return $(C, \text{rst}_{\mathcal{U}}, \pi)$. The user sends the attribute vector $\mathbf{L}$ in plain to the signer instead of sending a commitment of the attributes; therefore, the proof is not required ($\pi = \perp$).
- $\mathcal{RG}_{\mathcal{S}}(C, \pi, \text{rst}'_{\mathcal{S}})$: Parse $\mathbf{L} := C$. Update and output the state $\text{rst}_{\mathcal{S}} := \text{rst}'_{\mathcal{S}} \cup \mathbf{L}$.
- $\mathcal{S}$: The signer algorithm is split into four algorithms $\mathcal{S}_{1-4}$. $\mathcal{S}_1$ acts as stateful initialization phase. After a session is initialized via an $\mathcal{S}_1$ query, the algorithms $\mathcal{S}_{2-4}$ are executed K times to conclude the session.
 - $\mathcal{S}_1(\text{sk}_i, \text{rst}_{\mathcal{S}}, C)$: If $C \notin \text{rst}_{\mathcal{S}}$, abort and output $\perp$. Otherwise, increment $\text{ctr} := \text{ctr} + 1$, and compute $N := f(\text{ctr})$. Sample $\text{seed}_i \leftarrow_{\$} \{0, 1\}^\lambda$ and compute $\text{com}_{\text{seed}_i} := H_J(\text{seed}_i)$ for all $i \in [K]$. Define a signer state $st_{\mathcal{S}} := (\text{ctr}, N, \{\text{seed}_i, \text{com}_{\text{seed}_i}\}_{i \in [K]})$. Return $(st_{\mathcal{S}}, N, \text{com}_{\text{seed}_i})$.
 - $\mathcal{S}_2(\text{sk}_i, st_{\mathcal{S}}, \{\text{com}_j\}_{j \in [N]})$: Parse N from $st_{\mathcal{S}}$. For all $j \in [N]$, compute $(\text{msgS1}_j, \text{rnd}_{\mathcal{S}_j}) \leftarrow_{\$} \text{BS}.\mathcal{S}_1(\text{sk}_i)$. Compute $J_i := H_J(\text{seed}_i, \text{pk}_i)$, update the signer state $st_{\mathcal{S}} := st_{\mathcal{S}} \cup \{J_i, \{\text{rnd}_{\mathcal{S}_j}, \text{msgS1}_j, \text{com}_j\}_{j \in [N]}\}$ and return $(st_{\mathcal{S}}, \{\text{msgS1}_j\}_{j \in [N]})$.
 - $\mathcal{S}_3(\text{sk}_i, st_{\mathcal{S}}, \{\text{msgU}_j\}_{j \in [N]})$: Parse seed_i from $st_{\mathcal{S}}$. Update the signer state $st_{\mathcal{S}} := st_{\mathcal{S}} \cup \{\text{msgU}_j\}_{j \in [N]}$. Return $(st_{\mathcal{S}}, \text{seed}_i)$.
 - $\mathcal{S}_4(\text{sk}_i, st_{\mathcal{S}}, \{\mathbf{R}_j, \mu_j, \text{Urnd}_j\}_{j \in [N]})$: Parse $(\{\text{com}_j, \text{msgU}_j, \text{rnd}_{\mathcal{S}_j}, \text{msgS1}_j\}_{j \in [N]}, J_i, C)$ from $st_{\mathcal{S}}$. For all $J_i \neq j \in [N]$, compute $\zeta_j := \text{MT}.\mathcal{C}(\mathbf{L}; \mathbf{R}_j)$, $\gamma_j := H_\gamma(\text{pk}_i, \mu_j, \zeta_j)$, and abort and output $\perp$ if $\text{com}_j \neq H_{\text{rnd}}(\text{Urnd}_j)$ or $\text{msgU}_j \neq \text{BS}.\mathcal{U}_1(\text{pk}_i, \gamma_j, \text{msgS1}_j, \text{Urnd}_j)$. Compute and output $\text{msgS2}_{J_i} := \text{BS}.\mathcal{S}_2(\text{sk}_i, \text{rnd}_{\mathcal{S}_{J_i}}, \text{msgU}_{J_i})$.

$\mathcal{S}$ outputs 1 (COMPLETE) if $\mathcal{S}_4(\text{sk}_K, \cdot)$ terminates successfully.

- $\mathcal{U}$: The user algorithm consists of four algorithms $\mathcal{U}_1$ – $\mathcal{U}_4$. Upon receiving $(N, \{com_{seed_i}\}_{i \in [K]})$ from the signer. Let M be the message being signed. The user initializes the session by setting $m_0 := M$, $\sigma_0 := \zeta_0 := \eta_0 := \perp$ and the state $st_{\mathcal{U}} := (m_0, \sigma_0, \zeta_0, \eta_0)$. Then, algorithms $\mathcal{U}_{1-4}$ are executed K times sequentially.
 - $\mathcal{U}_1(pk_i, st_{\mathcal{U}}, \mathbf{L}, N, \{com_{seed_i}\}_{i \in [K]})$: Parse $st_{\mathcal{U}}$ to extract $(m_{i-1}, \sigma_{i-1}, \zeta_{i-1}, \eta_{i-1})$. Compute $m_i := H_m(\sigma_{i-1}, \eta_{i-1}, \zeta_{i-1}, m_{i-1})$. For $j \in [N]$, sample $Urnd_j \leftarrow_{\$} \{0, 1\}^\lambda$, compute $com_j := H_{rnd}(Urnd_j)$. Update the state $st_{\mathcal{U}} := st_{\mathcal{U}} \cup \{m_i, \mathbf{L}, N, \{com_{seed_i}\}_{i \in [K]}, \{Urnd_j\}_{j \in [N]}\}$. Return $(st_{\mathcal{U}}, \{com_j\}_{j \in [N]})$.
 - $\mathcal{U}_2(pk_i, st_{\mathcal{U}}, \{msgS1_j\}_{j \in [N]})$: Parse $(m_i, \mathbf{L})$ from $st_{\mathcal{U}}$. For all $j \in [N]$, do the following: Sample $\eta_j \leftarrow_{\$} \{0, 1\}^\lambda$, $\mathbf{R}_j \leftarrow_{\$} (\{0, 1\}^\lambda)^\nu$, compute $\mu_j := H_\mu(m_i, \eta_j)$, $\zeta_j := MT.C(\mathbf{L}; \mathbf{R}_j)$, $\gamma_j := H_\gamma(pk_i, \mu_j, \zeta_j)$, and $msgU_j := BS.U_1(pk_i, \gamma_j, msgS1_j; Urnd_j)$. Update $st_{\mathcal{U}} := st_{\mathcal{U}} \cup \{\eta_j, \mathbf{R}_j, \mu_j, \zeta_j, \gamma_j, msgU_j\}_{j \in [N]}$. Return $(st_{\mathcal{U}}, \{msgU_j\}_{j \in [N]})$.
 - $\mathcal{U}_3(pk_i, st_{\mathcal{U}}, seed_i)$: Parse $(\{com_{seed_i}\}_{i \in [K]}, \{Urnd_j, \mathbf{R}_j, \mu_j\}_{j \in [N]})$ from the state $st_{\mathcal{U}}$. Abort and output $\perp$ if $com_{seed_i} \neq H_J(seed_i)$. Compute $J_i := H_J(seed_i, pk_i)$. Return $(st_{\mathcal{U}}, \{\mathbf{R}_j, \mu_j, Urnd_j\}_{j \neq J_i})$.
 - $\mathcal{U}_4(pk_i, st_{\mathcal{U}}, msgS2_{J_i})$: Parse $(m_i, \{Urnd_j, \eta_j, \zeta_j, \mathbf{R}_j, \gamma_j\}_{j \in [N]})$ from $st_{\mathcal{U}}$. Compute $\sigma_i := BS.U_2(pk_i, \gamma_{J_i}, msgS2_{J_i}; Urnd_{J_i})$. Update $st_{\mathcal{U}} := st_{\mathcal{U}} \cup \{\sigma_i, \eta_{J_i}, \zeta_{J_i}\}$. Return a signature $(\sigma_i, \eta_{J_i}, \zeta_{J_i})$ and an opening parameter $ost_{\mathcal{U}i} := \mathbf{R}_{J_i}$.
- $\mathcal{SH}_{\mathcal{U}}(pk, rst_{\mathcal{U}}, ost_{\mathcal{U}}, M, \Sigma, \mathbf{I}_{op})$: Parse $(\mathbf{R}_1, \dots, \mathbf{R}_K) := ost_{\mathcal{U}}.\mathbf{R}$. For all $i \in [K]$, compute $(\mathbf{L}_i^*, \mathbf{R}_i^*) := MT.O(rst_{\mathcal{U}}.C, ost_{\mathcal{U}}.\mathbf{L}, \mathbf{R}_i, \mathbf{I}_{op})$. Set $\mathbf{L}^* := (\mathbf{L}_1^*, \dots, \mathbf{L}_K^*)$ and $\pi := (\mathbf{R}_1^*, \dots, \mathbf{R}_K^*)$. Return $(\mathbf{L}^*, \pi)$.
- $\mathcal{SH}_{\mathcal{V}}(pk, M, \Sigma, \mathbf{L}^*, \mathbf{I}_{op}, \pi)$: Parse $(\sigma, \eta, \zeta) := \Sigma$, $(\zeta_1, \dots, \zeta_K) := \zeta$, $(\mathbf{L}_1^*, \dots, \mathbf{L}_K^*) := \mathbf{L}^*$, and $(\mathbf{R}_1^*, \dots, \mathbf{R}_K^*) := \pi$. Output 0 if $MT.CV(\zeta_i, (\mathbf{L}_i^*, \mathbf{R}_i^*), \mathbf{I}_{op}) = 0$ for an $i \in [K]$. If $\mathcal{V}(pk, M, \Sigma) = 0$ return 0, and 1 otherwise.
- $\mathcal{V}(pk, M, \Sigma)$: Parse $(\sigma, \eta, \zeta) := \Sigma$, $(\sigma_1, \dots, \sigma_K) := \sigma$, $(\eta_1, \dots, \eta_K) := \eta$, $(\zeta_1, \dots, \zeta_K) := \zeta$, and $(pk_1, \dots, pk_K) := pk$. Set $\sigma_0 := \eta_0 := \zeta_0 := \perp$ and $m_0 := M$. For $i \in [K]$, compute $m_i := H_m(\sigma_{i-1}, \eta_{i-1}, \zeta_{i-1}, m_{i-1})$, $\mu_i := H_\mu(m_i, \eta_i)$, and $\gamma_i := H_\gamma(pk_i, \mu_i, \zeta_i)$. If $BS.V(pk_i, \gamma_i, \sigma_i) = 1$ for all $i \in [K]$, return 1, and 0 otherwise.

C Deferred Full Proofs for Pachirisu

C.1 OMUF

Theorem 1 (ℓ -OMUF). *Let $H, H_r : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_{rnd} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be hash functions treated as random oracles. Let BS^H be an arbitrary three-move simulatable blind signature scheme with the properties described in Definition 4, and let $\mathbf{P}[BS^H]$ be the scheme resulting from applying our transform on BS^H . Assume there exists $\vartheta > 0$ and $f : \mathbb{N} \rightarrow \mathbb{N}$ with*

$$f(\text{ctr}) = \lceil 3 \cdot \vartheta \cdot \ln(\ell + 1) \cdot \text{ctr} \rceil.$$

Then for all adversaries $\mathcal{A}$ making at most $Q_H, Q_{rnd}, Q_{\Upsilon}, Q_{mt}$ queries to the oracles H, H_{rnd}, H_{Υ} and H_{MT} , respectively, and Q_{S_2} and Q_{S_3} queries to $\mathbf{P}[\mathbf{BS}^H].S_2$ and $\mathbf{P}[\mathbf{BS}^H].S_3$, respectively, and for all public parameters $\mathbf{pp}$ with a security parameter λ , and all non-negative integers ℓ , there exists $\delta > 0$ with $(1 - \delta) \cdot \vartheta > 1$, and a reduction $\mathcal{R}_3$ such that if $\mathbf{BS}$ is (t, ε) -0-OMUF, then $\mathbf{P}[\mathbf{BS}]$ is (t', ε') - ℓ -OMUF with $t' \approx t$ and

$$\varepsilon' \leq K \cdot (Q_H - \ell) \cdot \varepsilon + \epsilon_{cheat} + \epsilon_{ro} + \epsilon_{bs}.$$

where:

$$\begin{aligned} \epsilon_{cheat} &:= \frac{K \cdot \ell}{Q_H} + \exp(-\delta \cdot K), \\ \epsilon_{ro} &:= \frac{Q_{\Upsilon}^2 + Q_{rnd}^2 + Q_{mt}^2}{2^{\lambda+1}} + \frac{N \cdot Q_{S_3} \cdot Q_{\Upsilon}}{2^{\lambda}} + \frac{N \cdot Q_{S_2} \cdot Q_{rnd}}{2^{\lambda}}, \\ \epsilon_{bs} &:= \varepsilon_{SHL} \cdot K \cdot (\ell + 1) + \ell \cdot K \cdot (N - 1) \cdot (\varepsilon_{UUF} + \varepsilon_{UUO}) \\ &\quad + K \cdot \varepsilon_{UL} + K \cdot (Q_H - \ell) \cdot Q_H \cdot \varepsilon_{DS}. \end{aligned}$$

Simplifying Notation. Once $\mathcal{A}$ queries oracle S_1 , it starts a new signing session $\mathbf{ctr}$. A session $\mathbf{ctr}$ consists of K sequential subsessions $\mathbf{ctr}_1, \dots, \mathbf{ctr}_K$, where each subsession $(\mathbf{ctr}, i)$ is a sequence of $S_2 \rightarrow S_3 \rightarrow S_4$ calls with respect to the secret key $\mathbf{sk}_i$. A session $\mathbf{ctr}$ is successfully closed after all subsessions are completed successfully, that is, a S_4 query for the K -th subsession closes the respective session. Each valid message-signature pair (M, Σ) consists of K nested message-subsignature pairs $(\Upsilon_1, \sigma_1), \dots, (\Upsilon_K, \sigma_K)$ (see $\mathcal{V}$ of $\mathbf{P}[\mathbf{BS}]$), which are valid $\mathbf{BS}$ message-signature pairs.

We say that a $\mathbf{BS}$ signature Σ on a message Υ is linked to a hash query $H(Q)$ if the verification procedure $\mathbf{BS}.\mathcal{V}$ makes the same query Q to H (in the sense of Definition 2).

Proof. To rely on the technique described above, we need to show that it works even if $\mathcal{A}$ behaves maliciously. More specifically, $\mathcal{A}$ may cheat in multiple ways that render the technique above useless. Therefore, we proceed via a sequence of games to rule out some bad events.

Game $\mathbf{G}_0$. This is Game ℓ -OMUF $_{\mathbf{P}[\mathbf{BS}]}^{\mathcal{A}}$.

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event **BAD**₂, which is defined as follows:

BAD₂: $\mathcal{A}$ makes a S_2 query for some signing session $\mathbf{ctr}$ with a value com_j while $\Upsilon_{rnd}[\text{com}_j] = \perp$, and later $\mathcal{A}$ finds a preimage Urnd_j for com_j satisfying $H_{rnd}(\text{Urnd}_j) = \text{com}_j$. We modify the way the random oracle H_{rnd} is simulated to abort if the adversary submits a fresh query Q , $H_{rnd}(Q) = \text{com}_j$, and com_j was received by the simulator previously in an S_2 query made by $\mathcal{A}$.

Claim 5. Let Q_{rnd} be the maximum number of hash queries $\mathcal{A}$ may make to the oracle H_{rnd} , and let Q_{S_2} be the maximum number of S_2 queries $\mathcal{A}$ may make. Then it holds that

$$\text{Adv}_{\mathbf{G}_1}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_0}^{\mathcal{A}} - \frac{N \cdot Q_{S_2} \cdot Q_{\text{rnd}}}{2^\lambda}.$$

Proof. Since the output of the random oracle is uniformly random, the probability that a fresh hash query Q to H_{rnd} results in a desired fixed value $com_j \in \{0, 1\}^\lambda$ is at most $\frac{1}{2^\lambda}$. In the worst case, $\mathcal{A}$ may submit up to $N \cdot Q_{S_2}$ commitments since it makes up to Q_{S_2} queries and each query contains commitments $com_1, \dots, com_N$. Thus, for each H query $\mathcal{A}$ makes, the probability of finding the preimage of one of the submitted commitments is at most $\frac{N \cdot Q_{S_2}}{2^\lambda}$. As $\mathcal{A}$ makes up to Q_{rnd} queries, the probability of finding a preimage of some commitment com is at most $\frac{N \cdot Q_{S_2} \cdot Q_{\text{rnd}}}{2^\lambda}$.

Game $\mathbf{G}_2$. This game is identical to $\mathbf{G}_1$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event **BAD**₃:

BAD₃: $\mathcal{A}$ causes a collision in one of its queries to the random oracles H_{rnd} , $H_{\mathcal{I}}$, or H_{MT} . That is, for $ro \in \{\text{rnd}, \mathcal{I}, \text{mt}\}$, $\mathcal{A}$ sends a fresh query Q , such that $H_{ro}(Q) = H_{ro}(Q')$ for some previous query Q' .

Claim 6. Let Q_{ro} be the maximum number of hash queries $\mathcal{A}$ may make to the oracle H_{ro} for $ro \in \{\text{rnd}, \mathcal{I}, \text{mt}\}$. Then it holds

$$\text{Adv}_{\mathbf{G}_2}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_1}^{\mathcal{A}} - \frac{Q_{\text{mt}} \cdot (Q_{\text{mt}} - 1) + Q_{\mathcal{I}} \cdot (Q_{\mathcal{I}} - 1) + Q_{\text{rnd}} \cdot (Q_{\text{rnd}} - 1)}{2^{\lambda+1}}.$$

Proof. As the output of H_{ro} is uniformly random, it holds for any pair (Q_1, Q_2) that $\Pr[H_{ro}(Q_1) = H_{ro}(Q_2) \mid Q_1 \neq Q_2] \leq \frac{1}{2^\lambda}$. Assuming $\mathcal{A}$ makes at most Q_{ro} queries to the oracle H_{ro} , the number of pairs of inputs is $\frac{Q_{ro} \cdot (Q_{ro} - 1)}{2}$. Consequently, the probability that a pair causes a collision is at most $\frac{Q_{ro} \cdot (Q_{ro} - 1)}{2^{\lambda+1}}$. It follows that the probability of causing a collision in one of the oracles $H_{\mathcal{I}}$ and H_{rnd} is at most $\frac{Q_{\text{mt}} \cdot (Q_{\text{mt}} - 1) + Q_{\mathcal{I}} \cdot (Q_{\mathcal{I}} - 1) + Q_{\text{rnd}} \cdot (Q_{\text{rnd}} - 1)}{2^{\lambda+1}}$.

Per this game, for $ro \in \{\text{mt}, \mathcal{I}, \text{rnd}\}$, if $T_{ro}[Q] \neq \perp$, then there exists a unique value that can be extracted from $T_{ro}[Q]$.

Game $\mathbf{G}_3$. This game is identical to $\mathbf{G}_2$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event **BAD**₄, which is defined as follows:

BAD₄: $\mathcal{A}$ terminates and outputs a valid message-signature pair (M, Σ) for $\mathbf{P}[\text{BS}]$, which consists of K valid message-signature pairs $(\mathcal{I}_1, \sigma_1, \dots, \mathcal{I}_K, \sigma_K)$ for BS , and $\exists i \in [K]$ such that $\text{BS}.\mathcal{V}(\text{pk}_i, \mathcal{I}_i, \sigma_i)$ does not query the random oracle H with a query Q that was queried before to H , i.e. $T_H[Q] \neq \perp$, where T_H is the hash table of the random oracle H .

Claim 7.

$$\mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_2}^{\mathcal{A}} - \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1).$$

Proof. The verification of a valid BS signature using $\text{BS.V}(\text{pk}_i, \Upsilon_i, \sigma_i)$ makes a singular hash query Q to H , and the same hash query was made previously. Per Definition 2, $\mathcal{A}$ may provoke $\mathbf{BAD}_4$ for a single BS message-signature pair (Υ_i, σ_i) with probability at most ε_{SHL} . Since $\mathcal{A}$ outputs up to $\ell + 1$ $\mathbf{P}[\text{BS}]$ signatures, and each signature consists of K BS signatures, the probability that $\mathbf{BAD}_4$ occurs is at most $\varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1)$, hence the claim.

Game $\mathbf{G}_4$. This game is identical to $\mathbf{G}_3$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_5$, which is defined as follows:

$\mathbf{BAD}_5$: After $\mathcal{A}$ terminates and outputs a valid message-signature pair (M, Σ) for $\mathbf{P}[\text{BS}]$, which consists of K valid message-signature pairs $(\Upsilon_1, \sigma_1, \dots, \Upsilon_K, \sigma_K)$ for BS, and for all $i \in [K]$, $\text{BS.V}(\text{pk}_i, \Upsilon_i, \sigma_i)$ makes an H query Q_i with $\text{T}_\text{H}[Q_i] \neq \perp$ (Q_i has been queried previously), but $\exists j \in [K]$ such that Q_j is not of the form $(\dots, \Upsilon_j, \dots)$ (i.e. the signed message is not in the hash query).

Claim 8.

$$\mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} = \mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}}.$$

Proof. Per Definition 5, running $\text{V}(\text{pk}, \Upsilon, \sigma)$ makes a hash query to H , and this query contains the signed message Υ , i.e. the query is of the form $(\dots, \Upsilon, \dots)$.

Game $\mathbf{G}_5$. This game is identical to $\mathbf{G}_4$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_6$, which occurs if $\mathcal{A}$ generates a valid user message msgU without relying on the random oracle H .

$\mathbf{BAD}_6$: For a subsession (ctr, i) for $i \in [K]$, $\mathcal{A}$ makes an $\mathcal{S}_3$ query with user message $\{\text{msgU}_j\}_{j \in [N]}$, and then it makes an $\mathcal{S}_4$ query with input $\{\mathbf{R}_j, \mu_j, \text{Urnd}_j\}_{J_i \neq j \in [N]}$ for $J_i := \text{H}_J(\text{seed}, \text{pk}_i)$. The event occurs if, after $\mathcal{A}$ terminates, there exists $J_i \neq j' \in [N]$ satisfying the following:

1. Let $\zeta_{j'} := \text{MT.C}(\mathbf{L}; R_{j'}), \Upsilon_{j'} := \text{H}_T(\text{pk}_i, \mu_{j'}, \zeta_{j'})$, where $\mathbf{L}$ is the attribute vector used in the session ctr .
2. Let $\hat{\text{T}}$ be a snapshot of the hash table of H by the time $\mathcal{A}$ makes the $\mathcal{S}_4$ query.
3. Let Q be the unique hash query made by the execution of $\text{BS.U}_1(\text{pk}_i, \Upsilon_{j'}, \text{msgS1}_{j'}, \text{Urnd}_{j'})$.
4. It holds that $\text{msgU}_{j'} = \text{BS.U}_1(\text{pk}_i, \Upsilon_{j'}, \text{msgS1}_{j'}, \text{Urnd}_{j'})$ and $\hat{\text{T}}[Q] = \perp$.

Claim 9.

$$\mathbf{Adv}_{\mathbf{G}_5}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} - \varepsilon_{\text{UUF}} \cdot \ell \cdot K \cdot (N - 1).$$

Proof. We note that the occurrence of **BAD**₆ is a direct violation for the property described in Property 3, because $\mathcal{A}$'s output $(\text{msgU}'_j, \mathbf{R}_{j'}, \mu_{j'}, \text{Urnd}_{j'})$ can be directly transformed into an attack $(\text{pk}_i, \gamma_{j'}, \text{msgS1}_{j'}; \text{Urnd}_{j'})$ against the said property. The probability of this happening for each tuple $\mathcal{A}$ submits is per Property 3 at most ε_{UUF} . Since $\mathcal{A}$ makes up to $\ell \cdot K$ $\mathcal{S}_4$ queries, each containing $N - 1$ tuples, we take the union bound over the $\ell \cdot K \cdot (N - 1)$ tuples $\mathcal{A}$ outputs, which implies the claim.

Game G₆. This game is identical to **G₅**, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event **BAD**₇, which we define as follows:

BAD₇: $\mathcal{A}$ makes an $\mathcal{S}_3$ query for a session ctr and a subsession $i \in [K]$ with user messages $\{\text{msgU}_j\}_{j \in [N]}$. Then, for some $\text{H}_J(\text{seed}, \text{pk}_i) \neq j' \in [N]$, $\mathcal{A}$ makes an $\mathcal{S}_4$ query for the subsession (ctr, i) with input $(R_{j'}, \mu_{j'}, \text{Urnd}_{j'})$. Let $\zeta_{j'} := \text{MT}(\mathbf{L}; R_{j'})$, $\gamma_{j'} := \text{H}_\gamma(\text{pk}_i, \mu_{j'}, \zeta_{j'})$, where $\mathbf{L}$ is the attribute vector used in the session ctr , and it holds $\text{msgU}_{j'} = \text{BS.U}_1(\text{pk}_i, \gamma_{j'}, \text{msgS1}_{j'}; \text{Urnd}_{j'})$. Let $\widehat{\mathbf{T}}_{\text{H}}$ be a snapshot of the hash table of H at the time the $\mathcal{S}_4$ query is made. The event occurs if

1. there is no query $Q' \in \widehat{\mathbf{T}}_{\text{H}}$ such that Q' is of the form $(\dots, \gamma_{j'}, \dots)$, or
2. there is a query $Q \in \widehat{\mathbf{T}}_{\text{H}}$ such that Q is of the form $(\dots, \gamma, \dots)$, $\gamma \neq \gamma_{j'}$, and $\text{msgU}_{j'} = \text{BS.U}_1(\text{pk}_i, \gamma, \text{msgS1}_{j'}; \text{Urnd}_{j'})$.

Claim 10.

$$\text{Adv}_{\mathbf{G}_6}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_5}^{\mathcal{A}} - \ell \cdot K \cdot (N - 1) \cdot \varepsilon_{\text{UUO}}.$$

Proof. Since $\text{msgU}_{j'} = \text{BS.U}_1(\text{pk}_i, \gamma_{j'}, \text{msgS1}_{j'}; \text{Urnd}_{j'})$, $\mathcal{A}$ must have made a hash query to H identical to the hash query that would have been made by an honest $\text{BS.U}_1(\text{pk}_i, \gamma_{j'}, \text{msgS1}_{j'}; \text{Urnd}_{j'})$ execution; otherwise, Game **G₅** would have aborted. Additionally, this H query contains the message $\gamma_{j'}$ (due to Game **G₄**). Furthermore, there cannot be another query with a different message $\gamma \neq \gamma_{j'}$ satisfying $\text{msgU}_{j'} = \text{BS.U}_1(\text{pk}_i, \gamma, \text{msgS1}_{j'}; \text{Urnd}_{j'}) = \text{BS.U}_1(\text{pk}_i, \gamma_{j'}, \text{msgS1}_{j'}; \text{Urnd}_{j'})$ as this would violate the property described in Property 4, which occurs with probability at most ε_{UUO} . Taking the union bound over $\ell \cdot \mathbf{P}[\text{BS}]$ sessions, each having K subsessions, and $N - 1$ user message per subsession (excluding the cut-and-choose user message), this event occurs with probability at most $\ell \cdot K \cdot (N - 1) \cdot \varepsilon_{\text{UUO}}$.

Game G₇. This game is identical to **G₆**, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event **BAD**₈, which we define as follows:

BAD₈: $\mathcal{A}$ makes an $\mathcal{S}_3$ query for a session ctr with a user message msgU_j . Let $\widehat{\mathbf{T}}_{\gamma}$ be a snapshot of the hash table of H_{γ} at the time this $\mathcal{S}_3$ query is made. Then, $\mathcal{A}$ makes an $\mathcal{S}_4$ query for this session with input $(R_j, \mu_j, \text{Urnd}_j)$. Let

$\zeta_j := \text{MT.C}(\mathbf{L}; R_j), \mathcal{Y}_j := \text{H}_{\mathcal{R}}(\text{pk}, \mu_j, \zeta_j)$, where $\mathbf{L}$ is the attribute vector used in this signing session. Then it holds $\widehat{\text{T}}_{\mathcal{R}}[\mathcal{Y}_j] = \perp$ and $\text{msgU}_j = \text{BS.U}_1(\text{pk}, \mathcal{Y}_j, \text{msgS1}_j; \text{Urnd}_j)$.

Claim 11. Let $\mathbf{Q}_{\mathcal{R}}$ and $\mathbf{Q}_{\mathcal{S}_3}$ be the maximum number of hash queries $\mathcal{A}$ makes to $\text{H}_{\mathcal{R}}$ and $\mathcal{S}_3$, respectively. Then it holds

$$\text{Adv}_{\mathbf{G}_7}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_6}^{\mathcal{A}} - \frac{N \cdot \mathbf{Q}_{\mathcal{S}_3} \cdot \mathbf{Q}_{\mathcal{R}}}{2^\lambda}.$$

Proof. Due to Game $\mathbf{G}_6$, we know that the equality $\text{msgU}_j = \text{BS.U}_1(\text{pk}, \mathcal{Y}_j, \text{msgS1}_j; \text{Urnd}_j)$ implies that $\mathcal{Y}_j$ is the only message existing in the table of the oracle H , and no other message in that table satisfies this equality. Consequently, $\mathcal{Y}_j$ is identical to the value $\mathcal{A}$ used to generate msgU_j , because otherwise, $\mathcal{A}$ must have made an H with that message (due to Game $\mathbf{G}_5$). Since $\widehat{\text{T}}_{\mathcal{R}}[\mathcal{Y}_j] = \perp$, $\mathcal{A}$ managed to find the preimage $(\text{pk}, \mu_j, \zeta_j)$ of $\mathcal{Y}_j$ under $\text{H}_{\mathcal{R}}$. Similar to Claim 5, this occurs with probability at most $\frac{N \cdot \mathbf{Q}_{\mathcal{R}} \cdot \mathbf{Q}_{\mathcal{S}_3}}{2^\lambda}$.

At this point, we know that, for each subsession ctr_i that will be closed via $\mathcal{S}_4$ successfully and for all $j \neq J = \text{H}_J(\text{seed}, \text{pk}_i)$:

- Upon $\mathcal{S}_2$ queries, all com_j must have a corresponding entry Urnd_j in the table T_{rnd} and this is extractable by $\mathcal{R}_3$; otherwise, if no entry in T_{rnd} , $\mathcal{A}$ will fail to find a preimage for com_j (Game $\mathbf{G}_1$), hence it will fail to close $\mathcal{S}_4$.
- Upon $\mathcal{S}_4$ queries, $\mathcal{R}_3$ can obtain the message $\mathcal{Y}_j$, and it is the same message $\mathcal{A}$ used to generate the user message msgU_j received previously in the $\mathcal{S}_3$ query (Game $\mathbf{G}_6$).

$\mathcal{R}_3$ requires this to hold even for $j = J$ because it needs Urnd_J and $\mathcal{Y}_J$ to use $\text{STM}_{\mathcal{S}}$ to answer $\mathcal{A}$'s queries in the subsessions corresponding to the challenger's key. Since $\mathcal{A}$ does not send the openings for $\mathcal{Y}_J$ and com_J for any subsession, the argument above does not cover $j = J$. Concretely, $\mathcal{A}$ may guess the cut-and-choose index J and behaves maliciously with the respective values (e.g., by computing com_J and $\mathcal{Y}_J$ without relying on H_{rnd} and $\text{H}_{\mathcal{R}}$). In the following, we rule out that $\mathcal{A}$ successfully computes com_J or $\mathcal{Y}_J$ in this manner.

Define Event **BAD**₉ as follows:

BAD₉: $\forall i \in [K], \exists \text{ctr} \in \{2, \dots, \ell + 1\}$, such that in the subsession (ctr, i) , $\mathcal{A}$

- **either** queries $\mathcal{S}_2$ with a commitment com_{J_i} that was not generated using the random oracle H_{rnd} (i.e., $\text{T}_{\text{rnd}}[\text{com}_{J_i}] = \perp$ by the time of $\mathcal{S}_2$ query) and then closes this subsession (ctr, i) successfully by making $\mathcal{S}_4$ query,
- **or** queries $\mathcal{S}_3$ with a user message msgU_{J_i} that was generated by $\text{BS.U}_1(\dots, \mathcal{Y}_{J_i}, \dots)$, while no hash query in the table of H contains $\mathcal{Y}_{J_i}$, and then closes the subsession (ctr, i) successfully by making $\mathcal{S}_4$ query.

Note that **BAD**₉ can be efficiently checked: The first case can be determined by checking the hash table of H_{rnd} for every commitment submitted in an $\mathcal{S}_2$ query. The second case can be checked for $(\text{pk}_i, \text{msgU}_{J_i}, \text{msgS1}_{J_i})$ by searching the table of H for a query $Q = (\dots, \mathcal{T}, \dots)$ with a message $\mathcal{T}$ satisfying $\text{msgU}_{J_i} = \text{BS.U}_1(\text{pk}_i, \mathcal{T}, \text{msgS1}_{J_i}; \text{Urnd}_{J_i})$. Such a unique message exists; otherwise, Game $\mathbf{G}_6$ would have aborted.

Game $\mathbf{G}_8$. This game is the same as $\mathbf{G}_7$, except that it aborts and outputs 0 if Event **BAD**₉ occurs.

Claim 12. Recall that $f(\text{ctr}) = \lceil 3 \cdot \vartheta \cdot \ln(\ell + 1) \cdot \text{ctr} \rceil$ and $\delta > 0$ with $(1 - \delta) \cdot \vartheta > 1$. It holds

$$\text{Adv}_{\mathbf{G}_8}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_7}^{\mathcal{A}} - \exp(-\delta \cdot K).$$

Proof. Assume for the sake of contradiction that $\mathcal{A}$ provokes **BAD**₉. This means that for each key i , there exists a signing session ctr and a subsession (ctr, i) , in which $\mathcal{A}$ does the following: $\mathcal{A}$ chooses $j \in [N]$ arbitrarily and computes com_j or $\mathcal{T}_j$ without using the respective random oracle H_{rnd} or $H_{\mathcal{T}}$. We distinguish the following cases:

- **Case 1** ($\mathbf{j} \neq \mathbf{J}_i$). $\mathcal{A}$ does not guess the cut-and-choose index successfully, hence it has to open the commitment com_j and $\mathcal{T}_j$, which corresponds to finding the preimage of $H_{\mathcal{T}}$ (on $\mathcal{T}_j$) or H_{rnd} (on com_j). This is not possible because otherwise, Game $\mathbf{G}_1$ or $\mathbf{G}_7$ would have aborted.
- **Case 2** ($\mathbf{j} = \mathbf{J}_i$). $\mathcal{A}$ guesses successfully the cut-and-choose index J_i for key pk_i , hence it does not have to open the commitments com_j and $\mathcal{T}_j$.

Since Case 1 does not occur, we focus on Case 2. More specifically, we want to analyze the probability that $\mathcal{A}$ cheats with respect to all keys pk_i by guessing the cut-and-choose index J_i .

We first compute the expected number of cheats per key pk_i for $i \in [K]$.

Define independent Bernoulli random variables $\text{cheat}_{i,\text{ctr}}$ for $i = 1 \dots K$ and $\text{ctr} = 2 \dots \ell + 1$, such that $\text{cheat}_{i,\text{ctr}} = 1$ if $\mathcal{A}$ cheats by successfully guessing J_i in session ctr , and $\text{cheat}_{i,\text{ctr}} = 0$ otherwise. Let $\text{cheat}_i := \sum_{\text{ctr}=2}^{\ell+1} \text{cheat}_{i,\text{ctr}}$. Since J_i is uniformly random in $[N]$, it holds that $\Pr[\text{cheat}_{i,\text{ctr}} = 1] \leq \frac{1}{N}$. Consequently,

$$\mathbb{E}(\text{cheat}_i) \leq \sum_{\text{ctr}=2}^{\ell+1} \frac{1}{f(\text{ctr})} = \frac{1}{3 \cdot \vartheta \cdot \ln(\ell + 1)} \cdot \sum_{\text{ctr}=2}^{\ell+1} \frac{1}{\text{ctr}} \leq \frac{\ln(\ell + 1)}{3 \cdot \vartheta \cdot \ln(\ell + 1)} = \frac{1}{3 \cdot \vartheta}.$$

The second inequality follows from the fact that $\sum_{i=1}^N \frac{1}{i} \leq \ln(N) + 1$. Substituting $X := \text{cheat}_i$ and $s := 3 \cdot \mathbb{E}(\text{cheat}_i) + \delta = \frac{1}{\vartheta} + \delta$ in Lemma 1 yields that for any $i \in [K]$

$$\Pr[\text{cheat}_i \geq \frac{1}{\vartheta} + \delta] \leq \exp(3 \cdot \mathbb{E}(\text{cheat}_i) - s) = \exp(-\delta). \quad (1)$$

Note that this equation states the probability of $\mathcal{A}$ successfully cheating ≥ 1 times for $\mathbf{pk}_i$. That is, $\text{cheat}_i < \frac{1}{\vartheta} + \delta \Rightarrow \text{cheat}_i < 1$ because we defined $\vartheta > 0$ and $\delta > 0$ as $(1 - \delta) \cdot \vartheta > 1 \Leftrightarrow 1 > \delta + \frac{1}{\vartheta}$. This means that for any key $\mathbf{pk}_i$, the probability that $\mathcal{A}$ cheats successfully is at most $\exp(-\delta)$. Thus, the probability that $\mathcal{A}$ provokes Event **BAD**₉ by cheating for all keys $\mathbf{pk}_i$ successfully is at most

$$\prod_{i \in [K]} \Pr[\text{cheat}_i \geq \frac{1}{\vartheta} + \delta] \leq \exp(-\delta \cdot K),$$

hence the claim.

Now, we want to fix a uniformly random index $\iota \in [K]$. We want to rule out that $\mathcal{A}$ cheats for $\mathbf{pk}_\iota$ (this key will be the challenger's key in our reduction)

*Game **G**₉.* Identical to **G**₈, except that it aborts and outputs 0 if Even **BAD**₁₀ occurs:

BAD₁₀: *The event occurs if $\mathcal{A}$ cheats for $\mathbf{pk}_\iota$ (i.e., $\text{cheat}_\iota \geq 1$).*

Claim 13.

$$\text{Adv}_{\mathbf{G}_9}^{\mathcal{A}} \geq \frac{1}{K} \cdot \text{Adv}_{\mathbf{G}_8}^{\mathcal{A}}.$$

Proof. Due to Game **G**₈, there exists $i^* \in [K]$ such that $\mathcal{A}$ does not cheat for $\mathbf{pk}_{i^*}$ ($\text{cheat}_{i^*} < 1$). Since $\iota \in [K]$ is uniformly random and hidden from $\mathcal{A}$, the probability that $i^* = \iota$ is at least $\frac{1}{K}$.

*Game **G**₁₀.* This game is identical to **G**₉, except that it aborts if $\mathcal{A}$ provokes Event **BAD**₁₁.

BAD₁₁: *The game samples $\kappa \leftarrow_{\$} [\mathbf{Q}_H]$, where $\mathbf{Q}_H$ is an upper bound for the number of hash queries $\mathcal{A}$ makes to the oracle H . During the online phase, $\mathcal{A}$ makes an $\mathcal{S}_4$ query for the ι -th subsession of a session ctr , for which $\mathcal{A}$ made $\mathcal{S}_3$ query with input $\{\text{msgU}_j\}_{j \in [N]}$, and the singular hash query made during the computation of $\text{msgU}_{J_\iota} = \text{BS.U}_1(\mathbf{pk}_\iota, \mathcal{I}_{J_\iota}, \text{msgS1}_{J_\iota}; \text{Urnd}_{J_\iota})$ is the κ -th hash query made to H .*

Claim 14.

$$\text{Adv}_{\mathbf{G}_{10}}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_9}^{\mathcal{A}} - \frac{\ell}{\mathbf{Q}_H}.$$

Proof. Since $\kappa \in [\mathbf{Q}_H]$ is uniformly random and hidden from $\mathcal{A}$, the probability that $\mathcal{A}$ guesses and uses the κ -th hash query in the ι -th subsession to provoke **BAD**₁₁ is $\frac{1}{\mathbf{Q}_H}$. As $\mathcal{A}$ may close up to ℓ sessions, the probability that it uses the κ -th query for at least one of them is $\frac{\ell}{\mathbf{Q}_H}$.

Extracting the Keys from H Queries. Note that the definition of three-move simulatable blind signatures (Section 4) does not enforce honest hash queries (e.g., those queries sent by $\text{BS}\mathcal{U}_1$ during signing) to include the public key of the signer. Without the ability to extract the key from the corresponding (honest) hash query and to program the oracle H respectively, it is not clear how to complete this proof. For this reason, our transform embeds the key inside the message being signed. Since the message must be included in the hash (otherwise Game $\mathbf{G}_4$ aborts), it remains to show how the oracle H exploits the message to extract the key from honest queries. Given a query Q , the simulator of H does the following:

- Parse $(\dots, \Upsilon, \dots) \leftarrow Q$, and abort if the parsing fails (recall that queries sent by $\text{BS}\mathcal{U}_1$ have this format due to Game $\mathbf{G}_4$).
- Look Υ up in Table $\mathbf{T}_\Upsilon$ and extract the corresponding query $Q_\Upsilon := \mathbf{T}_\Upsilon^{-1}[\Upsilon]$. If Q_Υ is not of the form $(\text{pk}_i, \mu, \zeta)$, abort.

Note that if the procedure aborts, the query is not honest (e.g., it corresponds to some arbitrary query made by $\mathcal{A}$) and does not contain a public key.

Game $\mathbf{G}_{11}$. This game is identical to $\mathbf{G}_{10}$, except that it aborts if $\mathcal{A}$ provokes Event $\mathbf{BAD}_{12}$, which we define as follows.

$\mathbf{BAD}_{12}$: *The event occurs if the following holds:*

1. *The game simulates everything as in $\mathbf{G}_{10}$, but it extracts the key from honest queries as described above and maintains a counter to track the number of queries per key.*
2. *$\mathcal{A}$ terminates and outputs $\ell+1$ valid $\mathbf{P}[\text{BS}]$ message-signature pairs $(M_1, \Sigma_1), \dots, (M_{\ell+1}, \Sigma_{\ell+1})$.*
3. *For all $t \in [\ell+1]$:*
 - a. *Parse $(\sigma_t, \eta_t, \zeta_t) := \Sigma_t$. Then, parse $(\sigma_1^{(t)}, \dots, \sigma_K^{(t)}) := \sigma_t$, $(\eta_1^{(t)}, \dots, \eta_K^{(t)}) := \eta_t$, and $(\zeta_1^{(t)}, \dots, \zeta_K^{(t)}) := \zeta_t$.*
 - b. *Set $\sigma_0^{(t)} := \eta_0^{(t)} := \zeta_0^{(t)} := \perp$, $m_0^{(t)} := M_t$.*
 - c. *For all $i = 1 \dots \iota$, compute $m_i^{(t)} := \text{H}_m(\sigma_{i-1}^{(t)}, \eta_{i-1}^{(t)}, \zeta_{i-1}^{(t)}, m_{i-1}^{(t)})$, $\mu_i^{(t)} := \text{H}_\mu(m_i^{(t)}, \eta_i^{(t)})$, $\Upsilon_i^{(t)} := \text{H}_\Upsilon(\text{pk}_i, \mu_i^{(t)}, \zeta_i^{(t)})$.*
 - d. *It holds $\text{BS}.\mathcal{V}^{\text{H}}(\text{pk}_\iota, \Upsilon_\iota^{(t)}, \sigma_\iota^{(t)}) = 1$, and the singular hash query made by $\text{BS}.\mathcal{V}$ (Game $\mathbf{G}_3$) is not the κ -th H query.*

Claim 15.

$$\text{Adv}_{\mathbf{G}_{11}}^{\mathcal{A}} \geq \frac{1}{Q_{\text{H}} - \ell} \cdot \text{Adv}_{\mathbf{G}_{10}}^{\mathcal{A}} - Q_{\text{H}} \cdot \varepsilon_{\text{DS}}.$$

Proof. First, due to Game $\mathbf{G}_4$, the hash query must contain the signed message Υ , hence $\mathbf{G}_{11}$ can identify the occurrence of $\mathbf{BAD}_{12}$. Second, we recall Definition Property 7 that implies that each hash query may be linked to at most a single signature (except with probability at most ε_{DS}). $\mathcal{A}$ can distinguish two types of queries:

- **Type 1** Queries used in honest $\text{BS}\mathcal{U}_1$ executions, and the resulting user messages msgU were sent to the signer via $\mathcal{S}_3$ queries, and these sessions were closed later via $\mathcal{S}_4$ queries. These are ℓ queries.
- **Type 2** The remainder of the queries $\mathcal{A}$ makes to H . These are $Q_H - \ell$ queries.

Due to Game $\mathbf{G}_{10}$, the κ -th query belongs to the second type; otherwise, the game would have aborted. Thus, $\mathcal{A}$'s best strategy to provoke $\mathbf{BAD}_{12}$ is to link ℓ forgeries to the queries of the first type, and the remaining forgery to a randomly chosen query of the second type. In this case, $\mathbf{G}_{11}$ aborts with probability at most $1 - \frac{1}{Q_H - \ell}$, hence $\text{Adv}_{\mathbf{G}_{11}}^{\mathcal{A}} \geq \frac{1}{Q_H - \ell} \cdot \text{Adv}_{\mathbf{G}_{10}}^{\mathcal{A}} - \varepsilon_{\text{DS}}$.

The following game hop happens only when the underlying scheme is a three-move scheme.

Game $\mathbf{G}_{12}$. This game is identical to $\mathbf{G}_{11}$, except that it aborts if $\mathcal{A}$ provokes Event $\mathbf{BAD}_{13}$, which we define as follows.

$\mathbf{BAD}_{13}$: *$\mathcal{A}$ makes a hash query to H that has the format of honest hash queries, and this query can be explained via two distinct tuples $(\text{msgS1}, \text{Urnd})$ and $(\text{msgS1}', \text{Urnd}')$ that were provided previously by $\mathcal{A}$ for the same message-key pair $(\mathcal{T}, \text{pk})$, i.e. $\text{hq}(\text{pk}, \mathcal{T}, \text{msgS1}, \text{Urnd})$ and $\text{hq}(\text{pk}, \mathcal{T}, \text{msgS1}', \text{Urnd}')$, where hq is the function defined in Property 1.*

Claim 16.

$$\text{Adv}_{\mathbf{G}_{12}}^{\mathcal{A}} \geq \frac{1}{Q_H - \ell} \cdot \text{Adv}_{\mathbf{G}_{11}}^{\mathcal{A}} - \varepsilon_{\text{UL}}.$$

Proof. First, we note that the occurrence of $\mathbf{BAD}_{13}$ can be checked efficiently, since for every hash query, we know that the query contains signed message $\mathcal{T}$, and the corresponding public key can be extracted from the table of $\text{H}_{\mathcal{T}}$ as described above. Having pk and $\mathcal{T}$, $\mathbf{G}_{12}$ searches all $(\text{msgS1}, \text{Urnd})$ and checks if two combinations $(\text{pk}, \mathcal{T}, \text{msgS1}, \text{Urnd})$ and $(\text{pk}, \mathcal{T}, \text{msgS1}', \text{Urnd}')$ explain the incoming query. Provoking $\mathbf{BAD}_{13}$ requires $\mathcal{A}$ to open two signing subsessions (ctr, i) and (ctr', i) for $i \in [K]$, such that the signing algorithm generates two BS signer messages msgS1 and $\text{msgS1}'$ for a public key pk_i such that these signer messages along with the user randomness Urnd and Urnd' satisfy $\text{hq}(\text{pk}_i, \mathcal{T}, \text{msgS1}, \text{Urnd}) = \text{hq}(\text{pk}_i, \mathcal{T}, \text{msgS1}', \text{Urnd}')$. This, however, directly violates Property 6, which must hold except with probability ε_{UL} .

Reducing 0-OMUF_{BS} to $\mathbf{G}_{12}$. In the following we describe our reduction $\mathcal{R}_3$.

- **Setup Phase.** $\mathcal{R}_3$ takes as input the 0-OMUF_{BS} challenger's public key pk' , samples $\iota \leftarrow_{\S} [K]$, sets $\text{pk}_{\iota} := \text{pk}'$, samples $(\text{sk}_i, \text{pk}_i) \leftarrow_{\S} \text{BS.G}$ for $i \in [K] \setminus \{\iota\}$, sets $\text{pk} := (\text{pk}_1, \dots, \text{pk}_K)$. $\mathcal{R}_3$ initializes a global counter for completed sessions $\text{closed} := 0$, a global session ID counter $\text{ctr} := 1$, and a state map state . $\mathcal{R}_3$ then invokes $\mathcal{A}(\text{pk})$.

- **Online Phase.** $\mathcal{R}_3$ simulates the signer oracles and the random oracles for $\mathcal{A}$. $\mathcal{A}$ may query the oracles concurrently and in an interleaved manner. $\mathcal{A}$ may complete at most ℓ sessions. $\mathcal{R}_3$ simulates the oracles as follows:
 - $\mathcal{RG}_S(\mathbf{L})$. Exactly as described by Construction 6.
 - $\mathcal{S}_1(\mathbf{L})$. If $\mathbf{L}$ is not registered, abort and output $\perp$. Increment $\text{ctr} := \text{ctr} + 1$, and set $N := f(\text{ctr})$. For $i \in [K]$, sample $\text{seed}_i \leftarrow_{\$} \{0,1\}^\lambda$, compute $\text{com}_{\text{seed}_i} := \text{H}_J(\text{seed}_i)$, compute $J_i := \text{H}_J(\text{seed}_i, \text{pk}_i)$. Store $(\mathbf{L}, N, \{\text{seed}_i, J_i\}_{i \in [K]}, i := 1, \text{round} := 1)$ in $\text{state}[\text{ctr}]$. Return $(\text{ctr}, N, \{\text{com}_{\text{seed}_i}\}_{i \in [K]})$.
 - $\mathcal{S}_2(\text{sk}, \text{ctr}, \{\text{com}_j\}_{j \in [N]})$. Retrieve (i, round) from $\text{state}[\text{ctr}]$. Abort and output $\perp$ if $\text{round} \neq 1$. Parse $\text{sk} := (\text{sk}_1, \dots, \text{sk}_K)$.
 - * **If** $(i = \iota)$: Set $\text{cheat} := 0$. Compute $(\text{msgS1}_j, \text{simSt}_j) \leftarrow_{\$} \text{BS.SIM}_{\mathcal{S}_1}(\text{pk}_i)$. Store $\text{msgS1}_{i,j} := \text{msgS1}_j$, $\text{simSt}_{i,j} := \text{simSt}_j$, $\text{Urnd}'_{i,j} := \text{Urnd}'_j$, $\text{com}_{i,j} := \text{com}_j$ in $\text{state}[\text{ctr}]$.
 - * **Else** $(i \neq \iota)$: For $j \in [N]$: compute $(\text{msgS1}_j, \text{rnd}_{\mathcal{S}_j}) \leftarrow_{\$} \text{BS.S1}(\text{sk}_i)$. Update the state and store $\text{msgS1}_{i,j} := \text{msgS1}_j$, $\text{rnd}_{\mathcal{S}_{i,j}} := \text{rnd}_{\mathcal{S}_j}$, $\text{com}_{i,j} := \text{com}_j$ in $\text{state}[\text{ctr}]$.
 Store $(\text{round} := 2, \text{cheat})$ in $\text{state}[\text{ctr}]$ (if ‘cheat’ was defined). Return $\{\text{msgS1}_j\}_{j \in [N]}$.
 - $\mathcal{S}_3(\text{ctr}, \{\text{msgU}_j\}_{j \in [N]})$. Retrieve $(i, \text{round}, \text{seed})$ from $\text{state}[\text{ctr}]$. Abort and output $\perp$ if $\text{round} \neq 2$. For $j \in [N]$, store $\text{msgU}_{i,j} := \text{msgU}_j$ and $\text{round} := 3$ in $\text{state}[\text{ctr}]$. Return seed_i .
 - $\mathcal{S}_4(\text{sk}, \text{ctr}, \{R_j, \mu_j, \text{Urnd}_j\}_{j \neq J_i})$. Retrieve $(\mathbf{L}, N, \{J_i\}_{i \in [K]}, i, \text{round}, \text{cheat}, \text{msgS1}, \text{rnd}_{\mathcal{S}_{i,j}}, \text{simSt}_{i,j}, \text{msgU}_{i,j})$ from $\text{state}[\text{ctr}]$. Retrieve $\text{Urnd}'_{i,j}$ from the table $\mathbf{T}_{\text{com}}$. If BS is a two-round scheme, find the (unique due to $\mathbf{G}_6$) unassigned tuple $(\text{pk}, \mathcal{Y}, \text{simSt})$ in state such that $\text{msgU}_{i,j} = \mathcal{U}_1(\text{pk}, \mathcal{Y}, \perp, \text{Urnd}_{i,j})$ and set $\text{simSt}_{i,j} := \text{simSt}$. Abort and output $\perp$ if $\text{round} \neq 3$ or $\text{closed} = \ell$. Parse $\text{sk} := (\text{sk}_1, \dots, \text{sk}_K)$. For $j \in [N] \setminus \{J_i\}$:
 1. Abort and output $\perp$ if $\text{com}_j \neq \text{H}_{\text{rnd}}(\text{Urnd}_j)$.
 2. Compute $\zeta_j := \text{MT.C}(\mathbf{L}; R_j)$ and $\mathcal{Y}_j := \text{H}_{\mathcal{Y}}(\text{pk}_i, \mu_j, \zeta_j)$.
 3. Abort and output $\perp$ if $\text{msgU}_{i,j} \neq \text{BS.U}_1(\text{pk}_i, \mathcal{Y}_j, \text{msgS1}_{i,j}; \text{Urnd}_j)$.**If** $(i = \iota)$:
 - * If the hash query corresponding to $\text{Urnd}_{i,j}$ (as described in Game $\mathbf{G}_5$) was generated by calling the challenger’s oracle H_{ch} (see the simulation of H when $\text{ctr}_{\text{H}} = \kappa$), abort and output $\perp$ (this is excluded in Game $\mathbf{G}_{10}$).
 - * Search $\mathbf{T}_{\mathcal{Y}}$ for $\mathcal{Y}_{J_i}$ such that $\text{BS.U}_1(\text{pk}_i, \mathcal{Y}_{J_i}, \text{msgS1}_{i,J_i}; \text{Urnd}'_{i,J_i}) = \text{msgU}_{i,J_i}$. If $\mathcal{Y}_{J_i} = \perp$ or $\text{cheat} = 1$, abort and output $\perp$ (this is excluded in Game $\mathbf{G}_8$).
 - * Generate the signer second message $\text{msgS2}_i \leftarrow \text{BS.SIM}_{\mathcal{S}_2}(\text{msgS1}_{i,J_i}, \text{simSt}_{i,J_i}, \text{msgU}_{i,J_i}, \mathcal{Y}_{J_i}, \text{Urnd}'_{i,J_i})$.**Else** $(i \neq \iota)$:
 - * Compute $\text{msgS2}_i \leftarrow \text{BS.S2}(\text{sk}_i, \text{rnd}_{\mathcal{S}_{i,J_i}}, \text{msgU}_{i,J_i})$.
 - * **If** $(i = K)$: Increment $\text{closed} := \text{closed} + 1$ and store $\text{round} := 4$ in $\text{state}[\text{ctr}]$.
 - * **Else** $(i < K)$: Store $(i := i + 1, \text{round} := 1)$ in $\text{state}[\text{ctr}]$.

Return msgS2_i .

- $[\text{H}_{\text{rnd}}, \text{H}_{\mathcal{Y}}](Q)$. Simulate via lazy sampling, but abort if $\mathcal{A}$ finds a preimage (ruled out in Games $\mathbf{G}_1$ and $\mathbf{G}_7$) or a collision (ruled out in Game $\mathbf{G}_2$).
- $[\text{H}_{\mu}, \text{H}_{\text{MT}}, \text{H}_{\mathcal{J}}, \text{H}_m](Q)$. Via lazy sampling.
- $\text{H}(Q)$. Intuitively, the oracle attempts to parse the query Q as it would do with an honest hash query sent by $\text{BS}.\mathcal{U}_1(\text{pk}_i, \dots)$. If the parsing fails (i.e., Q is not an honest query), it answers via lazy sampling. Otherwise, it extracts the public key pk_i from the query. If this key is not the challenger's key pk_{ι} , it answers via lazy sampling. Otherwise, i.e., $i = \iota$, it answers the queries using the signer simulation algorithm $\text{SIM}_{\mathcal{S}}$, except for one random query (the κ -th), it forwards the query to the challenger's oracle and answers with $\text{H}_{\text{ch}}(Q)$. Concretely, the oracle behaves as follows:
 - * Initially, sample an index $\kappa \leftarrow_{\S} [\mathcal{Q}]$ uniformly at random and start a counter $\text{ctr}_{\text{H}} := 0$ that increments by one after each query.
 - * If $\text{T}[Q] \neq \perp$, return $\text{T}[Q]$.
 - * Parse $(\dots, \mathcal{Y}, \dots) \leftarrow Q$ (recall that queries sent by $\text{BS}.\mathcal{U}_1$ have this format due to Game $\mathbf{G}_4$). If the parsing fails, answer via lazy sampling.
 - * Look up $\mathcal{Y}$ in Table $\text{T}_{\mathcal{Y}}$ and extract the corresponding query $Q_{\mathcal{Y}} \leftarrow \text{T}_{\mathcal{Y}}^{-1}[\mathcal{Y}]$. If $Q_{\mathcal{Y}} = \perp$, simulate via lazy sampling (this corresponds to a message that will not be used to close any signing session due to Game $\mathbf{G}_7$).
 - * If $Q_{\mathcal{Y}}$ is not of the form $(\text{pk}_i, \mu, \zeta)$ or $i \neq \iota$, simulate via lazy sampling.
 - * If $\text{ctr}_{\text{H}} = \kappa$, set $\text{T}[Q] := \text{H}_{\text{ch}}(Q)$, and return $\text{T}[Q]$.
 - * Otherwise ($\kappa \neq \text{ctr}_{\text{H}}$). Knowing which signing session this query belongs to (due to Game $\mathbf{G}_{12}$), identify the value $(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, \mathcal{Y})$ corresponding to that session. Let $\text{T}[Q] \leftarrow_{\S} \text{SIM}_{\text{H}}(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, \mathcal{Y})$, and return $\text{T}[Q]$. In the case that the underlying scheme BS is a two-move scheme, identify $(\text{pk}, \mathcal{Y})$ and run SIM_{H} to obtain c and simSt . Let $\text{T}[Q] = c$ and store $(\text{pk}, \mathcal{Y}, \text{simSt})$ in state, not yet assigned to any session.

- **Winning 0-OMUF $_{\text{BS}}^{\mathcal{R}_3}$** . When $\mathcal{A}$ terminates and outputs $\ell+1$ valid message-signature pairs $(M_1, \Sigma_1), \dots, (M_{\ell+1}, \Sigma_{\ell+1})$ for our transform $\mathbf{P}[\text{BS}]$, the reduction $\mathcal{R}_3$ searches for a *sub-signature* σ_{ι} on a message $\mathcal{Y}_{\iota}$ such that $\text{BS}.\mathcal{V}^{\text{Hch}}(\text{pk}_{\iota}, \mathcal{Y}_{\iota}, \sigma_{\iota}) = 1$. More specifically, $\mathcal{R}_3$ does the following. For all $t \in [\ell+1]$:

1. Parse $(\sigma_t, \eta_t, \zeta_t) \leftarrow \Sigma_t$.
2. Parse $(\sigma_1^{(t)}, \dots, \sigma_K^{(t)}) \leftarrow \sigma_t$, $(\eta_1^{(t)}, \dots, \eta_K^{(t)}) \leftarrow \eta_t$, and $(\zeta_1^{(t)}, \dots, \zeta_K^{(t)}) \leftarrow \zeta_t$.
3. Set $\sigma_0^{(t)} := \eta_0^{(t)} := \zeta_0^{(t)} := \perp$, $m_0^{(t)} := M_t$.
4. For $i = 1 \dots \iota$:
 - a. Compute $m_i^{(t)} := \text{H}_m(\sigma_{i-1}^{(t)}, \eta_{i-1}^{(t)}, \zeta_{i-1}^{(t)}, m_{i-1}^{(t)})$, $\mu_i^{(t)} := \text{H}_{\mu}(m_i^{(t)}, \eta_i^{(t)})$, $\mathcal{Y}_i^{(t)} := \text{H}_{\mathcal{Y}}(\text{pk}_i, \mu_i^{(t)}, \zeta_i^{(t)})$.

b. If $\text{BS}.\mathcal{V}^{\text{Hch}}(\text{pk}_l, \gamma_l^{(t)}, \sigma_l^{(t)}) = 1$: Abort and output the forgery (γ_l, σ_l) . Obviously, this is a valid forgery against $\text{0-OMUF}_{\text{BS}}$ allowing $\mathcal{R}_3$ to win the game. Given that Game $\mathbf{G}_{11}$ does not abort, $\mathcal{R}_3$ can find a subsignature for the challenger's public key pk_l that is valid under the κ -th hash query (the one programmed using the challenger's oracle).

Overall, we have:

$$\begin{aligned}
\text{Adv}_{\mathbf{P}[\text{BS}]}^{\ell\text{-OMUF}}(\mathcal{A}) &:= \text{Adv}^{\mathbf{G}_0} \\
&\leq \text{Adv}^{\mathbf{G}_1} + \frac{N \cdot \mathbf{Q}_{S_2} \cdot \mathbf{Q}_{\text{rnd}}}{2^\lambda} \\
&\leq \text{Adv}^{\mathbf{G}_2} + \frac{\mathbf{Q}_T^2 + \mathbf{Q}_{\text{rnd}}^2 + \mathbf{Q}_{\text{mt}}^2}{2^{\lambda+1}} + \frac{N \cdot \mathbf{Q}_{S_2} \cdot \mathbf{Q}_{\text{rnd}}}{2^\lambda} \\
&\leq \text{Adv}^{\mathbf{G}_3} + \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \epsilon'_{\text{ro}} \\
&\leq \text{Adv}^{\mathbf{G}_4} + \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \epsilon'_{\text{ro}} \\
&\leq \text{Adv}^{\mathbf{G}_5} + \varepsilon_{\text{UUF}} \cdot \ell \cdot K \cdot (N - 1) + \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \epsilon'_{\text{ro}} \\
&\leq \text{Adv}^{\mathbf{G}_6} + (\varepsilon_{\text{UUF}} + \varepsilon_{\text{UUO}}) \cdot \ell \cdot K \cdot (N - 1) + \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \epsilon'_{\text{ro}} \\
&\leq \text{Adv}^{\mathbf{G}_7} + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq \text{Adv}^{\mathbf{G}_8} + \exp(-\delta \cdot K) + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq K \cdot \text{Adv}^{\mathbf{G}_9} + \exp(-\delta \cdot K) + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq K \cdot \left(\text{Adv}^{\mathbf{G}_{10}} + \frac{\ell}{\mathbf{Q}_H} \right) + \exp(-\delta \cdot K) + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq K \cdot \left(\text{Adv}^{\mathbf{G}_{12}} + \varepsilon_{\text{UL}} \right) + \epsilon_{\text{cheat}} + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq K \cdot (\mathbf{Q}_H - \ell) \cdot \left(\text{Adv}^{\mathbf{G}_{11}} + \mathbf{Q}_H \cdot \varepsilon_{\text{DS}} \right) + K \cdot \varepsilon_{\text{UL}} + \epsilon_{\text{cheat}} + \epsilon_{\text{bad}} + \epsilon_{\text{ro}} \\
&\leq K \cdot (\mathbf{Q}_H - \ell) \cdot \text{Adv}_{\text{BS}}^{\text{0-OMUF}}(\mathcal{R}_3) + \epsilon_{\text{cheat}} + \epsilon_{\text{ro}} + \epsilon_{\text{bs}}.
\end{aligned}$$

where ϵ'_{ro} is the partial sum of random oracle terms before adding the pre-image resistance of H_T , and $\epsilon_{\text{bad}} = \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1) + \ell \cdot K \cdot (N - 1) \cdot (\varepsilon_{\text{UUF}} + \varepsilon_{\text{UUO}})$.

C.2 Full Proof of Secure Showing

Theorem 2. *Let BS be a blind signature scheme satisfying the properties of definition 4. Let $\mathbf{P}[\text{BS}]$ be the scheme resulting from applying our transform on BS. If $\mathbf{P}[\text{BS}]$ is (t, ε) - ℓ -OMUF, then $\mathbf{P}[\text{BS}]$ is (t', ε') - ℓ -SH with $t' \approx t$ and*

$$\varepsilon' \leq K \cdot \text{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_4) + \varepsilon.$$

Proof. Assume there is an adversary $\mathcal{A}$ winning Game ℓ -SH by outputting ℓ pairs $(\text{rst}_{\text{U}}^{\text{ctr}}, \mathbf{L}'_{\text{ctr}})$ along with ℓ tuples $(M_{\text{ctr}}, \Sigma_{\text{ctr}}, \mathbf{L}_{\text{ctr}}, I_{\text{op,ctr}}, \pi_{\text{ctr}})$ for $\text{ctr} \in [\ell]$ satisfying the constraints defined in Game ℓ -SH.

Recall that in our transform, the user does not send a commitment of its attributes to the signer in the registration phase, instead it sends the attribute vector in plain $\mathbf{L}$. Therefore, the winning condition regarding the pair $(\text{rst}_{\text{U}}^{\text{ctr}}, \mathbf{L}'_{\text{ctr}})$ requires $\mathcal{A}$ to only output the vectors $\mathbf{L}'_{\text{ctr}}$ if $\mathcal{A}$ opens the ctr -th signing session.

Akin to the one-more unforgeability proof, we want to show that the adversary cannot cheat for all keys $\mathbf{pk}_i$ for $i \in [K]$. That is, there must exist some public key $\mathbf{pk}_i$, for which $\mathcal{A}$ does not cheat at all. We note that the simulation of both games ℓ -OMUF and ℓ -SH is identical, since the simulator must simulate at most ℓ signing sessions for $\mathcal{A}$, in addition to the signer registration oracle and the random oracles. Thus, we use the same game hops we used in the OMUF proof to rule out the same bad events ruled out in the OMUF proof.

We proceed via game series.

Game $\mathbf{G}_0$. This is Game ℓ -SH.

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_2$.

Claim 17. Let Q_{rnd} be the maximum number of hash queries $\mathcal{A}$ may make to the oracle H_{rnd} , and let Q_{S_2} be the maximum number of S_2 queries $\mathcal{A}$ may make. Then it holds that

$$\mathbf{Adv}_{\mathbf{G}_1}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_0}^{\mathcal{A}} - \frac{N \cdot Q_{S_2} \cdot Q_{\text{rnd}}}{2^\lambda}.$$

The proof is analogous to the proof of Claim 5.

Game $\mathbf{G}_2$. This game is identical to $\mathbf{G}_1$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_3$.

Claim 18.

$$\mathbf{Adv}_{\mathbf{G}_2}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_1}^{\mathcal{A}} - \frac{Q_{\text{r}} \cdot (Q_{\text{r}} - 1) + Q_{\text{rnd}} \cdot (Q_{\text{rnd}} - 1)}{2^{\lambda+1}}.$$

The proof is analogous to the proof of Claim 6.

Game $\mathbf{G}_3$. This game is identical to $\mathbf{G}_2$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_4$.

Claim 19.

$$\mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_2}^{\mathcal{A}} - \varepsilon_{\text{SHL}} \cdot K \cdot (\ell + 1).$$

The proof is analogous to the proof of Claim 7.

Game $\mathbf{G}_4$. This game is identical to $\mathbf{G}_3$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_5$.

Claim 20.

$$\mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} = \mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}}$$

The proof is analogous to the proof of Claim 8.

Game $\mathbf{G}_5$. This game is identical to $\mathbf{G}_4$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_7$.

Claim 21.

$$\mathbf{Adv}_{\mathbf{G}_5}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} - \varepsilon_{\text{UUF}} \cdot \ell \cdot K \cdot (N - 1).$$

The proof is analogous to the proof of Claim 9

Game $\mathbf{G}_6$. This game is identical to $\mathbf{G}_5$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_7$.

Claim 22.

$$\mathbf{Adv}_{\mathbf{G}_6}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_5}^{\mathcal{A}} - \ell \cdot K \cdot (N - 1) \cdot \varepsilon_{\text{UUO}}.$$

The proof is analogous to the proof of Claim10

Game $\mathbf{G}_7$. This game is identical to $\mathbf{G}_6$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_8$.

Claim 23. Let $Q_{\mathcal{H}}$ and $Q_{\mathcal{S}_3}$ be the maximum number of hash queries $\mathcal{A}$ makes to $\mathcal{H}_{\mathcal{H}}$ and $\mathcal{S}_3$, respectively. Then it holds

$$\mathbf{Adv}_{\mathbf{G}_7}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_6}^{\mathcal{A}} - \frac{N \cdot Q_{\mathcal{S}_3} \cdot Q_{\mathcal{H}}}{2^\lambda}.$$

The proof is analogous to the proof of Claim 11.

Game $\mathbf{G}_8$. This game is identical to $\mathbf{G}_7$, except that it aborts and outputs 0 if $\mathcal{A}$ provokes Event $\mathbf{BAD}_9$.

Claim 24. Recall that $f(\text{ctr}) = \lceil 3 \cdot \vartheta \cdot \ln(\ell + 1) \cdot \text{ctr} \rceil$ and $\delta > 0$ with $(1 - \delta) \cdot \vartheta > 1$. It holds

$$\mathbf{Adv}_{\mathbf{G}_8}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_7}^{\mathcal{A}} - \exp(-\delta \cdot K).$$

The proof is analogous to the proof of claim 12.

Game $\mathbf{G}_9$. Identical to $\mathbf{G}_8$, except that it aborts and outputs 0 if Even $\mathbf{BAD}_{10}$ occurs.

Claim 25.

$$\mathbf{Adv}_{\mathbf{G}_9}^{\mathcal{A}} \geq \frac{1}{K} \cdot \mathbf{Adv}_{\mathbf{G}_8}^{\mathcal{A}}.$$

The proof is analogous to the proof of Claim 13.

Per this game, we know there is a uniformly chosen $\iota \in [K]$ such that $\mathcal{A}$ does not cheat for pk_ι . Recall that $\mathcal{A}$ outputs $(M_j, \Sigma_j, \mathbf{L}_j, I_{\text{op},j}, \pi_j)$ for $j \in [\ell]$. Each signature-message pair (M_j, Σ_j) consists of K message-subsignature pairs (γ_i, σ_i) for $i \in [K]$. Additionally, recall that $\mathcal{A}$ closes at most ℓ signing sessions and each session consists of K subsessions, a subsession per key pk_i . Now, we want to rule out that all subsignatures corresponding to the ι -th key pk_ι are linked to some signing subsession corresponding to that key. That is, we want a game ensuring that there exists a message-subsignature pair $(\gamma_\iota^{(\text{ctr}^*)}, \sigma_\iota^{(\text{ctr}^*)})$ that is not linked to any signing session (ctr, ι) .

Game $\mathbf{G}_{10}$. This game is identical to $\mathbf{G}_9$, except that it aborts and outputs 0 if Event $\mathbf{BAD}_{14}$ occurs, which we define as follows:

BAD₁₄: $\forall \text{ctr} \exists \text{ctr}'$ such that the message-subsignature pair $(\gamma_{\iota}^{(\text{ctr})}, \sigma_{\iota}^{(\text{ctr}')})$ is linked to the signing subsession (ctr', ι) .

Claim 26.

$$\mathbf{Adv}^{\mathbf{G}_{10}}(\mathcal{A}) \geq \mathbf{Adv}^{\mathbf{G}_9}(\mathcal{A}) - \mathbf{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_4)$$

Proof. Assume $\mathcal{A}$ provokes Event $\mathbf{BAD}_{14}$, hence each message-subsignature pair is linked to some signing subsession. We provide a reduction $\mathcal{R}_4$ that uses $\mathcal{A}$ to break the binding property of the commitment scheme MT (Lemma 7).

$\mathcal{R}_4$ simulates $\mathbf{G}_{10}$ for $\mathcal{A}$ perfectly. Once $\mathcal{A}$ terminates, it outputs $\mathbf{L}'_j$ and $(M_i, \Sigma_i, \mathbf{L}_i, I_{\text{op},i}, \pi_i)$ for all $i, j \in [\ell]$. Per our assumption, $\exists \text{ctr}^* \in [\ell] : \mathbf{L}_{\text{ctr}^*} \not\subseteq \mathbf{L}'_j$ for all $j \in [\ell]$.

Let (ctr^{**}, ι) for $\text{ctr}^{**} \in [\ell]$ be the signing subsession linked to $(\gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$, and $(\zeta_1^{(\text{ctr}^*)}, \dots, \zeta_K^{(\text{ctr}^*)}) := \zeta_{\text{ctr}^*}$. $\mathcal{R}_4$ gets the commitment and the first opening $(\zeta_{\iota}^{(\text{ctr}^*)}, (\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*}))$ directly from $\mathcal{A}$'s output. Now, $\mathcal{R}_4$ has to find another opening, which it can extract from the tables of the random oracles from the entries corresponding to the subsession (ctr^{**}, ι) .

Let $\text{msgU}_{\iota, J_{\iota}}^{(\text{ctr}^{**})} := \text{BS}.\mathcal{U}_1(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}; \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ be the user message corresponding to the cut-and-choose index J_{ι} for the subsession (ctr^{**}, ι) . Due to Games $\mathbf{G}_6$ and $\mathbf{G}_8$, $\mathcal{R}_4$ can extract the message $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ used to generate $\text{msgU}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$. Additionally, due to Game $\mathbf{G}_7$, $\mathcal{R}_4$ can look up the message $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ in the table of $\mathbf{H}_{\gamma}$ to extract the query $(\text{pk}_{\iota}, \mu_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ used to generate $\gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$. Similarly, $\mathcal{R}_4$ looks up the table of $\mathbf{H}_{\text{MT}}$ to reverse the tree root $\zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$ level by level until it recovers the leaves $\mathbf{L}$ and $\mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}$, which is possible due to Game $\mathbf{G}_2$. The tuple $(\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ constitutes the second opening for $\zeta_{\iota}^{(\text{ctr}^*)}$.

Finally, $\mathcal{A}$ outputs $(\zeta_{\iota}^{(\text{ctr}^*)}, (\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*})), (\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ as an attack against $\text{vcBINDING}_{\text{MT}}^{\mathcal{R}_4}$.

Next, we show that both openings are valid for $\zeta_{\iota}^{(\text{ctr}^*)}$. Obviously, $(\mathbf{L}_{\text{ctr}^*}, \pi_{\text{ctr}^*})$ is a valid opening since $\mathcal{SH}_{\mathcal{V}}(\text{pk}, M_i, \sigma_i, \mathbf{L}_i, I_{\text{op},i}, \pi_i) = 1$ for $i = \text{ctr}^*$. (Recall that $\mathcal{SH}_{\mathcal{V}}$ (figure 3) checks the opening validity of each $\zeta_i^{(\text{ctr}^*)}$).

The second opening $(\mathbf{L}'_{\text{ctr}^{**}}, \mathbf{R}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ is valid because the commitment extracted from the table of $\mathbf{H}_{\gamma}$ is identical to the commitment in the signature, i.e. $\zeta_{\iota, J_{\iota}}^{(\text{ctr}^{**})} = \zeta_{\iota}^{(\text{ctr}^*)}$ since the subsignature (ctr^*, ι) and the signing subsession (ctr^{**}, ι) are linked. To see this, we analyze the implication of the linking between the message-subsignature pair $(\gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$ and the signing subsession (ctr^{**}, ι) . Recall that the linking between the subsignature and the subsession implies that $\text{BS}.\mathcal{V}(\text{pk}_{\iota}, \gamma_{\iota}^{(\text{ctr}^*)}, \sigma_{\iota}^{(\text{ctr}^*)})$ and $\text{BS}.\mathcal{U}_1(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}; \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$ make the same query $Q = \text{hq}(\text{pk}_{\iota}, \gamma_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{msgS1}_{\iota, J_{\iota}}^{(\text{ctr}^{**})}, \text{Urnd}_{\iota, J_{\iota}}^{(\text{ctr}^{**})})$. However, this is only possible if the message being verified and the message

passed to the user's algorithm $\text{BS}\mathcal{U}_1$ are identical, i.e. $\mathcal{I}_l^{(\text{ctr}^*)} = \mathcal{I}_{l,J_l}^{(\text{ctr}^{**})}$ since the message is part of the hash (see Game $\mathbf{G}_4$), unless $\mathcal{A}$ can cause a collision for $\text{H}_\mathcal{I}$, which is ruled out in Game $\mathbf{G}_2$. Again, if the messages are identical, this means that the query (pk, η, ζ) sent to the random oracle $\text{H}_\mathcal{I}$ to generate $\mathcal{I}_l^{(\text{ctr}^*)}$ and $\mathcal{I}_{l,J_l}^{(\text{ctr}^{**})}$ is identical, unless $\mathcal{A}$ provokes a collision for $\text{H}_\mathcal{I}$ (Game $\mathbf{G}_2$). Consequently, it holds $\zeta_l^{(\text{ctr}^*)} = \zeta_{l,J_l}^{(\text{ctr}^{**})}$.

Finally, the openings $\mathcal{R}_4$ are distinct since $L_{\text{ctr}^*} \not\subseteq L'_{\text{ctr}^{**}}$.

By this game, $\mathcal{A}$ outputs a message-subsignature pair that is not linked to any signing subsession. This is equivalent to $\mathcal{A}$ outputting a BS signature without relying on the signing oracle. We reduce this to 0-OMUF using the same reduction from the proof of Lemma 1. Note that all games from that proof are adopted for this proof, hence non of the abort conditions makes the reduction abort, which concludes the proof. Since the only difference compared to the proof of Lemma 1 is the additional game $\mathbf{G}_{10}$, we compute $\mathcal{A}$ ' advantage as

$$\text{Adv}_{\mathbf{P}[\text{BS}]}^{\ell\text{-SH}}(\mathcal{A}) \leq K \cdot \text{Adv}_{\text{MT}}^{\text{vcBINDING}}(\mathcal{R}_4) + \text{Adv}_{\mathbf{P}[\text{BS}]}^{\ell\text{-OMUF}}(\mathcal{R}_3).$$

C.3 Blindness

Theorem 3. *For a blind signature scheme BS fulfilling definition 4 that is (t, ε) -blind, the scheme $\mathbf{P}[\text{BS}]$ is (t', ε') -blind with $t' \approx t$ and $\varepsilon' = K \cdot \varepsilon + 2 \cdot \frac{Q_{\text{H}_\mathcal{I}}^2}{2^\lambda} + 2 \frac{Q_{\text{H}_\mathcal{I}}}{2^{\lambda-1}} + 2 \frac{2K \cdot Q_{\text{Hcom}}}{2^\lambda} + 2 \frac{2K Q_{\text{HMT}}}{\lambda \nu}$.*

Proof. In the following, we gradually transition from the blindness game with $b = 0$ to the blindness game with $b = 1$. We denote by $\mathbf{G}_i = 1$ that the adversary outputs 1 when playing the game.

Game $\mathbf{G}_0$. This is the original blindness game of the scheme $\mathbf{P}[\text{BS}]$ with $b = 0$.

Game $\mathbf{G}_1$. This game introduces an abort condition that aborts when there is a collision in $\text{H}_\mathcal{J}$, i.e., whenever the adversary queries $x \neq x'$ such that $\text{H}_\mathcal{J}(x) = \text{H}_\mathcal{J}(x')$. It holds that

$$|\Pr[\mathbf{G}_1 = 1] - \Pr[\mathbf{G}_0 = 1]| \leq \frac{Q_{\text{H}_\mathcal{J}}^2}{2^\lambda}.$$

Game $\mathbf{G}_2$. In this game, when the adversary opens a new signing session by sending a cut-and-choose parameter N and a commitment $\text{com}_{\text{seed}_i}$ to either of the user oracles, the game aborts if there exists no query $\text{H}_\mathcal{J}(\text{seed}_{J_i}) = \text{com}_{\text{seed}_i}$ at that point, but the adversary successfully opens the commitment $\text{com}_{\text{seed}_i}$ later on. The probability that the adversary can later open $\text{com}_{\text{seed}_i}$ to a value seed_i is $\frac{Q_{\text{H}_\mathcal{J}}}{2^\lambda}$. Union-bounding over the two user oracles yields

$$|\Pr[\mathbf{G}_2 = 1] - \Pr[\mathbf{G}_1 = 1]| \leq \frac{Q_{\text{H}_\mathcal{J}}}{2^{\lambda-1}}.$$

Game $\mathbf{G}_3$. In this game, we change how the commitments $com_{\text{rnd}}^{(i,J_i)}$ are generated. Instead of picking $\text{Urnd}_{i,J_i} \leftarrow_{\$} \{0,1\}^\lambda$ and computing $com_{\text{rnd}}^{(i,J_i)} := H_{\text{rnd}}(\text{Urnd}_{i,J_i})$, the game samples $com_{\text{rnd}}^{(i,J_i)} \leftarrow_{\$} \{0,1\}^\lambda$. Clearly, the adversary does not notice this difference unless it queries one of the Urnd_{i,J_i} of one of the sessions to H_{com} . Thus, it holds that

$$|\Pr[\mathbf{G}_3 = 1] - \Pr[\mathbf{G}_2 = 1]| \leq \frac{2KQ_{H_{\text{com}}}}{2^\lambda}.$$

Game $\mathbf{G}_4$. In this game, we change how the commitments $\mu_{i,j}$ for $j \neq J_i$ are generated. Namely, instead of computing $\mu_{i,j} = H_\mu(m, \eta_{i,j})$, we randomly sample $\mu_{i,j} \leftarrow_{\$} \{0,1\}^\lambda$. This is noticeable if the adversary queried one of the η_{i,j_i} to the RO which happens with probability at most $\frac{K \cdot N \cdot Q_{H_\mu}}{2^\lambda}$. Thus it holds that $|\Pr[\mathbf{G}_4 = 1] - \Pr[\mathbf{G}_3 = 1]| \leq \frac{K \cdot N \cdot Q_{H_\mu}}{2^\lambda}$.

Game $\mathbf{G}_5$. This game aborts whenever the adversary makes a query to H_{MT} that is made in the Merkle-Trees of the indices from $\mathbf{J}$ for some challenge message before receiving the openings during the showing phase.

This happens with probability at most $\frac{2 \cdot K \cdot Q_{H_{\text{MT}}}}{\lambda \cdot \nu}$ and thus

$$|\Pr[\mathbf{G}_5 = 1] - \Pr[\mathbf{G}_4 = 1]| \leq \frac{2 \cdot K \cdot Q_{H_{\text{MT}}}}{\lambda \cdot \nu}.$$

Game $\mathbf{G}_6$. In this game, we change how the commitments ζ_{i,J_i} are generated. Namely, instead of computing a Merkle-Tree $\zeta_{i,J_i} := \text{MT}(\mathbf{L}; \mathbf{R}_{i,J_i})$, the game samples $\zeta_{i,J_i} \leftarrow_{\$} \{0,1\}^\lambda$. When the adversary asks to show the signature, the game picks random values $\mathbf{R}_{i,J_i,k}, k \in \mathbf{I}_{\text{op}}$ as well as values for the co-path of the opened indices. It then programs the Random oracle to so that the root of the Merkle-Tree is ζ_{i,J_i} . By the changes introduced in $\mathbf{G}_2$ and $\mathbf{G}_5$, $\mathbf{G}_6$ is identically distributed to $\mathbf{G}_5$.

Game $\mathbf{G}_7$. In this game, we swap the bits b to 1. We do a hybrid argument over the underlying public keys. We start out with $\mathbf{G}_7^0$, which is the same as $\mathbf{G}_6$. In $\mathbf{G}_7^i$, for the first i public keys, the roles of M_0 and M_1 are swapped in the generation of the messages Υ_{k,J_k} for $k \leq i$. In the end, the underlying message-signature pairs are reassembled according to the messages, i.e. the first i pairs are swapped back. Showing including the opening of Merkle-Trees works as before. It is easy to see that $\mathbf{G}_7$ is the same as $\mathbf{G}_7^K$. Each of these steps can be bounded by the blindness of the underlying scheme. We provide a reduction for the game hop from $\mathbf{G}_7^{i-1}$ to $\mathbf{G}_7^i$:

- The reduction receives the public key from the adversary. It forwards pk_i to its challenger. Using the messages M_0 and M_1 from the signer, it generates μ_{i,J_i} for each session, samples ζ_{i,J_i} , and computes $\Upsilon_{i,J_i,0}$ and $\Upsilon_{i,J_i,1}$ accordingly. It provides $\Upsilon_{i,J_i,0}$ and $\Upsilon_{i,J_i,1}$ to its challenger as the challenge messages.

- It implements the signing oracles as in $\mathbf{G}_7^{i-1}$ using its control over the random oracles, except that to generate the user messages of the i -th public key, it queries the user oracles of its challenger.
- To generate the output signatures, it reassembles the signatures as described above.
- When showing the signatures, it programs the random oracles to open the Merkle-Trees as described in $\mathbf{G}_6$.
- When asked to output a bit, it outputs the bit the adversary outputs.

It is easy to see that the reduction simulates $\mathbf{G}_7^{i-1}$ or $\mathbf{G}_7^i$ perfectly, depending on the challenger's bit. It thus holds that

$$|\Pr[\mathbf{G}_7^i = 1] - \Pr[\mathbf{G}_7^{i-1} = 1]| \leq \mathbf{Adv}_{\text{BS}}^{\text{BLIND}}(\mathcal{R}).$$

Summing over $i = 1, \dots, K$ yields

$$|\Pr[\mathbf{G}_7 = 1] - \Pr[\mathbf{G}_6 = 1]| \leq K \cdot \mathbf{Adv}_{\text{BS}}^{\text{BLIND}}(\mathcal{R}).$$

Game $\mathbf{G}_8$. In this game, we revert the change from $\mathbf{G}_6$. By the same argument as in $\mathbf{G}_6$, it holds that

$$|\Pr[\mathbf{G}_8 = 1] - \Pr[\mathbf{G}_7 = 1]| \leq \frac{2KQ_{\text{HMT}}}{\lambda \times \nu}.$$

Game $\mathbf{G}_9$. This game reverts the changes introduced in $\mathbf{G}_5$. By the same argument as in $\mathbf{G}_5$ it holds that

$$|\Pr[\mathbf{G}_9 = 1] - \Pr[\mathbf{G}_8 = 1]| \leq \frac{2KQ_{\text{HMT}}}{\lambda \times \nu}.$$

Game $\mathbf{G}_{10}$. In this game, we revert the change from $\mathbf{G}_4$, but swapping M_0 and M_1 . By the same argument as in $\mathbf{G}_4$, it holds that

$$|\Pr[\mathbf{G}_{10} = 1] - \Pr[\mathbf{G}_9 = 1]| \leq \frac{K \cdot N \cdot Q_{\text{H}\mu}}{2^\lambda}.$$

Game $\mathbf{G}_{11}$. In this game, we revert the change from $\mathbf{G}_3$. By the same argument as in $\mathbf{G}_3$, it holds that

$$|\Pr[\mathbf{G}_{11} = 1] - \Pr[\mathbf{G}_{10} = 1]| \leq \frac{2KQ_{\text{Hcom}}}{2^\lambda}.$$

Game $\mathbf{G}_{12}$. In this game, we revert the change from $\mathbf{G}_2$. By the same argument as in $\mathbf{G}_2$, it holds that

$$|\Pr[\mathbf{G}_{12} = 1] - \Pr[\mathbf{G}_{11} = 1]| \leq \frac{Q_{\text{HJ}}}{2^{\lambda-1}}.$$

Game $\mathbf{G}_{13}$. In this game, we revert the change from $\mathbf{G}_1$. By the same argument as in $\mathbf{G}_1$, it holds that

$$|\Pr[\mathbf{G}_{13} = 1] - \Pr[\mathbf{G}_{12} = 1]| \leq \frac{Q_{\text{HJ}}^2}{2^\lambda}.$$

$\mathbf{G}_{13}$ is identical to the original blindness game but with $b = 1$. Summing up the probabilities we get that

$$\begin{aligned} \text{Adv}_{\mathbf{P}[\text{BS}]}^{\text{BLIND}}(\mathcal{A}) &= 2 \left| \Pr[b = b'] - \frac{1}{2} \right| = |\Pr[b' = 1|b = 1] - \Pr[b' = 1|b = 0]| \\ &\leq 2 \cdot \left(\frac{Q_{\text{HJ}}^2}{2^\lambda} + \frac{Q_{\text{HJ}}}{2^{\lambda-1}} + \frac{2 \cdot K \cdot Q_{\text{Hcom}}}{2^\lambda} + \frac{2 \cdot K \cdot Q_{\text{HMT}}}{\lambda \cdot \nu} \right) \\ &\quad + K \cdot \text{Adv}_{\text{BS}}^{\text{BLIND}}(\mathcal{R}). \end{aligned}$$

D Blind Schnorr Signatures as an Instantiation of Three-Move Simulatable Blind Signature Schemes

Construction 7 (Blind Schnorr Signature). For a security parameter λ , let $\text{pp} := (\mathbb{G}, p, g)$ be a public parameter specifying a group $\mathbb{G}$ of order p and a generator g , and $\text{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be a hash function treated as a random oracle. The Blind Schnorr signature scheme $\text{BS} = (\mathcal{G}, \mathcal{S}, \mathcal{IM}_S, \mathcal{U}, \mathcal{V})$ is defined as follows:

- **Key Generation** $\mathcal{G}(\text{pp})$: Sample a secret key $x \leftarrow_{\$} \mathbb{Z}_p$ and set $\text{pk} := g^x$. Output $(\text{pk}, \text{sk} := x)$.
- **Signer Algorithms** $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$:
 - $\mathcal{S}_1^{\text{H}}(\text{sk})$: Sample $r \leftarrow_{\$} \mathbb{Z}_p$. Compute $R := g^r$. Output $\text{msgS1} := R$ and $\text{rnd}_S := r$.
 - $\mathcal{S}_2^{\text{H}}(\text{sk}, \text{rnd}_S, \text{msgU})$: Parse rnd_S as r and msgU as c . Compute $s := r + c \cdot x \pmod{p}$. Output $\text{msgS2} := s$.
- **User Algorithms** $\mathcal{U}^{\text{H}} = (\mathcal{U}_1^{\text{H}}, \mathcal{U}_2^{\text{H}})$:
 - $\mathcal{U}_1^{\text{H}}(\text{pk}, m, \text{msgS1}; \text{Urnd})$: Parse msgS1 as R and Urnd as $(\alpha, \beta) \in \mathbb{Z}_p^2$. Compute $R' := R \cdot g^\alpha \cdot \text{pk}^{-\beta}$. Query $c' := \text{H}(R', m)$. Compute $c := c' - \beta \pmod{p}$. Output $\text{msgU} := c$.
 - $\mathcal{U}_2^{\text{H}}(\text{pk}, m, \text{Urnd}, \text{msgS2})$: Parse Urnd as (α, β) and msgS2 as s . Compute $s' := s + \alpha \pmod{p}$. Output $\sigma := (c', s')$.

- **Verification** $\mathcal{V}^H(\text{pk}, m, \sigma)$: Parse σ as (c', s') . Compute $R' := g^{s'} \cdot \text{pk}^{-c'}$. Output 1 if $c' = H(R', m)$, and 0 otherwise.
- **Signer Simulation Algorithms** $\text{SIM}_S^H = (\text{SIM}_{S_1}^H, \text{SIM}_{S_2}^H, \text{SIM}_H)$:
 - $\text{SIM}_{S_1}(\text{pk})$: Sample $s, c \leftarrow_{\$} \mathbb{Z}_p$. Compute $R := g^s \cdot \text{pk}^{-c}$. Output $\text{msgS1} := R$ and $\text{simSt} := (s, c)$.
 - $\text{SIM}_{S_2}(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$: Parse simSt as (s, c) . Run the User algorithm to obtain the challenge: $\text{msgU} \leftarrow \mathcal{U}_1^{\text{SIM}_H}(\text{pk}, m, \text{msgS1}; \text{Urnd})$. Output $\text{msgS2} := s$.
 - $\text{SIM}_H(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$: Parse simSt as (s, c) and Urnd as (α, β) . Compute $R' := R \cdot g^\alpha \cdot \text{pk}^{-\beta}$ (which equals $g^{s+\alpha} \text{pk}^{-\beta-c}$). Program the random oracle such that $H(R', m) := c + \beta \pmod{p}$. Output $c_H := c + \beta$.

We show that Blind Schnorr satisfies the properties described in Section 4.

Property 1: holds, because in an honest protocol execution, $\mathcal{U}_1$ and $\mathcal{V}$ make the same hash query to the random oracle. The hash query is derived from the signer's message msgS1 , the user's randomness Urnd , and the message m , and the key $\text{pkas } R' := \text{msgS1} \cdot g^\alpha \cdot \text{pk}^{-\beta}$; therefore, the hash can be uniquely identified by $\text{hq}(\text{pk}, m, \text{msgS1}, \text{Urnd} = (\alpha, \beta)) = (R', m)$ with R' as above.

Property 2: holds since violating this property is equivalent to guessing the output of H . More specifically, given a valid message-signature pair (m, σ) satisfying $\mathcal{V}(\text{pk}, m, \sigma) = 1$, it holds for (c', s') that $c' = H(g^{s'} \cdot \text{pk}^{-c'}, m)$. Since H behaves like a true random function, the probability of this property being violated is at most $\frac{1}{p} =: \varepsilon_{\text{UUF}}$.

Property 3: This property holds since the user message $\text{msgU} = c$ is derived from the output of the random oracle c' and the user randomness, i.e. $c = c' - \beta$, and outputting a user message without having queried the random oracle H beforehand is, again, equivalent to guessing the output of the random oracle. To see this, let $\mathcal{A}$ be an adversary violating Property 3 by outputting $(\text{msgU}, \text{pk}, m, \text{msgS1}, \text{Urnd})$, hence $\text{msgU} = \mathcal{U}_1(\text{pk}, m, \text{msgS1}, \text{Urnd}) = H(R', m) - \beta$ for $R' := \text{msgS1} \cdot g^\alpha \cdot \text{pk}^{-\beta}$ and $\text{Urnd} = (\alpha, \beta)$. It follows that $\text{msgU} + \beta = H(R', m)$. Since H is a truly random function, $\mathcal{A}$ succeeds with probability at most $\frac{1}{p}$.

Property 4: Let $\mathcal{A}$ be an adversary violating this property by outputting distinct tuples (m_1, Urnd_1) and (m_2, Urnd_2) with $\text{Urnd}_i = (\alpha_i, \beta_i)$ satisfying $\mathcal{U}_1(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) = \mathcal{U}_1(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$ for some public key pk and a signer's first message msgS1 .

Recall that the hash query is of the form $\text{hq}(\text{pk}, m_i, \text{msgS1}, \text{Urnd}_i) = (R' = \text{msgS1} \cdot g^{\alpha_i} \cdot \text{pk}^{\beta_i}, m_i)$. We consider the following cases:

- $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) = \text{hq}(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$. In this case, it holds that $m_1 = m_2$ and $R' = R \cdot g^{\alpha_1} \cdot \text{pk}^{-\beta_1} = R \cdot g^{\alpha_2} \cdot \text{pk}^{-\beta_2}$ with $(\alpha_1, \beta_1) \neq (\alpha_2, \beta_2)$. A reduction embedding a discrete logarithm challenge in pk can solve for $\text{dlog}_g \text{pk} = \frac{\alpha_1 - \alpha_2}{\beta_1 - \beta_2}$. The adversary's advantage can be bounded by ε_{DLP} .
- $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) \neq \text{hq}(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$ and at least one of the hash queries was not made by the adversary before. This boils down to the adversary guessing the hash output and succeeding with probability $\frac{1}{p}$.

- Otherwise: The two hash queries are distinct and have both been made. In this case, a reduction embedding a discrete logarithm challenge into pk can solve for pk via rewinding the adversary and reprogramming the RO at the latter of the two queries. Without loss of generality, consider the case that $\text{hq}(\text{pk}, m_1, \text{msgS1}; \text{Urnd}_1)$ was made first. Denote by $\text{msgU} = \mathcal{U}_1(\text{pk}, m_1, \text{msgS1}; \text{Urnd}_1)$. If the adversary succeeds in both runs at producing $(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$ and $(\text{pk}, m'_2, \text{msgS1}, \text{Urnd}'_2)$ that both produce msgU with the same hash query, then the reduction can solve for $\text{dlog}_g \text{pk}$ as above. The reduction succeeds with probability approximately $\frac{\varepsilon_{\text{UVO}}}{2} (\frac{\varepsilon_{\text{UVO}}}{2 \cdot \text{QH}} - \frac{1}{p}) \leq \varepsilon_{\text{DLP}}$.

Overall, we can bound $\varepsilon_{\text{UL}} \leq 2 \cdot \text{QH} \sqrt{\varepsilon_{\text{DLP}}} + \varepsilon_{\text{DLP}} + \frac{1}{p}$.

Property 5: holds since the message m must be included in the hash query.

Property 6: Assume an adversary $\mathcal{A}$ violates this property by outputting a tuple $(m, \text{Urnd}_1, \text{Urnd}_2)$ such that $\text{hq}(\text{pk}, m, \text{msgS1}_1, \text{Urnd}_1) = \text{hq}(\text{pk}, m, \text{msgS1}_2, \text{Urnd}_2)$ for some public key pk and honestly-generated signer's first messages msgS1_1 and msgS1_2 . Let $(\alpha_1, \beta_1) := \text{Urnd}_1$ and $(\alpha_2, \beta_2) := \text{Urnd}_2$. It must hold that $\text{msgS1}_1 \cdot g^{\alpha_1} \cdot \text{pk}^{-\beta_1} = \text{msgS1}_2 \cdot g^{\alpha_2} \cdot \text{pk}^{-\beta_2}$. If $\mathcal{A}$ violates Property 6, we distinguish two cases:

- Either $\text{msgS1}_1 = \text{msgS1}_2$. This implies that $\text{Urnd}_1 \neq \text{Urnd}_2$, hence $\beta_1 \neq \beta_2$. Since $\beta_2 - \beta_1 \neq 0$, a reduction that embeds in the public key pk can compute $\text{dlog}_g \text{pk} = \text{dlog}_g \text{ch} = \frac{\alpha_1 - \alpha_2}{\beta_1 - \beta_2}$ and solve the discrete logarithm problem.
- Or $\text{msgS1}_1 \neq \text{msgS1}_2$. In this case, the reduction embeds the discrete logarithm in msgS1 and generates $\text{msgS1}_2 = g^r$. It also picks the public key as g^x for $x \leftarrow_{\$} \mathbb{Z}_p$. When the adversary produces $m, \text{Urnd}_1, \text{Urnd}_2$ that break the property, it must hold that $\text{msgS1}_1 \cdot g^{\alpha_1} \cdot g^{-\beta_1 x} = g^r \cdot g^{\alpha_2} \cdot g^{-\beta_2 x}$ which the reduction can solve for $\text{dlog}_g \text{msgS1} = \text{dlog}_g \text{ch} = r + \alpha_2 - x \cdot \beta_2 - \alpha_1 + \beta_1 \cdot x$.

Overall, this means that $\varepsilon_{\text{UL}} \leq 2 \cdot \varepsilon_{\text{DLP}}$.

Property 7: holds for the Blind Schnorr scheme. To see this, assume an adversary provides two distinct signatures $\sigma_1 := (c_1, s_1)$ and $\sigma_2 := (c_2, s_2)$ for the same message m . The verification procedure $\mathcal{V}(\text{pk}, m, \sigma_i)$ ensures that $\text{H}(g^{s_i} \cdot \text{pk}^{-c_i}, m) = c_i$. If both signatures correspond to the same hash, it must hold that $c_1 = c_2$ and $g^{s_1} \cdot \text{pk}^{-c_1} = g^{s_2} \cdot \text{pk}^{-c_2}$. However, $c_1 = c_2 \Rightarrow s_1 = s_2$; otherwise, $g^{s_1} \cdot \text{pk}^{-c_1} \neq g^{s_2} \cdot \text{pk}^{-c_2}$. This is a contradiction.

E Abe-Okamoto Signatures as an Instantiation of Three-Move Simulatable Blind Signature Schemes

In this section, we discuss the compatibility between our definition for three-move simulatable blind signatures (Section 4) and Blind Signature Schemes based on the OR-Proof technique [22]. In this section, we study the Abe-Okamoto scheme (AO) [2]. Similar arguments apply to the other OR-proof based schemes such as Abe [1].

First, we write down the scheme in the style of Definition 4.

Construction 8 (Abe-Okamoto Blind Signature). For a security parameter λ , let $\text{pp} := (\mathbb{G}, p, g)$ be a public parameter specifying a group $\mathbb{G}$ of order p and a generator g , and $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ and $\mathcal{F} : \{0, 1\}^* \rightarrow \mathbb{G}$ be hash functions treated as random oracles. Let $\text{info} \in \{0, 1\}^*$ be a common string. The Abe-Okamoto blind signature scheme $\text{BS} = (\mathcal{G}, \mathcal{S}, \text{SIM}_{\mathcal{S}}, \mathcal{U}, \mathcal{V})$ is defined as follows:

- **Key Generation** $\mathcal{G}(\text{pp})$: Sample $x \leftarrow_{\$} \mathbb{Z}_p$, compute $y := g^x$, and output $(\text{pk}, \text{sk}) := (y, x)$.
- **Signer Algorithms** $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$:
 - $\mathcal{S}_1^H(\text{sk})$: Sample $u, s, d \leftarrow_{\$} \mathbb{Z}_p$. Compute $z := \mathcal{F}(\text{info})$, $a := g^u$, and $b := g^s \cdot z^d$. Output $\text{msgS1} := (a, b)$ and $\text{rnd}_S := (u, s, d)$.
 - $\mathcal{S}_2^H(\text{sk}, \text{rnd}_S, \text{msgU})$: Parse rnd_S as (u, s, d) and msgU as e . Compute $c := e - d \pmod{p}$ and $r := u - c \cdot x \pmod{p}$. Output $\text{msgS2} := (r, c, s, d)$.
- **User Algorithms** $\mathcal{U}^H = (\mathcal{U}_1^H, \mathcal{U}_2^H)$:
 - $\mathcal{U}_1^H(\text{pk}, m, \text{msgS1}; \text{Urnd})$: Parse msgS1 as (a, b) and Urnd as $(t_1, t_2, t_3, t_4) \in \mathbb{Z}_p^4$. Compute $z := \mathcal{F}(\text{info})$, $\alpha := a \cdot g^{t_1} \cdot y^{t_2}$, and $\beta := b \cdot g^{t_3} \cdot z^{t_4}$. Query $\varepsilon := H(\alpha, \beta, z, m)$. Compute $e := \varepsilon - t_2 - t_4 \pmod{p}$ and output $\text{msgU} := e$.
 - $\mathcal{U}_2^H(\text{pk}, m, \text{Urnd}, \text{msgS2})$: Parse Urnd as (t_1, t_2, t_3, t_4) and msgS2 as (r, c, s, d) . Compute $\rho := r + t_1 \pmod{p}$, $\omega := c + t_2 \pmod{p}$, $\sigma := s + t_3 \pmod{p}$, and $\delta := d + t_4 \pmod{p}$. Output $\sigma := (\rho, \omega, \sigma, \delta)$.
- **Verification** $\mathcal{V}^H(\text{pk}, m, \sigma)$: Parse σ as $(\rho, \omega, \sigma, \delta)$. Compute $z := \mathcal{F}(\text{info})$. Output 1 if $\omega + \delta \pmod{p} \equiv H(g^\rho \cdot y^\omega, g^\sigma \cdot z^\delta, z, m)$, and 0 otherwise.
- **Signer Simulation Algorithms** $\text{SIM}_{\mathcal{S}}^H = (\text{SIM}_{\mathcal{S}_1}^H, \text{SIM}_{\mathcal{S}_2}^H, \text{SIM}_{\mathcal{H}}^H)$:
 - $\text{SIM}_{\mathcal{S}_1}^H(\text{pk})$: By programming the random oracle $\mathcal{F}(\text{info}) := g^w$, the algorithm knows $\text{dlog}_g z = w$. Sample $v, r, c \leftarrow_{\$} \mathbb{Z}_p$. Compute $a := g^r \cdot y^c$ and $b := g^v$. Output $\text{msgS1} := (a, b)$ and $\text{simSt} := (w, v, r, c)$.
 - $\text{SIM}_{\mathcal{S}_2}^H(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$: Parse simSt as (w, v, r, c) . Run the User algorithm to obtain the challenge: $e \leftarrow \mathcal{U}_1^{\text{SIM}_{\mathcal{H}}^H}(\text{pk}, m, \text{msgS1}; \text{Urnd})$. Compute $d := e - c \pmod{p}$ and $s := v - d \cdot w \pmod{p}$. Output $\text{msgS2} := (r, c, s, d)$.
 - $\text{SIM}_{\mathcal{H}}^H(\text{pk}, \text{msgS1}, \text{simSt}, \text{Urnd}, m)$: Answer via lazy sampling. Upon fresh queries, sample and return $\varepsilon \leftarrow_{\$} \mathbb{Z}_p$.

Now, we show that AO satisfies the properties required by Definition 4.

Property 1: Since both $\mathcal{U}_1$ and $\mathcal{V}$ make a single hash query to the challenge random oracle H . This hash query contains the signed message m , and a function $\text{blind}(\text{msgS1}, \text{Urnd})$ of the signer's first message $\text{msgS1} = (a, b)$ and the user's randomness $\text{Urnd} = (t_1, t_2, t_3, t_4)$, where $\text{blind}((a, b), (t_1, \dots, t_4)) := (\alpha = a \cdot g^{t_1} \cdot y^{t_2}, b \cdot g^{t_3} \cdot z^{t_4})$. Finally, α is derived from the public key $\text{pk} = y$ and other values. Consequently, in an honest protocol execution, the tuple $(\text{pk}, m, \text{msgS1}, \text{Urnd})$ determines the hash.

Property 2: An adversary outputting a valid signature without having queried the respective hash query previously is essentially guessing the output of the oracle H successfully. To see this, let $(\rho, \omega, \sigma, \delta)$ be a signature for a message m

under pk . If the signature is valid, the adversary has successfully guessed the output of H on input $Q = (g^\rho \cdot y^\omega, g^\sigma \cdot z^\delta, z, m)$ since $\omega + \delta = H(Q)$ holds. Since the output of H is uniformly random in $\mathbb{Z}_p$, the probability of finding such a signature is at most $\frac{1}{p} =: \varepsilon_{\text{SHL}}$.

Property 3: The first user message $\text{msgU} = e := H(Q) - t_2 - t_4$ is a blinded version of the hash query. Similar to the argument above, outputting a user message without having queried the random oracle H is equivalent to guessing the output of H , which is possible with probability at most $\frac{1}{p} =: \varepsilon_{\text{UUF}}$.

Property 4: Breaking this property by an adversary implies that the adversary can solve the discrete logarithm problem. First, recall that the adversary outputs $(m_1, m_2, \text{Urnd}_1, \text{Urnd}_2)$ with $\mathcal{U}_1(\text{pk}, m_1, \text{msgS1}; \text{Urnd}_1) = \mathcal{U}_1(\text{pk}, m_2, \text{msgS1}; \text{Urnd}_2)$, where $\text{msgS1} = (a, b)$ and $\text{Urnd}_i = (t_{1,i}, \dots, t_{4,i})$. It follows

$$\begin{aligned} & H(a \cdot g^{t_{1,1}} \cdot \text{pk}^{t_{2,1}}, b \cdot g^{t_{3,1}} \cdot z^{t_{4,1}}, z, m_1) - t_{2,1} - t_{4,1} \\ &= H(a \cdot g^{t_{1,2}} \cdot \text{pk}^{t_{2,2}}, b \cdot g^{t_{3,2}} \cdot z^{t_{4,2}}, z, m_2) - t_{2,2} - t_{4,2}. \end{aligned}$$

We distinguish the following cases:

- **Case 1.** $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) = \text{hq}(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$. In this case, it holds that $m_1 = m_2$, $\alpha_1 = \alpha_2$, and $\beta = \beta_2$. Consequently, it must hold that $\text{Urnd}_1 \neq \text{Urnd}_2$, hence $t_{j,1} \neq t_{j,2}$ for some j . Again, we distinguish two cases:
 - $j = 1 \vee j = 2$. It follows that $g^{t_{1,1}} \cdot \text{pk}^{t_{2,1}} = g^{t_{1,2}} \cdot \text{pk}^{t_{2,2}}$. In this case, a reduction embedding a discrete logarithm challenge ch in pk can solve $\text{dlog}_g \text{ch} = \text{dlog}_g \text{ch} = \frac{t_{1,1} - t_{1,2}}{t_{2,2} - t_{2,1}}$. This equality is solvable since $t_{2,1} \neq t_{2,2}$ (note that this follows from $t_{1,1} \neq t_{1,2}$, too; otherwise, $\alpha_1 \neq \alpha_2$).
 - $j = 3 \vee j = 4$. It follows that $g^{t_{3,1}} \cdot z^{t_{4,1}} = g^{t_{3,2}} \cdot z^{t_{4,2}}$. In this case, a reduction embedding a discrete logarithm challenge ch in z by programming the random oracle $\mathcal{F}$ (while randomizing z for each fresh query) can solve $\text{dlog}_g z = \text{dlog}_g \text{ch} = \frac{t_{3,1} - t_{3,2}}{t_{4,2} - t_{4,1}}$. This equality is defined since $t_{4,1} \neq t_{4,2}$ (this follows from $t_{3,1} \neq t_{3,2}$, too; otherwise, $\beta_1 \neq \beta_2$).

The adversary's advantage in provoking this case is bounded by $2 \cdot \varepsilon_{\text{DLP}}$

- **Case 2.** $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) \neq \text{hq}(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$, and the adversary did not make one of the hash queries. In this case, the adversary succeeds with probability at most $\frac{1}{p}$.
- **Case 3.** $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1) \neq \text{hq}(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$ and the adversary makes both hash queries to the oracle H . We construct a reduction embedding a discrete logarithm challenge ch in pk or z and solving for $\text{dlog}_g \text{ch}$. Assume w.l.o.g. $\text{hq}(\text{pk}, m_1, \text{msgS1}, \text{Urnd}_1)$ is the first hash query the adversary makes and let $\text{msgU} := \mathcal{U}_1(\text{pk}, m_1, \text{msgS1}_1; \text{Urnd}_1)$. Upon the second hash query, the reduction answers the second query and the adversary produces $(\text{pk}, m_2, \text{msgS1}, \text{Urnd}_2)$. Then the reduction rewinds the adversary to the point before the second query and reprograms the random oracle H . The adversary then outputs another tuple $(\text{pk}, m'_2, \text{msgS1}, \text{Urnd}')$. The reduction proceeds similar to Case 1 and computes $\text{dlog}_g \text{pk}$ or $\text{dlog}_g z$, depending on which subcase occurs.

Note that for Cases 1 and 3, the reduction does not know which subcase occurs before the adversary terminates. Therefore, it flips a coin and embeds either in $\mathbf{pk}$ or z ; therefore, the probability that it extracts successfully for each subcase is $\frac{1}{2}$. Therefore, the advantage of the adversary in violating this property is bounded by $\varepsilon_{\text{UUO}} := \frac{1}{p} + 2 \cdot \varepsilon_{\text{DLP}} + \sqrt{Q_{\text{H}} \cdot \varepsilon_{\text{DLP}}}$.

Property 5: Obviously holds since m is part of the hash sent to the oracle $\mathbf{H}$ during the honest execution of $\mathcal{U}_1$ and signature verification.

Property 6: holds since violating this property is as hard as solving the discrete logarithm problem. Let $\mathcal{A}$ be an adversary violating this property by outputting a tuple $(m, \text{Urnd}_1, \text{Urnd}_2)$ with $\text{Urnd}_i = (t_{1,i}, \dots, t_{4,i})$ satisfying $\text{hq}(\mathbf{pk}, m, \text{msgS1}_1, \text{Urnd}_1) = \text{hq}(\mathbf{pk}, m, \text{msgS1}_2, \text{Urnd}_2)$ for some public key $\mathbf{pk}$ and a signer's first messages $\text{msgS1}_i = (a_i, b_i)$. It follows that

$$\begin{aligned} & \mathbf{H}(a_1 \cdot g^{t_{1,1}} \cdot \mathbf{pk}^{t_{2,1}}, b_1 \cdot g^{t_{3,1}} \cdot z^{t_{4,1}}, z, m) \\ &= \mathbf{H}(a_2 \cdot g^{t_{1,2}} \cdot \mathbf{pk}^{t_{2,2}}, b_2 \cdot g^{t_{3,2}} \cdot z^{t_{4,2}}, z, m). \end{aligned}$$

We distinguish two cases:

- $\text{msgS1}_1 = \text{msgS1}_2$. In this case, a reduction takes a DLP challenge $\mathbf{ch}$ as input, it flips a bit $B \leftarrow_{\$} \{0, 1\}$ and does the following
 - If $B = 0$, the reduction extracts $\text{dlog}_g \mathbf{ch} = \frac{t_{1,1} - t_{1,2}}{t_{2,2} - t_{2,1}}$ successfully if $t_{2,1} \neq t_{2,2}$.
 - If $B = 1$, the reduction embeds the challenge $\mathbf{ch}$ in z by programming the random oracle $\mathcal{F}$ (with the necessary randomization for each fresh query). It simulates the signing algorithm normally using the secret key. The reduction extracts $\text{dlog}_g \mathbf{ch} = \text{dlog}_g z = \frac{t_{3,1} - t_{3,2}}{t_{4,2} - t_{4,1}}$ if $t_{4,2} \neq t_{4,1}$.
- $\text{msgS1}_1 \neq \text{msgS1}_2$. In this case, a reduction embeds a DLP challenge $\mathbf{ch}$ in msgS1_1 , while generating $\text{msgS1}_2 = g^r$ honestly. Once the adversary terminates and submits its output, it must hold that $\text{msgS1}_1 \cdot g^{t_{1,1}} \cdot \mathbf{pk}^{t_{2,1}} = \text{msgS1}_2 \cdot g^{t_{1,2}} \cdot \mathbf{pk}^{t_{2,2}}$. Hence, the reduction solves $\text{dlog}_g \mathbf{ch} = \text{dlog}_g \text{msgS1}_1 = r + t_{1,2} - t_{2,2} + x \cdot (t_{2,2} - t_{2,1})$.

Overall, $\mathcal{A}$ violates this property with probability at most $3 \cdot \varepsilon_{\text{DLP}} + \frac{1}{p} =: \varepsilon_{\text{UL}}$.

Property 7: Recall that a signature-message pair $(m, (\rho, \omega, \sigma, \delta))$ is valid if $\omega + \delta = \mathbf{H}(g^\rho \cdot \mathbf{pk}^\omega, g^\sigma \cdot z^\delta, z, m)$. This verification equation does not directly tie a hash value to a specific message-signature pair. For example, knowing the secret key $\mathbf{sk}$ and $\text{dlog}_g z$, one can generate multiple signatures that are tied to the same hash value as follows: Choose ω' and δ' arbitrarily such that $\delta' \neq \delta$ and $\omega \neq \omega'$ with $\omega + \delta = \mathbf{H}(g^\rho \cdot \mathbf{pk}^\omega, g^\sigma \cdot z^\delta, z, m) = \omega' + \delta'$. Then compute $\rho' := \rho + (\omega - \omega') \cdot \mathbf{sk}$ and $\sigma' := \sigma + (\delta - \delta') \cdot \text{dlog}_g z$, and output the signature-message pair $(m, (\rho', \omega', \sigma', \delta'))$. Obviously, this is a valid pair since $\delta' + \omega' = \delta + \omega = \mathbf{H}(g^\rho \cdot \mathbf{pk}^\omega, g^\sigma \cdot z^\delta, z, m) = \mathbf{H}(g^{\rho'} \cdot \mathbf{pk}^{\omega'}, g^{\sigma'} \cdot z^{\delta'}, z, m)$. However, such behaviour is only possible if $\mathbf{sk}$ and $\text{dlog}_g z$ are known; therefore, no user should be able to find a pair of signatures linked to the same hash query, which we show in the following.

Let $\mathcal{A}$ be an adversary playing the double signature game described by Property 7. Assume $\mathcal{A}$ outputs two distinct valid message-signature pairs $(m_1, (\rho_1,$

$\omega_1, \sigma_1, \delta_1))$ and $(m_2, (\rho_2, \omega_2, \sigma_2, \delta_2))$ with $(m_1, (\rho_1, \omega_1, \sigma_1, \delta_1)) \neq (m_2, (\rho_2, \omega_2, \sigma_2, \delta_2))$ and $\omega_1 + \delta_1 = \omega_2 + \delta_2$, hence it wins the game. We provide a reduction $\mathcal{R}_5$ that solves the discrete logarithm problem.

Given a discrete logarithm challenge $\mathbf{ch}$, $\mathcal{R}_5$ behaves as follows: It sets $\mathbf{pk} := \mathbf{ch}$ and invokes $\mathcal{A}(\mathbf{pk})$ with access to oracles $\mathcal{S}_1, \mathcal{S}_2, \mathcal{F}, \mathcal{H}$. $\mathcal{R}_5$ simulates the oracles as follows:

- $\mathcal{S}_1(\mathbf{pk}, \text{info}_{\text{sid}})$. Increment sid . Compute $z_{\text{sid}} := \mathcal{F}(\text{info}_{\text{sid}})$. Sample $c_{\text{sid}}, r_{\text{sid}}, v_{\text{sid}} \leftarrow_{\$} \mathbb{Z}_p$. Compute $a_{\text{sid}} := g^{r_{\text{sid}}} \cdot \mathbf{pk}^{c_{\text{sid}}}$, $b_{\text{sid}} := g^{v_{\text{sid}}}$. Output $(a_{\text{sid}}, b_{\text{sid}})$.
- $\mathcal{S}_2(e_{\text{sid}})$. Compute $d_{\text{sid}} := e_{\text{sid}} - c_{\text{sid}}$. Compute $s_{\text{sid}} := v_{\text{sid}} - d_{\text{sid}} \cdot \text{dlog}_g z_{\text{sid}}$. Output $(r_{\text{sid}}, c_{\text{sid}}, s_{\text{sid}}, d_{\text{sid}})$.
- $\mathcal{H}(Q)$ via Lazy sampling.
- $\mathcal{F}(Q)$. via lazy sampling. Upon a fresh query, sample $w \leftarrow_{\$} \mathbb{Z}_p$, compute $z := g^w$, store $\mathbf{T}_{\mathcal{F}}[Q] := z$, and return z .

Assume $\mathcal{A}$ terminates and outputs two distinct forgeries $(m_1, (\rho_1, \omega_1, \sigma_1, \delta_1))$ and $(m_2, (\rho_2, \omega_2, \sigma_2, \delta_2))$ linked to the same hash query. Consequently, it holds that $\omega_1 + \delta_1 = \omega_2 + \delta_2$. Hence, we have $g^{\rho_1} \cdot \mathbf{pk}^{\omega_1} = g^{\rho_2} \cdot \mathbf{pk}^{\omega_2}$ and $m_1 = m_2$. It follows that

$$\rho_1 + \text{dlog}_g \mathbf{pk} \cdot \omega_1 = \rho_2 + \text{dlog}_g \mathbf{pk} \cdot \omega_2,$$

hence

$$\text{dlog}_g \mathbf{pk} = \frac{\rho_1 - \rho_2}{\omega_2 - \omega_1}. \quad (2)$$

This equation is solvable if $\omega_2 \neq \omega_1$.

Since the signatures are distinct, one of the following cases must hold.

- **Case 1.** $\omega_1 \neq \omega_2$. In this case, Equation (2) is defined and solvable for $\text{dlog}_g \mathbf{pk}$, so the reduction succeeds.
- **Case 2.** $\omega_1 = \omega_2$. Since $\omega_1 + \delta_1 = \omega_2 + \delta_2$, it follows that $\delta_1 = \delta_2$. Since $g^{\rho_1} \cdot \mathbf{pk}^{\omega_1} = g^{\rho_2} \cdot \mathbf{pk}^{\omega_2}$ and $\omega_1 = \omega_2$, it follows that $g^{\rho_1} = g^{\rho_2}$, which implies $\rho_1 = \rho_2$. Similarly, since $g^{\sigma_1} \cdot z^{\delta_1} = g^{\sigma_2} \cdot z^{\delta_2}$ and $\delta_1 = \delta_2$, it follows that $g^{\sigma_1} = g^{\sigma_2}$, which implies $\sigma_1 = \sigma_2$. Consequently, $(\rho_1, \omega_1, \sigma_1, \delta_1) = (\rho_2, \omega_2, \sigma_2, \delta_2)$. This contradicts the premise that the adversary outputs two distinct signatures. Therefore, this case is impossible.

Consequently, Eq. (2) is defined and $\mathcal{R}_5$ can compute $\text{dlog}_g \mathbf{ch}$ successfully and win Game DLP. Thus, it holds $\varepsilon_{\text{DS}} := \varepsilon_{\text{DLP}}$.

F Partially Opening Blinded Pedersen Commitments

F.1 Proof of Same Discrete Logarithm

Construction 9 (Sigma Protocol for Proof of Same Discrete Logarithm). We describe how a prover $\mathcal{P}$ can prove in zero knowledge that the elements of a vector of group elements $\mathbf{h} = (h_1, \dots, h_n)$ have the same discrete logarithms to the bases of another vector of group elements $\mathbf{h}' = (h'_1, \dots, h'_n)$. That is $\text{dlog}_{h_1} h'_1 = \dots = \text{dlog}_{h_n} h'_n$.

- $\mathcal{P}_1(x = (\mathbf{h}, \mathbf{h}'), w = \gamma)$ The prover takes as input two vectors of the same length and a witness γ which is the discrete logarithm relation between them. It samples $r_{sdl} \leftarrow_{\$} \mathbb{Z}_p$ and sets $\mathbf{h}_{sdl} = \mathbf{h}^{r_{sdl}}$, i.e. the component wise exponentiation of $\mathbf{h}$ with r_{sdl} . It outputs the commitment vector $\mathbf{h}_{sdl}$ to the verifier.
- $\mathcal{V}_1(x = (\mathbf{h}, \mathbf{h}'), \mathbf{h}_{sdl})$ The verifier takes as input the statement and the prover's first message. It samples a random value $c \leftarrow_{\$} \mathbb{Z}_p$ and outputs it to $\mathcal{P}$.
- $\mathcal{P}_2(x = (\mathbf{h}, \mathbf{h}'), w = \gamma, r_{sdl}, c)$ The prover takes as input the statement $\mathbf{h}, \mathbf{h}'$, the witness γ , its internal state r_{sdl} and the verifier's challenge c . It computes $s_{sdl} = r_{sdl} - c \cdot \gamma$.
- $\mathcal{V}_2(x = (\mathbf{h}, \mathbf{h}'), \mathbf{h}_{sdl}, c, s_{sdl})$ The verifier checks that $\mathbf{h}_{sdl} = \mathbf{h}^{s_{sdl}} \cdot \mathbf{h}'^c$ where exponentiation is taken component wise. It returns 1 if the above equation holds and 0 otherwise.

Lemma 8 (Soundness of construction 9). *The protocol from construction 9 is $\frac{1}{p}$ -sound.*

Proof. Let $\mathbf{h}, \mathbf{h}'$ be two vectors such that there exists indices i, j such that $\gamma_i = \text{dlog}_{h_i} h'_i \neq \text{dlog}_{h_j} h'_j = \gamma_j$. Let $\mathbf{h}_{sdl}^*$ be an adversarial prover's first message. Let $r_i = \text{dlog}_{h_i} h_{sdl,i}^*$ and $r_j = \text{dlog}_{h_j} h_{sdl,j}^*$. Then, there exists at most one $c^* \in \mathbb{Z}_p$ such that there exists a s_{sdl} such that $r_i = s_{sdl} + \gamma_i \cdot c^*$ and $r_j = s_{sdl} + \gamma_j \cdot c^*$. The probability that the honest verifier samples $c = c^*$ is $\frac{1}{p}$.

Lemma 9 (Special Knowledge Soundness of construction 9). *The protocol from construction 9 is 2-special sound.*

Proof. Given two transcripts $\mathbf{h}_{sdl}, c_1, s_1$ and $\mathbf{h}_{sdl}, c_2, s_2$ with the same initial message $\mathbf{h}_{sdl}$, the extractor computes $\gamma = \frac{s_1 - s_2}{c_2 - c_1}$.

Lemma 10 (Extraction in the AGM). *Let $\mathcal{P}^*$ be an algebraic adversary acting as the prover in the protocol from construction 9 that interacts with a verifier. Then there exists an extractor $\mathcal{E}_{alg}$ that extracts a witness γ with probability $1 - \frac{1+\nu}{p}$ from the algebraic adversary.*

Proof. The algebraic prover takes as input the group elements $\mathbf{h}$. It outputs a vector $\mathbf{h}'$ along with algebraic representations $\mathbf{a}_i$ s.t. $h'_i = \prod h_i^{a_i}$. For each proof transcript, the prover outputs a vector $\mathbf{h}_{sdl}$ along with algebraic representations $\{z_i\}_{i \in [\nu]}$ for each element $h_{sdl,i}$ to the basis $\mathbf{h}$. the verifier then picks a challenge $c \leftarrow_{\$} \mathbb{Z}_p$ uniformly at random.

The prover responds with s such that for all $i \in [\nu]$, it holds that $h_i^s \cdot h_i'^c = h_{sdl,i}$.

By lemma 8, the probability that the prover succeeds in outputting a valid response when $\mathbf{h}, \mathbf{h}'$ are not in the language is $\frac{1}{p}$. In the following, we assume that this doesn't happen.

This yields ν linear equations of the form

$$\sum_{j=1}^{\nu} z_{i,j} \cdot \text{dlog} h_j = s \cdot \text{dlog} h_i + c \cdot \sum_{j=1}^{\nu} a_{i,j} \cdot \text{dlog} h_i \quad (3)$$

where the discrete logarithms are unknown as well as ν equations of the form $\gamma \cdot \text{dlog} h_i = \sum_{j=1}^{\nu} a_{i,j} \cdot \text{dlog} h_j$ where both γ and the discrete logarithms are unknown. If no fixed generator is given for the group, we assume w.l.o.g. that discrete logarithms are taken to the base h_1 (i.e. $\text{dlog} h_1 = 1$).

W.l.o.g we assume that all h_i are pairwise distinct and no h_i is the neutral element. Otherwise, we conflate the equal elements as well as their representations.

We consider the following cases:

1. $\exists i: a_{i,i} \neq 0 \wedge \forall i' \neq i: a_{i,i'} = 0$: In this case, the extractor outputs the witness $\gamma = a_{i,i}$.
2. The system of linear equations is linearly dependent. We consider the following subcases:

- a. The matrix A consisting of the representation vectors $\mathbf{a}_i$ has full rank. We make an information-theoretic argument why the system of equations has to be linearly dependent with probability at most $\frac{\nu}{p}$ or with probability 1 pver the choice of c .

First, we note that information-theoretically, the choice of c already fixes s . Also, information-theoretically γ is fixed by the vectors $\mathbf{h}$ and $\mathbf{h}'$ already, as well as the adversary's implicit choice of r_{sdl} through $\mathbf{h}_{sdl}$. The determinant of the matrix A with c, s as formal variables is a polynomial of total degree at most n . As c information-theoretically fixes s , we can reduce the number of formal variables in the determinant polynomial to 1 by replacing s with $r_{sdl} - c \cdot \gamma$. This yields a polynomial in one variable c of degree at most n . If this polynomial is the constant 0 polynomial, the probability of linear dependence is 1, i.e. for any pair c, s that fulfills the linear relation above, the matrix does not have full rank. In this case the extractor picks a random $c' \leftarrow_{\$} \mathbb{Z}_p$. It plugs it into the matrix and computes the determinant of the matrix with s as a variable (but not c). It then computes the roots s' of the resulting polynomial. This yields at most ν possible values of s' . The extractor tests (using the verification equation of the Σ -protocol) which one is the correct one. It then solves for the witness $\gamma = \frac{s-s'}{c'-c}$. With probability $1 - \frac{1}{p}$ it holds that $c' \neq c$ and thus the extraction succeeds.

If it is not the 0 polynomial, by the Schwartz-Zippel-Lemma, the probability of finding a root of the polynomial when randomly choosing c is at most $\frac{n}{p}$. In this case, the extractor cannot extract, so this happens with probability $\frac{\nu}{p}$.

- b. The matrix A consisting of the representation vectors $\mathbf{a}_i$ does not have full rank. In this case, there exists a linear combination $\mathbf{b}$ such that $\sum_{i=1}^{\nu} b_i \cdot \mathbf{a}_i = 0$. Due to soundness of the sigma protocol (implying correctness of the statement), this means that with probability $1 - \frac{1}{p}$ it holds that $\prod_{i=1}^{\nu} h_i^{b_i} = 0$. We therefore know a way to express one of the h_i as a linear combination of the others. We remove this h_i from all systems of equations and replace it with its linear representation of the other h_i . This yields systems of equations with one variable less.

We consecutively apply this transformation until the matrix A' resulting from these transformations has full rank. If A' has rank 1, we proceed as in Case 1. Otherwise we continue as in Case 2.a.

3. Otherwise: As we excluded Case 1 and Case 2, the system of equations is linearly independent. The extractor can thus compute the discrete logarithms of all h_i w.r.t. h_1 by solving the linear equation system. It can use the representation of h'_1 to compute $\gamma = \text{dlog}_{h_1} h'_1$ and output γ .

Overall, the failure probability of the extractor can be bounded by $\frac{\nu}{p} + \frac{1}{p}$.

Lemma 11 (HVZK of construction 9). *The protocol from construction 9 fulfills special honest verifier zero-knowledge.*

Proof. We give a simulator. On input of a challenge c and a statement $\mathbf{h}, \mathbf{h}'$, the simulator samples $s_{sdl} \leftarrow_{\$} \mathbb{Z}_p$ and computes $\mathbf{h}_{sdl} := \mathbf{h}^{s_{sdl}} \cdot \mathbf{h}'^c$ where exponentiation and multiplication is component wise. It is easy to see that for $\mathbf{h}, \mathbf{h}'$ where there exists γ such that $\mathbf{h}^\gamma = \mathbf{h}'$ the transcripts generated by the simulator are identically distributed to real transcripts that use c as a challenge.

F.2 Proofs of Partial Openings

Construction 10 (Protocol for Partially Opening a Pedersen Commitment). We describe a sigma protocol that allows for partially opening a generalized Pedersen Commitment while proving knowledge of the remainder of the opening.

Let $\mathbf{h}$ be the base vector of the Pedersen commitment, and $\mathbf{a}$ be the full opening. Let C be the Pedersen commitment obtained by $C := \prod_{i=1}^n h_i^{a_i}$. For simplicity, we treat the randomness used for committing as part of the full opening. Let I be a set of indices where the commitment is to be opened.

- $\mathcal{P}_1(\mathbf{h}, \mathbf{a}, I)$ For all $j \notin I$, the prover samples $r_j \leftarrow_{\$} \mathbb{Z}_p$. It then computes $R := \prod_{j \notin I} h_j^{r_j}$. It outputs R and $A_I := \{a_i : i \in I\}$ to the verifier.
- $\mathcal{V}_1(\mathbf{h}, C, A_I, R)$ The verifier takes the prover's first message as input along with the commitment and the parameters $\mathbf{h}$ and samples $c \leftarrow_{\$} \mathbb{Z}_p$. It sends c to the prover.
- $\mathcal{P}_2(\mathbf{h}, \mathbf{a}, I, \{r_j : j \notin I\}, c)$ The prover takes the commitment parameters $\mathbf{h}$, the opening $\mathbf{a}$, the opening indices i , and the randomness for his first message r_j , along with the challenge from the verifier and for all $j \notin I$ computes $s_j := r_j - c \cdot a_j$. It then outputs $S = \{s_j : j \notin I\}$ to the verifier.
- $\mathcal{V}_2(\mathbf{h}, C, A_I, R, c, S)$ The verifier computes $C' = \frac{C}{\prod_{i \in I} h_i^{a_i}}$ and then checks that $R = C'^c \cdot \prod_{j \notin I} h_j^{s_j}$. It outputs 1 if this equality holds and 0 otherwise.

Lemma 12 (2-special soundness of construction 10). *The protocol from construction 10 is 2-special sound.*

Proof. Given two transcripts (R, c_1, S_1) and (R, c_2, S_2) with the same initial message, the extractor $\mathcal{E}$ computes the witness a_i for each $i \in I$ as $a_i = \frac{s_{1,i} - s_{2,i}}{c_2 - c_1}$.

Lemma 13 (HVZK of construction 10). *The protocol from construction 10 is HVZK.*

Proof. The simulator works similar to that from lemma 11.

Definition 24. *Let $\mathbf{h}$ be the public parameters of a Pedersen commitment and g the generator of the group over which the Pedersen commitment is instantiated. For a Pedersen commitment C , $C^\gamma \cdot g^{r^\gamma}$ is a blinded Pedersen commitment with public parameters $\mathbf{h}^\gamma, g^\gamma$.*

Lemma 14 (Extraction in the AGM). *Let $\mathcal{P}^*$ be an algebraic adversary acting as the prover in the protocol from construction 10 that interacts with a verifier. Then there exists an extractor $\mathcal{E}_{alg}$ that extracts a witness $\{a_i : i \notin I\}$ or a solution to the BINDING game of Pedersen commitments with probability $1 - \frac{2}{p}$ from the algebraic adversary.*

Proof. The adversary outputs a commitment C along with a representation $\mathbf{y}$ such that $C = \prod_{i=1}^\nu h_i^{y_i}$. In the first round of the protocol, the adversary $\mathcal{P}^*$ outputs R along with a representation $\mathbf{z}$ such that $R = \prod_{i=1}^\nu h_i^{z_i}$ as well as I, A_I .

We consider the following cases:

1. For all $i \in I$ it holds that $z_i = 0$: Then, the extractor simply computes all a_i where $i \notin I$ $a_i = \frac{z_i - s_i}{c}$. This works if $c \neq 0$ which happens with probability $1 - \frac{1}{p}$.
2. There exists at least one $i \in I$ such that $z_i \neq 0$.
 - a. For all $i \in I$ it holds that $y_i = a_i$. In this case, it holds that $C = \prod_{i \in I} h_i^{a_i} \cdot \prod_{i \notin I} h_i^{y_i}$, and the extractor outputs $\{y_i : i \notin I\}$ as the witness.
 - b. This yields an equation of the form $\prod_{i=1}^\nu h_i^{c \cdot y_i} \cdot \prod_{i \notin I} h_i^{s_i} = \prod_{i=1}^\nu h_i^{z_i}$. With probability $1 - \frac{1}{p}$ over the choice of c it holds that there exists $i \in I$ such that $z_i \neq c \cdot y_i$. Thus, the extractor can output a BINDING solution, namely the commitment R along with $\mathbf{z}$ and $\mathbf{z}'$ where $z'_i = c \cdot y_i$ for $i \in I$ and $z'_i = c \cdot y_i + s_i$ for $i \notin I$.

Theorem 4 (Opening a Blinded Pedersen Commitment). *For any algebraic adversary $\mathcal{U}$ that on input of commitment parameters $\mathbf{h}$ outputs blinded commitment parameters Ψ' , a commitment ζ_1 , a partial vector $\mathbf{L}'$ that has non- $\perp$ entries at index I a proof $\pi = (\pi_{sdl}, \pi_{op})$ auxiliary information aux such that $\pi = (\pi_{sdl}, \pi_{op})$ is the Fiat-Shamir transformation (see construction 5) of the AND-proof (see lemma 4) of the protocols described in constructions 9 and 10 along with group element representations for all group elements output by the adversary $\mathcal{U}$, there exists an extractor $\mathcal{E}$ that on input of the adversary's output including all algebraic representations that were made during the output or to the Random Oracle, outputs a witness γ with probability $1 - Q_h \frac{1+n}{p}$ (over the choice of the random oracle) and (if the former happens) additionally either a witness $\mathbf{L}$ which is a partial opening to the commitment ζ_1 on the indices $i \notin I$, or a solution $R, \mathbf{z}, \mathbf{z}'$ to the BINDING game of Pedersen commitments with probability $1 - \frac{2}{p}$ over the choice of the random oracle.*

Proof. The extractor proceeds as follows.

First, it calls the extractor from lemma 10 on the blinded commitment parameters transcript π_{sdl} along with the commitment parameters $\mathbf{h}$ and the blinded parameters ψ . This extractor outputs a witness γ with probability $1 - Q_h \cdot \frac{1+n}{p}$ as the adversary may query the RO Q_h many times, so we union bound over the choice of the challenges for the Σ -protocol transcript.

The extractor then uses γ to transform the commitment ζ_1 into $C = \zeta_1^{\frac{1}{\gamma}}$ and R_{op} from π_{op} into $R'_{op} = R_{op}^{\frac{1}{\gamma}}$. It transforms the algebraic representations by dividing by γ . It then runs the extractor from lemma 14 to extract either a witness or a binding game solution. It outputs the outputs of the two extractors.

F.3 Double-Spending Definition

In the following, we extend the definition of blind signature with attributes (Def. 1) to also support **double-spend protection**, by modifying the user side registration algorithm, $\mathcal{RG}_U$, and defining additional algorithms for double-spending detection, $\mathcal{DSI}$ (the *double-spend identification algorithm*) and $\mathcal{DSV}$ (the *double-spend verification algorithm*), resulting in a type of scheme called BSADS = $(\mathcal{G}, \mathcal{RG}, \mathcal{S}, \mathcal{U}, \mathcal{V}, \mathcal{SH}, \mathcal{DSI}, \mathcal{DSV})$. Unless stated otherwise below, the algorithms of a BSADS have the same input and output behaviour, correctness, and security notions as BSA.

- The user’s registration algorithm $\mathcal{RG}_U$ takes as input a vector of attributes $\mathbf{L}$ where the first attribute, denoted by ID , is a unique identifier of the user. It outputs a commitment a user’s registration state $\mathbf{rst}_U$ and a proof π . We assume $\mathbf{rst}_U$ contains a commitment C of $\mathbf{L}$ and access it as $\mathbf{rst}_U.C$. We use the proof of knowledge of a partial opening as described in construction 10 to reveal ID .
- The signer’s issuance algorithm $\mathcal{S}$ takes as input a secret key $\mathbf{sk}$, a registration state $\mathbf{rst}_S$, and a commitment C , and outputs 1 (COMPLETE) or $\perp$ (ERROR). Additionally, it outputs a signing state st_S if it outputs 1.
- The double-spend identification algorithm $\mathcal{DSI}$ takes as input the issuer’s public key $\mathbf{pk}$, a signature σ , the signer’s internal state st_S , and two different presentation proofs π_1, π_2 . It either outputs a user ID and a proof of guilt π_Q , or $\perp$.
- The double-spend verification algorithm $\mathcal{DSV}$ takes as input a user ID and a proof of guilt π_Q and it outputs 1 (ACCEPT) or 0 (REJECT).

Discussion. The specifics of the registration protocol will depend on the exact policy of each application scenario. For simplicity, in our definition above, we assume that the first attribute of the vector $\mathbf{L}$ is a unique identifier of the user, denoted as ID , and this uniqueness is guaranteed by the attached proof π . For example, in certain cases, during the registration process, the user might present a physical document, such as a passport, and prove (e.g., by partially opening the commitment C) that the encoded attributes match those in their passport. In such scenarios, ID could be the unique passport number. Regardless

of the specific implementation, ID must be unique for each user and should be a value that cannot be duplicated or obtained by another user to prevent framing attacks where a malicious user attempts to impersonate another by registering under their identity and engaging in fraudulent activities.

A blind signature with attributes and double-spend protection BSADS needs satisfy two additional security properties: double-spending identification and exculpability.

Double-spending identification guarantees that no adversary is able to present the same token twice without having their public key revealed.

Definition 25 (Double-spending Identification). *For a blind signature with attributes and double-spending protection $BSADS = (\mathcal{G}, \mathcal{RG}, \mathcal{S}, \mathcal{U}, \mathcal{V}, \mathcal{SH}, \mathcal{DSI}, \mathcal{DSV})$, and an adversary $\mathcal{A}$, we define the game $\mathcal{DSP}$ as follows.*

1. Generate a pair of keys $(pk, sk) \leftarrow_{\$} \mathcal{G}(pp)$, and invoke $\mathcal{A}^{\mathcal{RG}(\cdot), \mathcal{S}(sk, \cdot)}$.
2. $\mathcal{A}$ may query its oracles $\mathcal{RG}_S(\cdot), \mathcal{S}(sk, \cdot)$ arbitrarily and in an interleaved fashion.
3. The game outputs 1 iff $\mathcal{A}$ outputs: $(\sigma, \mathbf{L}'_1, \bar{\pi}_1)$ and $(\sigma, \mathbf{L}'_2, \bar{\pi}_2)$ such that $\mathbf{L}'_1 \neq \mathbf{L}'_2 \vee \bar{\pi}_1 \neq \bar{\pi}_2$ $\mathcal{SH}_V(\sigma, \mathbf{L}'_1, \bar{\pi}_1) = 1$ and $\mathcal{SH}_V(\sigma, \mathbf{L}'_2, \bar{\pi}_2) = 1$ and $\mathcal{DSI}(\sigma, \bar{\pi}_1, \bar{\pi}_2) = \perp$.

We define $\mathcal{A}$'s advantage in winning the game $\mathcal{DSP}$ against a blind signature scheme with attributes BSADS as

$$\mathbf{Adv}^{\mathcal{DSP}}_{BSADS}(\mathcal{A}) := \Pr[\mathcal{DSP}_{BSADS}^{\mathcal{A}} = 1].$$

We say that BSADS is (t, ε) - $\mathcal{DSP}$ -secure if for all adversaries $\mathcal{A}$ that run in time at most t , $\mathbf{Adv}^{\mathcal{DSP}}_{BSADS}(\mathcal{A}) \leq \varepsilon$.

Finally, we also define exculpability which guarantees that a malicious signer cannot wrongly accuse an honest user of double-spending.

Definition 26 (Exculpability). *For a blind signature with attributes and double-spending protection $BSADS = (\mathcal{G}, \mathcal{RG}, \mathcal{S}, \mathcal{U}, \mathcal{V}, \mathcal{SH}, \mathcal{DSI}, \mathcal{DSV})$, and an adversary $\mathcal{A}$, we define the game $\mathcal{Exc}$ as follows.*

1. Invoke $pk \leftarrow_{\$} \mathcal{A}$.
2. Invoke $\mathcal{A}^{\mathcal{RG}_U(\cdot), \mathcal{U}(\cdot), \mathcal{SH}_U(\cdot)}$ and allow it to query its oracles arbitrarily and in an interleaved fashion.
3. Initialize Q_U to hold the set of honest user IDs generated by the oracle calls.
4. Answer the oracle queries as follows:
 - a. the registration oracle $\mathcal{RG}_U(\cdot)$ takes as input an attributes vector $\mathbf{L}$ with a unique first attribute (recall that ID is the first attribute) and runs the user's attributes registration algorithm $\mathcal{RG}_U$ on it to obtain (rst_U, π) . It outputs $(rst_U.C, \pi)$ to $\mathcal{A}$.
 - b. The user oracle $\mathcal{U}(\cdot)$ takes as input a commitment C and a message m . If $\mathcal{A}$ previously queried the registration oracle $\mathcal{RG}_U$ with output state rst_U such that $rst_U.C = C$, the oracle simulates the user's algorithm for $\mathcal{A}$ and returns the resulting signature on m to $\mathcal{A}$. Otherwise, it outputs $\perp$. The showing state ost_U is stored by the experiment.

- c. The showing oracle $\mathcal{SH}_U(\cdot)$ takes as input a signature σ from the outputs of the user oracle (it outputs $\perp$ if σ was not output by U) and a set I_{op} of indices to open. If the signature has never been queried to the showing oracle before, it runs the showing algorithm $\mathcal{SH}_U$ for this signature using the internal states rst_U and ost_U it kept from registration and signature generation.
5. The game outputs 1 iff $\mathcal{A}$ outputs: (ID, π_Q) such that $ID \in Q_U$ and $\text{DSV}(ID, \pi_Q) = 1$.

We define $\mathcal{A}$'s advantage in winning the game Exc against a blind signature scheme with attributes BSADS as

$$\text{Adv}_{\text{BSADS}}^{\text{Exc}}(\mathcal{A}) := \Pr[\text{Exc}_{\text{BSADS}}^{\mathcal{A}} = 1].$$

We say that BSADS is (t, ε) -Exc-secure if for all adversaries $\mathcal{A}$ that run in time at most t , $\text{Adv}_{\text{BSADS}}^{\text{Exc}}(\mathcal{A}) \leq \varepsilon$.

G Anonymous Credentials Light

We now present Anonymous Credentials light scheme of [4] along with protocols for partial attribute showing and double spending detection.

G.1 The Main ACL Construction

We first recall the Anonymous Credentials Light scheme from [4]. For the reader's convenience, we have included a figure of the signing interaction in fig. 6 in appendix I.

Let $\mathbb{G}$ be a group with generator g and prime order p described by parameters $\text{pp}_{\mathbb{G}}$ and hash functions $H_{\text{pp}}, H_1: \{0, 1\}^* \rightarrow \mathbb{G}$, $H_{\text{reg}}, H_{\text{sh}}, H_3: \{0, 1\}^* \rightarrow \mathbb{Z}_p$ modeled as random oracles. Let rst_S and rst_U be empty lists. Then the scheme $\text{ACL} = (\mathcal{G}, \mathcal{RG}, S, U, \mathcal{V}, \mathcal{SH})$ is defined as follows:

- $\mathcal{G}(\text{pp}_{\mathbb{G}})$: Sample $x \leftarrow_{\$} \mathbb{Z}_p, h \leftarrow_{\$} \mathbb{G}$, compute $y := g^x$, generate $z := H_1(g, h, y)$, set $\text{sk} := x$ and $\text{pk} := (g, h, y, z)$, and output (pk, sk) .
- $\mathcal{RG}_U(\mathbf{L})$: Parse $(L_1, \dots, L_n) \leftarrow \mathbf{L}$, obtain $h_0, \dots, h_n$ by querying $0, \dots, \nu$ to H_{pp} , sample $r_{\text{com}} \leftarrow_{\$} \mathbb{Z}_p$ and compute $C := h_0^{r_{\text{com}}} \cdot h_1^{L_1} \cdot \dots \cdot h_n^{L_n}$. Compute a proof of knowledge π of an opening of C , using the Fiat-Shamir transform of the protocol described in construction 3 with random oracle H_{reg} , set $\text{rst}_U = (C, r_{\text{com}}, \mathbf{L})$, and outputs (rst_U, π) .
- $\mathcal{RG}_S(C, \pi, \text{rst}_S)$: Verify that π is a proof of knowledge of an opening of C using the verification of the protocol described in construction 3. If the proof verifies, output an updated signer registration state $\text{rst}_S := \text{rst}_S' \cup \{C\}$ else output $\perp$ (ERROR). Return 1 (COMPLETE) or 0 (ERROR) to the user.
- S : We describe the interactive signer algorithm in two parts, S_1 and S_2 :
 - $S_1(\text{sk}, \text{rst}_S, C)$: Abort and output $\perp$ if $C \notin \text{rst}_S$. Otherwise, sample $d, s_1, s_2, u, \text{rnd} \leftarrow_{\$} \mathbb{Z}_p$, generate $z_1 := g^{\text{rnd}} \cdot C, z_2 := z/z_1$, compute $a := g^u, b_1 := g^{s_1} \cdot z_1^d, b_2 := h^{s_2} \cdot z_2^d$, set internal state $st_S := (d, s_1, s_2, u), R := (\text{rnd}, a, b_1, b_2)$, and output (R, st_S) .

- $\mathcal{S}_2(\text{sk}, st_{\mathcal{S}}, R, e)$: Parse $d, s_1, s_2, u \leftarrow st_{\mathcal{S}}, x := \text{sk}$, compute $c := e - d \bmod q$ and $r := u - c \cdot x \bmod q$, and return $S := (c, d, r, s_1, s_2)$.
- $\mathcal{S}$ outputs 1 (COMPLETE) if $\mathcal{S}_2$ outputs S .
- $\mathcal{U}$: The user algorithm consists of two algorithms $\mathcal{U}_1$ and $\mathcal{U}_2$:
 - $\mathcal{U}_1(\text{pk}, \text{rst}_{\mathcal{U}}, m, R)$: Parse $(\text{rnd}, a, b_1, b_2) \leftarrow R$, sample $\gamma \leftarrow_{\$} \mathbb{Z}_p^*, \tau, t_1, t_2, t_3, t_4, t_5 \leftarrow_{\$} \mathbb{Z}_p$, generate $z_1 := g^{\text{rnd}} \cdot \text{rst}_{\mathcal{U}}.C, \zeta := z^{\gamma}, \zeta_1 := z_1^{\gamma}, \zeta_2 := \zeta / \zeta_1$, compute $\alpha := a \cdot g^{t_1} \cdot y^{t_2}, \beta_1 := b_1^{\gamma} \cdot g^{t_3} \cdot \zeta_1^{t_4}, \beta_2 := b_2^{\gamma} \cdot h^{t_5} \cdot \zeta_2^{t_4}, \eta := z^{\tau}$, generate $\varepsilon := H_3(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$, compute $e := \varepsilon - t_2 - t_4 \bmod q$, set internal state $st_{\mathcal{U}} := (\gamma, \tau, t_1, t_2, t_3, t_4, t_5, \text{rnd})$, and return $(e, st_{\mathcal{U}})$.
 - $\mathcal{U}_2(\text{pk}, st_{\mathcal{U}}, R, e, S, m)$: Parse $(\gamma, \tau, t_1, t_2, t_3, t_4, t_5, \text{rnd}) \leftarrow st_{\mathcal{U}}$, and $(c, d, r, s_1, s_2) \leftarrow S$, compute $\rho := r + t_1, \omega := c + t_2, \sigma_1 := \gamma \cdot s_1 + t_3, \sigma_2 := \gamma \cdot s_2 + t_5, \delta := d + t_4, \mu := \tau - \delta \cdot \gamma$, and return the signature $\hat{\sigma} := (\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \delta, \mu)$ and $\text{ost}_{\mathcal{U}} = (\gamma, \text{rnd})$ if $\omega + \delta \equiv H_3(\zeta, \zeta_1, g^{\rho} \cdot y^{\omega}, g^{\sigma_1} \cdot \zeta_1^{\delta}, h^{\sigma_2} \cdot \zeta_2^{\delta}, z^{\mu} \cdot \zeta^{\delta}, m), \perp$ otherwise.
- $\mathcal{SH}_{\mathcal{U}}, \mathcal{SH}_{\mathcal{V}}$: see fig. 4.
- $\mathcal{V}(\text{pk}, m, \hat{\sigma})$: Parse $\hat{\sigma}$ as $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \delta, \mu)$, and output 1 if $\zeta \neq 1$ and $\omega + \delta \equiv H_3(\zeta, \zeta_1, g^{\rho} \cdot y^{\omega}, g^{\sigma_1} \cdot \zeta_1^{\delta}, h^{\sigma_2} \cdot (\zeta / \zeta_1)^{\delta}, z^{\mu} \cdot \zeta^{\delta}), 0$ otherwise.

G.2 ACL Showing of Partial Attributes

Showing and verification of showing is depicted in fig. 4. We note that it is also possible to “show” a signature without revealing any attributes, i.e. when I_{op} is empty. In this case, the user can run a proof of same discrete logarithm (see construction 9) for the blinded commitment parameters, as well as a proof of knowledge of an opening of a Pedersen commitment to the blinded parameters like the one used at registration.

The procedure described in fig. 4 allows for extraction in the AGM:

Theorem 5. *For any algebraic adversary $\mathcal{U}$ that, on input of commitment parameters $\text{pp}_{\text{GPC}} = \mathbf{h}$, it outputs an ACL public key pk along with a tuple $(m, \sigma, \mathbf{L}', I_{op}, \bar{\pi})$ such that $\mathcal{SH}_{\mathcal{V}}(\text{pk}, m, \sigma, \mathbf{L}', I_{op}, \bar{\pi}) = 1$, there exists an extractor $\mathcal{E}$ that takes $\mathcal{U}$ ’s output $(\text{pk}, (m, \sigma, \mathbf{L}', I_{op}, \bar{\pi}))$ along with all algebraic representations $\mathcal{U}$ outputs as input, and it outputs a witness γ of the proof of same discrete logarithm with probability $1 - Q_h^{\frac{1+n}{p}}$ (over the choice of the random oracle) and (if the former happens) additionally either a witness $\mathbf{L}$ which is a partial opening to the commitment ζ_1 on the indices $i \notin I$, or a solution $R, \mathbf{z}, \mathbf{z}'$ to the BINDING game of Pedersen commitments with probability $1 - \frac{2}{p}$ over the choice of the random oracle.*

Proof. This follows directly from theorem 4.

G.3 Secure Showing of ACL

Theorem 6. *If the discrete logarithm problem is (t, ε) -hard in the group $\mathbb{G}$, then ACL is (t', ε') -SS secure according to definition 5 with arbitrary messages*

$\mathcal{SH}_U(\text{pk}, \text{rst}_U, m, \sigma, \text{ost}_U, I_{op})$	$\mathcal{SH}_V(\text{pk}, \sigma, m, \mathbf{L}', I_{op}, \bar{\pi})$
1 : $(C, r_{com}, \mathbf{L}) := \text{rst}_U$	1 : $(\pi_{sdl}, \pi_{op}) := \bar{\pi}$
2 : $(\text{rnd}, \gamma) := \text{ost}_U$	2 : if $\mathcal{V}(\text{pk}, m, (\sigma, \zeta_1)) = 0$
3 : $\Gamma := g^\gamma$	3 : $\vee (\exists i \in I_{op} : L'_i = \perp) \vee (\exists i \notin I_{op} : L'_i \neq \perp) :$
4 : $\Psi := \{\psi_k := h_k^\gamma\}_{k \in [\nu]}$	4 : return 0
5 : $r_{sdl} \leftarrow_{\$} \mathbb{Z}_p$	5 : $(\Psi, \Gamma, g_{sdl}, z_{sdl}, s_{sdl}, h_{sdl}) := \pi_{sdl}$
6 : $\mathbf{h}_{sdl} := \{h_{sdl,k} := h_k^{r_{sdl}}\}_{k \in [\nu]}$	6 : $(\psi_1, \dots, \psi_\nu) := \Psi$
7 : $z_{sdl} := z^{r_{sdl}}$	7 : $(h_{sdl,1}, \dots, h_{sdl,\nu}) := h_{sdl}$
8 : $g_{sdl} := g^{r_{sdl}}$	8 : $c \leftarrow_{\$} \text{H}_{sh}(\text{pk}, \sigma, \zeta_1, m, \Psi, \Gamma, g_{sdl}, z_{sdl}, \mathbf{h}_{sdl}, R, \mathbf{L}')$
9 : $r_g, \{r_i\}_{i \in [\nu]_0 \setminus I_{op}} \leftarrow_{\$} \mathbb{Z}_p$	9 : if $\Gamma^c \cdot g^{s_{sdl}} \neq g_{sdl} \vee \zeta^c \cdot z^{s_{sdl}} \neq z_{sdl}$
10 : $R := g^{r_g} \cdot \prod_{i \in [\nu]_0 \setminus I_{op}} \psi_i^{r_i}$	10 : $\vee \bigvee_{i \in [\nu]} \psi_i^c \cdot h_i^{s_{sdl}} \neq h_{sdl,i} :$
11 : for $i \in [\nu] \setminus I_{op} :$	11 : return 0
12 : $L'_i := \perp$	12 : $(R, S) := \pi_{op}$
13 : for $i \in I_{op} :$	13 : $\zeta'_1 := \zeta_1 / \prod_{i \in I_{op}} \psi_i^{L'_i}$
14 : $L'_i := L_i$	14 : if $\zeta_1'^c \cdot \Gamma^{s_\Gamma} \cdot \prod_{i \in [\nu]_0 \setminus I_{op}} \psi_i^{s_i} \neq R :$
15 : $c \leftarrow_{\$} \text{H}_{sh}(\text{pk}, \sigma, \sigma, \zeta_1, m, \Psi, \Gamma, g_{sdl}, z_{sdl}, \mathbf{h}_{sdl}, R, \mathbf{L}')$	15 : return 0
16 : $s_{sdl} := r_{sdl} - c \cdot \gamma$	16 : return 1
17 : $s_0 := r_0 - c \cdot r_{com}$	
18 : $\{s_i := r_i - L_i \cdot c\}_{i \in [\nu]_0 \setminus I_{op}}$	
19 : $s_\Gamma := r_g - \text{rnd} \cdot c$	
20 : $S := \{s_i\}_{i \in I_{op}} \cup \{s_\Gamma\}$	
21 : $\pi_{sdl} := (\Psi, \Gamma, g_{sdl}, z_{sdl}, s_{sdl}, h_{sdl})$	
22 : $\pi_{op} := (R, S)$	
23 : return $(\mathbf{L}', (\pi_{sdl}, \pi_{op}))$	

Fig. 4: The Showing procedure. The ζ_1 component of the signature σ , denoted here by $\sigma.\zeta_1$ is a blinded commitment C , for each a signature σ was issued via an interaction with an honest signer. (γ, rnd) is the blinding factor such that $\zeta_1 = (g^{\text{rnd}} \cdot C)^\gamma$. The proof consists of **a part that proves the correct blinding of the commitment parameters via a proof of same discrete logarithm** and of **a part that proves knowledge of an opening part of the commitment that is not revealed by the set of opened indices I_{op} .**

(item 4.c) and selective reveal (item 4.d.ii) in the AGM + ROM with $t' \approx t$ and

$$\varepsilon' \leq (\nu + 21) \cdot \varepsilon + \frac{(\nu + 3) \cdot \ell \cdot Q_H + 52 + 4 \cdot Q_C}{p} + \frac{1}{\nu},$$

where ℓ is the number of signing sessions closed, Q_H is the number of hash queries the adversary made to H_{sh} , Q_C is the number of commitments $\mathcal{A}$ reg-

isters, and ν is the maximal number of attributes contained in each attribute vector.

Proof. We work in the AGM and show that an adversary that breaks the secure showing property can be used to break the discrete logarithm assumption in the underlying group. Our proof technique varies depending on the adversary's strategy for breaking the secure showing property - namely it can either provide a set of signatures where for some signatures, the blinded ζ_1 components do not match the $C \cdot g^{\text{rnd}}$ components of the signing sessions. This can happen either in the form of one "odd one out" signature that matches no session (we rule this out in $\mathbf{G}_1$), or in two signatures that match the same signing session (we rule this out in $\mathbf{G}_2$). The last possible strategy is for the adversary to break the binding property of the underlying Pedersen commitment scheme which we rule out in lemma 17.

Game $\mathbf{G}_0$. This is the secure showing game for ACL.

Game $\mathbf{G}_1$. This game aborts if the following happens: The adversary outputs at least one signature σ^* such that the commitment ζ^*, ζ_1^* does not "match" any of the session tags on the signer's side. This means, there exists no $z_{1,i}$ such that $\text{dlog}_z \zeta^* = \text{dlog}_{z_{1,i}} \zeta_1^*$.

We will show that this contradicts the so-called *restrictive blinding lemma*, see lemma 15. We prove this lemma using mostly techniques from [37] to show that an adversary can be used to break DLP if it wins this game. A formal definition of the game RB_{ACL} can be found in definition 27 in appendix H.

Lemma 15 (Restrictive Blinding). *In the ROM and AGM, if DLP is (t, ε) -hard in $\mathbb{G}$, it holds that for any adversary running in time at most t , the advantage is at most*

$$\text{Adv}_{\text{ACL}}^{\text{RB}}(\mathcal{A}) \leq 13 \cdot \varepsilon + \frac{4 \cdot Q_C + 31}{p} + \frac{1}{\nu}.$$

where Q_C is the number of registration queries made by the adversary.

The proof can be found in appendix H

Corollary 1. $\Pr[\mathbf{G}_0 = 1] \geq \Pr[\mathbf{G}_1 = 1] + 13 \cdot \varepsilon + \frac{4 \cdot Q_C + 31}{p} + \frac{1}{\nu}.$

Proof. We provide a reduction: The reduction simulates the secure showing game to the adversary by forwarding its queries to all oracles and their responses. The reduction then checks whether the adversary has won the secure showing game. If yes, it outputs the messages and signatures from the adversary's output to the RB game. It thus wins whenever the abort condition of $\mathbf{G}_1$ occurs.

Game $\mathbf{G}_2$. In this game, we abort if the adversary does the following: The adversary outputs at least two signatures σ', σ'' such that the commitments ζ', ζ_1' and ζ'', ζ_1'' map to the same signing session, i.e. there exists a single $z_{1,i}$ such that both $\text{dlog}_z \zeta' = \text{dlog}_{z_{1,i}} \zeta_1'$ and $\text{dlog}_z \zeta'' = \text{dlog}_{z_{1,i}} \zeta_1''$. We will show that this contradicts what we call the *2-to-1-lemma*, see lemma 16 - this lemma can be derived from techniques used in the OMUF-proof of [37]. A formal definition of the game $\ell\text{-TTO}_{\text{ACL}}$ can be found in definition 29 in appendix H.

Lemma 16 (2-to-1 blinding). *In the ROM and AGM, if DLP is (t, ε) -hard in $\mathbb{G}$, it holds that for any adversary running in time at most t , the advantage is at most*

$$\text{Adv}_{\text{ACL}}^{\ell\text{-TTO}}(\mathcal{A}) \leq 7 \cdot \varepsilon + \frac{20}{p}.$$

The proof can be found in appendix H

We now apply this to the current game hop.

Corollary 2.

$$\Pr[\mathbf{G}_1 = 1] \leq \Pr[\mathbf{G}_2 = 1] + (7\varepsilon + \frac{20}{p}).$$

Proof. Again, the reduction forwards all queries from the adversary to the $\ell\text{-TTO}$ and also forwards the responses. When the adversary outputs a solution, it checks that the adversary has won the secure showing game, and then outputs the messages and signatures from the adversary's output to the $\ell\text{-TTO}$ game. It thus wins whenever the abort condition of $\mathbf{G}_2$ occurs.

Lemma 17 (Partial Opening). *In the ROM and AGM, if DLP is (t, ε) -hard in $\mathbb{G}$, it holds that for any adversary $\mathcal{A}$ that runs in time at most t , the probability of winning $\mathbf{G}_2$ is at most:*

$$\Pr[\mathbf{G}_2^{\mathcal{A}}] \leq (\nu + 1) \cdot \varepsilon + \frac{(\nu + 3) \cdot \ell \cdot Q_{\text{H}_{\text{sh}}} + 1}{p}$$

where ℓ is the number of signing sessions closed by the adversary $\mathcal{A}$ and $Q_{\text{H}_{\text{sh}}}$ is the number of hash queries the adversary makes to the random oracle H_{sh}

Proof. We construct an adversary $\mathcal{R}_5$ against the binding property of the Pedersen commitment C . The adversary simulates the signing procedure to the adversary using the y -side key like an honest signer would, and extracts openings for the commitments made by the adversary. We describe this below in more details.

First of all, the reduction constructs a wrapper $\mathcal{B}$ around the adversary which takes as input the commitment parameters and has access to the random oracle H_{sh} .

Setup. The wrapper $\mathcal{B}$ takes as input the commitment parameters $g, \mathbf{h}$ for the Pedersen commitment. It samples a key for ACL using the $\mathcal{G}(\text{pp}_{\mathbb{G}})$ algorithm while programming the RO H_1 so that it knows the discrete logarithms of its outputs (i.e. including z). It outputs the key to the adversary $\mathcal{A}$ and programs the random oracle H_{pp} to output h_i when queried on i for $i \in [\nu]$.

Online Phase The wrapper simulates the secure showing game to the adversary by lazy-sampling all random oracles (in case of group elements with known discrete logarithm to base g) and by using the secret key to answer signing queries like an honest signer would. Whenever the adversary makes a query to a random oracle that contains a group element, the wrapper $\mathcal{B}$ takes a note of the algebraic representations submitted for that group element. It computes a reduced representation where it replaces all occurrences of $y, z, h, a_i, b_{1,i}, b_{2,i}$ with their respective discrete logarithms to base $g, \mathbf{h}$, concretely, replace y by g^x , z by $g^{\text{dlog}z}$ (known from programming $\mathbf{H}_1$), h by $\text{dlog}h$, a_i by g^{u_i} , $b_{1,i}$ by $g^{s_{1,i}+d_i \cdot \text{rnd}_i} \cdot C_i^{d_i}$, $b_{2,i}$ by $g^{s_{2,i} \cdot \text{dlog}h} \cdot (g^{\text{dlog}z} / (g^{\text{rnd}_i} \cdot C_i))^{d_i}$ where C_i is recursively replaced by replacing the values in its representation. Note that by the time $b_{1,i}, b_{2,i}$ is output, the representation of C_i is already fixed and thus there exist no circular dependencies.

Output determination The wrapper outputs whatever the adversary outputs, that is the public key pk , a list of tuples $\{(m_i, \sigma_i, \zeta_{1,i}, L_i, \pi_i)\}_{i \in k}$ and tuples $\{(L'_i, r_i)\}_{i \in [\ell]}$, alongside the algebraic representations that the adversary output the first time it output a specific group element (i.e. if the group element was queried to a random oracle before, the wrapper $\mathcal{B}$ outputs the representation that was submitted along with the RO query).

The reduction now employs the extractor $\mathcal{E}$ from theorem 5 of Π for each of the proofs $\pi_i = (\pi_{i,\text{sdl}}, \pi_{i,\text{op}})$ along with the commitments and algebraic representations that $\mathcal{B}$ has output. Note that $\mathcal{B}$ is a valid adversary that behaves like required in theorem 5. With probability at least $(1 - \frac{\ell \cdot Q_{\text{Hsh}} \cdot (\nu+1)}{p})(1 - \frac{\ell \cdot Q_{\text{Hsh}} \cdot 2}{p}) \geq 1 - \frac{(\nu+3) \cdot \ell \cdot Q_{\text{Hsh}} + 1}{p}$, the extractor outputs witnesses of the form γ_i , and either a commitment R with two openings $\mathbf{Z}, \mathbf{Z}'$, or partial openings of the commitments $\zeta_{1,i}$ of the form $\text{rnd}_i, r_{i,0}, r_{i,j} \forall j$ s.t. $L_{i,j} = \perp$.

If there is one proof that yielded a commitment R and two openings $\mathbf{Z}, \mathbf{Z}'$, the reduction outputs these as its solution to the binding game. Otherwise, the reduction proceeds as follows.

As part of the output of the adversary, the reduction obtains the (full) opening vectors $\mathbf{L}'_i$ for each commitment C_i that was used during registration.

The reduction now picks a session-signature-pair $(\text{rnd}, C), (m, \hat{\sigma}, \gamma)$ with $\text{dlog}_{C \cdot g^{\text{rnd}}} \zeta_1 = \text{dlog}_z \zeta$ and $\mathbf{L} \not\subseteq \mathbf{L}'$ where $\mathbf{L}$ is the vector that $\mathcal{A}$ partially opened in the showing algorithm, and $\mathbf{L}'$ is the vector of the signing session that is matched through γ .

It unites the partial vector $\mathbf{L}$ with the corresponding partial opening $\text{rnd}_i, r_{i,0}, r_{i,j} \forall j$ s.t. $L_{i,j} = \perp$ obtained from the extractor into an opening $\mathbf{L}''_i$.

The reduction now outputs the commitment C along with the two openings $\mathbf{L}'_i$ and $\mathbf{L}''_i$ to the challenger.

Thus, the reduction wins the binding game for the Pedersen commitment scheme with probability of at least

$$\Pr[\mathbf{G}_2^{\mathcal{A}} = 1] \cdot (1 - \frac{(\nu+3) \cdot \ell \cdot Q_{\text{Hsh}} + 1}{p}) \geq \Pr[\mathbf{G}_2^{\mathcal{A}} = 1] - \frac{(\nu+3) \cdot \ell \cdot Q_{\text{Hsh}} + 1}{p}.$$

Putting this together with lemma 3 yields the statement.

Putting everything together we get that

$$\varepsilon' \leq (\nu + 21) \cdot \varepsilon + \frac{(\nu + 3) \cdot \ell \cdot Q_H + 52 + 4 \cdot Q_C}{p} + \frac{1}{\nu}.$$

G.4 ACL Blindness

We show that ACL fulfills our stronger definition of blindness that takes into account the showing algorithm.

Theorem 7. *If the DDH problem is (t, ε) -hard in $\mathbb{G}$, then ACL is (t', ε') -blind in the ROM with $t' \approx t$ and $\varepsilon' \leq 2\varepsilon + \frac{2Q_{H_{\text{reg}}} + 2Q_{H_{\text{sh}}} + 2Q_{H_3} + 3}{p}$ where $Q_{H_{\text{reg}}}$, $Q_{H_{\text{sh}}}$, and Q_{H_3} are the adversary's numbers of queries to H_{reg} , H_{sh} , and H_3 , respectively.*

Proof. The proof mostly follows the same strategy as [4], but we make the registration and showing procedures more explicit.

We proceed through a series of games where we switch to simulating the proofs provided at registration and during showing, such that finally we can provide two reductions that embed DDH challenges in the signatures.

Game $\mathbf{G}_0$. This is the original blindness game.

Game $\mathbf{G}_1$. This game aborts whenever the challenger makes a query to H_{reg} , H_{sh} , H_3 that has been made before. Each of the inputs to these oracles contain at least one component that is randomized over the entire group, in the case of H_{reg} and H_{sh} this is the first flow of the sigma protocol proof, and in the case of H_3 this is (among others) the group element α . Thus, the probability for colliding with a previously made query is $\frac{2Q_{H_{\text{reg}}} + 1 + 2Q_{H_{\text{sh}}} + 1 + 2Q_{H_3} + 1}{p}$ and the difference to the previous game is

$$\text{Adv}_{\mathbf{G}_1}^{\text{ACL}}(\mathcal{A}) - \text{Adv}_{\mathbf{G}_0}^{\text{ACL}}(\mathcal{A}) \leq \frac{2Q_{H_{\text{reg}}} + 2Q_{H_{\text{sh}}} + 2Q_{H_3} + 3}{p}.$$

Game $\mathbf{G}_2$. This game switches to simulating the proofs generated during showing. That is, for each session it picks values c , s_{sdl} , s_{Γ} , $\{s_i\}_{i \in [n]_0 \setminus I}$ (where I is I_0 or I_1 respectively) and sets $h_{\text{sdl},k} = h_k^{s_{\text{sdl}}} \cdot \psi_k^c$, and $R := \zeta_1^{c_{\Gamma}} \cdot \Gamma^{s_{\Gamma}} \cdot \prod_{i \in [n]_0 \setminus I} \psi_i^{s_i}$. All other elements it computes the same as the honest protocol. It sets $H_{\text{sh}}(\text{pk}, \sigma, \sigma, \zeta_1, m, \Psi, \gamma, g_{\text{sdl}}, z_{\text{sdl}}, h_{\text{sdl}}, R, \mathbf{L}')$ to output c . This last step works as the input to H_{sh} is identically distributed to $\mathbf{G}_1$ and $\mathbf{G}_1$ aborts in the case that H_{sh} has been queried on this value before. Thus, it holds that

$$\text{Adv}_{\mathbf{G}_2}^{\text{ACL}}(\mathcal{A}) = \text{Adv}_{\mathbf{G}_1}^{\text{ACL}}(\mathcal{A}).$$

Game $\mathbf{G}_3$. In this game, we switch to simulating the proof π generated at registration. The game samples $c, s_0, \dots, s_\nu$ and sets $R := C^c \cdot \prod_{i=0}^\nu h_i^{s_i}$ in each of the two sessions. It sets $c =: \mathbf{H}_{\text{reg}}(\mathbf{h}, C, R)$ and outputs π . This game is identically distributed to $\mathbf{G}_2$ by the same argument as the previous game hop and thus

$$\text{Adv}_{\mathbf{G}_3}^{\text{ACL}}(\mathcal{A}) = \text{Adv}_{\mathbf{G}_2}^{\text{ACL}}(\mathcal{A}).$$

Game $\mathbf{G}_4$. In this game, we switch to simulating the signatures. For each signature, the game samples values $\gamma, \omega, \rho, \delta, \sigma_1, \sigma_2, \mu \leftarrow \mathbb{Z}_p$ and sets $\zeta = z^\gamma, \zeta_1 = (C \cdot g^{\text{rnd}})^\gamma$ and $\mathbf{H}_3(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \zeta_1^\delta, h^{\sigma_2} (\zeta / \zeta_1)^\delta, z^\mu \zeta^\delta) := \omega + \delta$. As a challenge during signing, it sends a random value $e \leftarrow \mathbb{Z}_p$ and it checks after the signer's response (c, d, r, s_1, s_2) that $a = g^r y^c, b_1 = g^{s_1} (C \cdot g^{\text{rnd}})^d$, and $b_2 = h^{s_2} (z / (C \cdot g^{\text{rnd}}))^d$. If any of these checks fail it sets the signature to $\perp$. This game is identically distributed to $\mathbf{G}_3$ by the same argument as the previous game hops and thus

$$\text{Adv}_{\mathbf{G}_4}^{\text{ACL}}(\mathcal{A}) = \text{Adv}_{\mathbf{G}_3}^{\text{ACL}}(\mathcal{A}).$$

Game $\mathbf{G}_5$. In this game, we switch the ζ, ζ_1 pair of the signature on message m_b to random group elements. This game hop is bounded by the hardness of DDH in $\mathbb{G}$. We provide a reduction $\mathcal{R}_1$:

- **Setup.** The reduction takes as input a DDH challenge g, X, Y, Z . It programs the random oracle $\mathbf{H}_1$ to output $X \cdot g^u$ for randomly chosen u at each fresh query. It furthermore programs the random oracle $\mathbf{H}_{\text{pp}}(i)$ to output g^{v_i} for $i = 0, \dots, \nu$ with randomly chosen v_i . It then calls the adversary to obtain $\text{pk}, m_0, m_1, \mathbf{L}_0, \mathbf{L}_1$. The reduction samples a bit $b \leftarrow \{0, 1\}$ and calls the adversary again to run. Denote by u the value u such that $z = X^u$ in the public key.
- **Online Phase** The reduction simulates the oracles as follows.
 - $\mathcal{RG}_{\mathcal{U}}(\mathbf{L}_0)$ sets the commitment to $X \cdot g^w$ for randomly chosen $w \leftarrow \mathbb{Z}_p$. It proceeds as $\mathbf{G}_4$.
 - $\mathcal{RG}_{\mathcal{U}}(\mathbf{L}_1)$ sets the commitment honestly to $\mathbf{h}^{\mathbf{L}_1}$
 - $\mathbf{U}^0$ sets $\zeta = Z \cdot Y^u$ and $\zeta_1 = Z \cdot Y^{w+\text{rnd}_0}$. It proceeds with signing as in $\mathbf{G}_4$.
 - $\mathbf{U}^1$ proceeds with signing as in $\mathbf{G}_4$.
- **Showing Phase.** If the two signing sessions result in valid signatures, the game produces the proof for $(m_b, \sigma_b, \zeta_{1,b})$ as follows: It sets $\Gamma = Y, \psi_k = Y^{v_k}$ for $k \in \llbracket n \rrbracket$ and then proceeds as in $\mathbf{G}_4$. For the other signature it proceeds as in $\mathbf{G}_4$.
- **Output Determination.** The reduction outputs 0 if $b = b'$ and 1 otherwise.

It is easy to see that the reduction perfectly simulates $\mathbf{G}_4$ if the bit of the DDH game is 0 (REAL) and $\mathbf{G}_4$ if the bit of the DDH game is 1 (RANDOM). Thus it holds that

$$\text{Adv}_{\mathbf{G}_5}^{\text{ACL}}(\mathcal{A}) - \text{Adv}_{\mathbf{G}_4}^{\text{ACL}}(\mathcal{A}) \leq \text{Adv}_{\text{DDH}}^{\mathbb{G}}(\mathcal{R}_1)$$

Game $\mathbf{G}_6$. In this game, we switch the ζ, ζ_1 pair of the second signature to a random value.

- **Setup.** The reduction takes as input a DDH challenge g, X, Y, Z . It programs the random oracle H_1 to output $X \cdot g^u$ for randomly chosen u at each fresh query. It furthermore programs the random oracle $H_{pp}(i)$ to output g^{v_i} for $i = 0, \dots, \nu$ with randomly chosen v_i . It then calls the adversary to obtain pk, m_0, m_1, L_0, L_1 . The reduction samples a bit $b \leftarrow \{0, 1\}$ and calls the adversary again to run. Denote by u the value u such that $z = X^u$ in the public key.
- **Online Phase** The reduction simulates the oracles as follows.
 - $\mathcal{RG}_U(L_0)$ sets the commitment to a randomly chosen value $C \leftarrow \mathbb{G}$ and proceeds as in $\mathbf{G}_5$.
 - $\mathcal{RG}_U(L_1)$ sets the commitment to $X \cdot g^w$ for randomly chosen $w \leftarrow \mathbb{Z}_p$. It proceeds as $\mathbf{G}_5$.
 - U^0 proceeds with signing as in $\mathbf{G}_5$, i.e. it sets ζ and ζ_1 to be uniformly random group elements and simulates the signature.
 - U^1 sets $\zeta = Z \cdot Y^u$ and $\zeta_1 = Z \cdot Y^{w+\text{rnd}_0}$. It proceeds with signing as in $\mathbf{G}_5$.
- **Showing Phase.** If the two signing sessions result in valid signatures, the game produces the proof for $(m_{1-b}, \sigma_{1-b}, \zeta_{1,1-b})$ as follows: It sets $\Gamma = Y$, $\psi_k = Y^{v_k}$ for $k \in \llbracket n \rrbracket$ and then proceeds as in $\mathbf{G}_5$. For the other signature it proceeds as in $\mathbf{G}_5$, i.e. it sets $\psi_i = h_i^\gamma$ for a randomly chosen γ .
- **Output Determination.** The reduction outputs 0 if $b = b'$ and 1 otherwise.

It is easy to see that the reduction perfectly simulates $\mathbf{G}_5$ if the bit of the DDH game is 0 (REAL) and $\mathbf{G}_5$ if the bit of the DDH game is 1 (RANDOM). Thus it holds that

$$\mathbf{Adv}_{\mathbf{G}_6}^{\text{ACL}}(\mathcal{A}) - \mathbf{Adv}_{\mathbf{G}_5}^{\text{ACL}}(\mathcal{A}) \leq \mathbf{Adv}_{\text{DDH}}^{\mathbb{G}}(\mathcal{R}_1)$$

In $\mathbf{G}_6$, it holds that $\mathbf{Adv}_{\mathbf{G}_6}^{\text{ACL}}(\mathcal{A}) = 0$ as the signatures are completely unrelated to the signing sessions. Summing up the probabilities yields the claim

G.5 ACL Double-Spending Detection

We now explain how the ACL protocol can also support Double-Spending detection. We require the following modifications:

- In the main protocol, as described in Fig. 6, we add the following *additional* steps to the User side of the protocol: (1) Pick $\tau_2 \leftarrow_{\$} \mathbb{Z}_p$, (2) compute $\eta_2 := z^{\tau_2}$, (3) include η_2 in the hash computation of ε . The signature verification algorithm can either be modified to include the element η_2 in the hash computation, or one can treat η_2 as if it was part of the message when verifying signatures which enables using the verification algorithm of the plain scheme.

- In the showing protocol, as described in Fig. 4, we add the following steps: (1) The user includes V_{name} and timestamp as part of the hash computation in c . (2) The user computes $\mu_2 = \tau_2 - c\gamma$ and outputs μ_2 along with (π_{sdl}, π_{op}) . (3) The verifier runs an updated signature verification algorithm which also includes $z^{\mu_2} \cdot \zeta^c$ as part of H_3 (which verifies with η_2 part of ε in the signature).
- $\mathcal{DSI}$: If a user double spends the same signature σ using showings $\bar{\pi}_1, \bar{\pi}_2$, it results into two different values c, c' and μ_2, μ'_2 from which one can extract $\gamma = \frac{\mu'_2 - \mu_2}{c - c'}$ and thus ζ_1 which uniquely identifies the issuing session. This is then used by the signer to identify the signing session. It outputs the proof of double spending as the user's commitment C , the user's proof of partial opening to ID , the signer's randomization factor rnd , and the user's blinding factor γ , along with the signature σ and the two showings $\bar{\pi}_1, \bar{\pi}_2$.
- $\mathcal{DSV}$: On input of an ID and a double spending proof, the $\mathcal{DSV}$ algorithm outputs 1 iff both showings of the signature are valid, the proof of partial opening of C to ID is valid, and it holds that $(C \cdot g^{\text{rnd}})^\gamma = \zeta_1$.

Discussion. We note that double-spending detection for ACL was already discussed in [34] (a work that implements [4] as an e-cash scheme), but it was done in a slightly different way and without a formal security proof. Namely, in [34] the value μ is omitted from the signature⁸ instead of adding the additional τ_2, η_2 as we did above. Then, the showing protocol uses τ to compute μ_2 . It is critical that μ is removed from the signature, since otherwise a single credential showing already links it to a signing session because the signer can use $\frac{\mu_2 - \mu}{\delta - c}$ to compute γ and then check which $z_1^\gamma = \zeta_1$.

While this double-spending protocol for ACL is correct, it does not compose well with the existing security proofs for ACL. Namely, removing μ from the ACL signature removes the guarantees provided by existing proofs of one-more unforgeability [4,37] as the signatures can no longer be verified on their own. Thus, we went for a modular approach, that does not affect the ACL proof, by computing additional elements (τ_2, η_2) of the same form as (τ, η) and adding η_2 to the hash during the signature generation in order to use this for the double spending detection. As noted above, this modification does not affect the ACL verification algorithm and thus the ACL security proofs still hold.

Lemma 18. *With the modification described above, for any adversary $\mathcal{A}$ making at most $Q_{H_{sh}}$ queries to the random oracle H_{sh} , it holds that $\text{Adv}_{\text{BSADS}}^{\text{DSP}}(\mathcal{A}) \leq \frac{Q_{H_{sh}}^2}{p}$.*

Proof. Intuitively, the only way for the adversary to avoid being identified when double spending is to provoke a hash collision such that $c \neq c'$. We show this formally below using some game hops

Game $\mathbf{G}_0$. This is the original DSP-game.

⁸ This approach was also used by Abe for its blind signature scheme [1], on which ACL was based.

Game $\mathbf{G}_1$. This game is identical to $\mathbf{G}_0$ except that it aborts if there is a hash collision in the random oracle H , i.e. two different queries by the adversary that lead to different outcomes.

It is easy to see that $|\Pr[\mathbf{G}_1^{\mathcal{A}} = 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} = 1]| \leq \frac{Q_H^2}{p}$.

We now show that the adversary $\mathcal{A}$ has success probability 0 in $\mathbf{G}_1$.

Since there is no hash collision, the two hashes c and c' are different. of the proof.

Lemma 19. *If the discrete logarithm problem is (t, ε) -hard in the group $\mathbb{G}$, then ACLI is (t', ε') -Exc secure in the $\text{AGM} + \text{ROM}$ with $t' \approx t$ and*

$$\varepsilon' = \varepsilon + \frac{2Q_{H_{\text{sh}}} + \kappa + Q_{H_{\text{reg}}} + \ell}{p},$$

where $Q_{H_{\text{sh}}}$ is the number of queries the adversary makes to H_{sh} , $Q_{H_{\text{reg}}}$ is the number of queries the adversary makes to H_{reg} , ℓ is the number of signing sessions closed with the user oracles, κ is the number of registered users through the registration oracle.

Proof Idea. In order to successfully frame a user, the adversary needs to output a signature along with two showing proofs π_1, π_2 as the $\mathcal{DSI}$ algorithm and provide values γ and rnd that blinds the user's commitment into ζ_1 . As the signer can interact with itself to issue a signature, thus knowing γ and rnd , we show that it is hard for the signer to provide the partial proofs of opening of the commitment by extracting a discrete logarithm solution from a signer that does so.

Proof. We do a quick game hop to exclude some situations where the reduction cannot win related to RO queries.

Game $\mathbf{G}_0$. This is the original Exc-game.

Game $\mathbf{G}_1$. This game aborts whenever the challenger queries the hash oracles H_{reg} and H_{sh} during computation of a proof π or $\bar{\pi}$, and the hash oracle has been queried on the same value before. As the challenger's queries to its own oracle contain first flows of the proofs $\pi, \bar{\pi}$, the probability for each first flow to have been queried before is at most $\frac{Q_{H_{\text{reg}}} + \kappa}{p}$ for H_{reg} and $\frac{Q_{H_{\text{sh}}} + \ell}{p}$ for H_{sh} .

Game $\mathbf{G}_2$. This game introduces an additional abort condition related to the representations submitted by the adversary to the oracle. When the adversary outputs a proof of double spending $(C, ID, \pi, \text{rnd}, \gamma, m, \sigma, \pi_1, \pi_2)$, at least one of π_1 and π_2 was not generated by the showing oracle. Let $\bar{\pi} = (\pi_{\text{sdl}}, \pi_{\text{op}})$ be the showing proof that was not an output of the showing oracle. We consider the $\pi_{\text{op}} = (R_{\text{op}}, c, \mathbf{S}_{\text{op}})$ component.

We denote by $G_{[R]}$ the exponent of a group element G in the representation of R , where G can be any group element that was output by the challenger to

the adversary through one of the oracles. The game aborts if

$$c = \sum_{j=1}^{\kappa} C_{j[R]} + \pi \cdot c \cdot \pi \cdot R_{j[R]} + \sum_{j=1}^{\ell} \gamma_j \cdot \zeta_{1,j[R]} + \sum_{j=1}^{\ell} \pi_{op} \cdot c_j \cdot R_{j[R]},$$

where $\pi_{op} \cdot c_j$ is the value c computed as part of the showing, γ_j is the γ value chosen in the j_{th} signing session, κ is the number of registered users, and C_j is the commitment of the j_{th} registered user, $\pi \cdot R_j$ is the first flow of the proof of (partial) opening of the registration commitment of the j_{th} registered user, and c is the hash output used in π_{op} . As the representations of γ_j and $\pi \cdot c$ are already fixed by the time the adversary receives the hash output, the probability of the above event happening is at most $\frac{Q_{H_{sh}}}{p}$.

We describe a reduction that breaks the discrete logarithm problem if the adversary wins $\mathbf{G}_2$.

The reduction simulates the $\mathbf{G}_2$ game as follows to the adversary:

Setup. The reduction takes as input a discrete logarithm challenge g, X . It invokes the adversary $\mathcal{A}$ with the oracles described below. Keep a list of all group elements submitted by the adversary to the oracles along with their algebraic representations.

Online Phase. The reduction simulates the oracles as follows to the adversary:

Hash oracles $H_{pp}, H_{reg}, H_1, H_3, H_{sh}$ lazy sampling from the appropriate image spaces, in case of group element image spaces, sample the discrete logarithm to base g of the group element from $\mathbb{Z}_p$ and keep the list of discrete logarithms along with the inputs and group element oracle outputs.

Registration queries Takes as input the vector to be registered L . Sample $r_{com}' \leftarrow_{\$} \mathbb{Z}_p$. Compute the commitment $C := h_0^{r_{com}'} X \cdot \prod_{i=1}^{\nu} h_i^{L_i}$ which implicitly sets $r_{com} := r_{com}' \cdot \text{dlog} X$. Simulate the proof π by programming the RO H_{reg} .

User queries participate in the signing protocol like an honest user would and output the resulting signatures to the signer.

Showing queries On input of a signature previously output by the user oracle and an opening set I_{op} , produce an opening $\bar{\pi}$ to this opening index set by simulating the proof by programming the random oracle H_{sh} .

Output Determination. The reduction retrieves the algebraic representation of R that the adversary submitted when querying the random oracle H_{sh} to generate $\bar{\pi}$. It computes a so-called *reduced representation*⁹ of R to the base $I = (g, X)$ as follows: We write $R = X^{X_{[R]I}} \cdot g^{g_{[R]I}}$, where

$$X_{[R]I} := \sum_{j=1}^{\kappa} C_{j[R]} + \pi \cdot c \cdot \pi \cdot R_{j[R]} + \sum_{j=1}^{\ell} \gamma_j \cdot \zeta_{1,j[R]} + \sum_{j=1}^{\ell} \pi_{op} \cdot c_j \cdot R_{j[R]}$$

⁹ This strategy has been used before in [39,37].

and $g_{[X]_I} := \text{dlog}_g(R \cdot X^{-X_{[R]_I}})$. The reduction can compute $g_{[X]_I}$ using the discrete logarithms of all group elements in its output to the base g , as any X components have been removed from this equation.

If $c = X_{[R]_I}$, the reduction aborts as this corresponds to the abort condition introduced in $\mathbf{G}_2$. Otherwise, it computes

$$\text{dlog}_g X = \frac{g_{[R]_I} - (\sum_{i=0}^{\nu} s_i + s_\Gamma + c \cdot (r_{com}' + \sum_{i=1}^{\nu} L_i + \text{rnd}))}{c - X_{[R]_I}}.$$

The reduction outputs a valid DLP solution whenever the adversary wins $\mathbf{G}_2$. Thus, putting together the probabilities from the game hops yields

$$\text{Adv}_{\mathbb{G}}^{\text{DLP}}(\mathcal{R}) = \text{Adv}_{\text{ACL}}^{\text{Exc}}(\mathcal{A}) - \frac{Q_{\text{H}_{\text{sh}}}}{p} - \frac{Q_{\text{H}_{\text{reg}}} + \kappa}{p} - \frac{Q_{\text{H}_{\text{sh}}} + \ell}{p}$$

which yields the claim.

H Proofs of Lemmas 15 and 16

Next, we propose a simplified variant of the restrictive blinding game from [1,39] where we do not require that the adversary's output contains as many signatures as it requested signing sessions, but rather only a “mismatched” signature.

Definition 27 (Restrictive Blinding Game). *We describe the restrictive blinding game for an adversary $\mathcal{A}$:*

Setup: *The challenger samples a key pair (pk, sk) according to $\mathcal{G}(\text{pp}_{\mathbb{G}})$. This uses the random oracle H_1 which allows the challenger to keep track of the discrete logarithms of its outputs. It outputs pk to the adversary.*

Online Phase: *The adversary gets access to the following oracles*

Oracle H_1 : *lazily samples $w \leftarrow_{\$} \mathbb{Z}_p$ and returns g^w . Keeps a list of the hash inputs, w , and the output g^w .*

Oracle H_{pp} : *lazily samples $v \leftarrow_{\$} \mathbb{Z}_p$ and returns g^v . Keeps a list of the hash inputs v , and the output g^v .*

Oracle H_3 : *lazily samples $\varepsilon \leftarrow_{\$} \mathbb{Z}_p$ and returns ε*

RG_5 : *Register new commitments and store them in list to ensure all commitments coming in $\mathcal{S}_1$ queries were already registered.*

$\mathcal{S}_1$: *runs the signing procedure $\mathcal{S}_1$ of ACL and returns its output along with a fresh session ID. Keeps a list of the commitment C and the random value rnd .*

$\mathcal{S}_2$: *takes as input a session ID and a challenge e . If the session ID has never been used before in a $\mathcal{S}_2$ query, it runs the $\mathcal{S}_2$ procedure of ACL using the internal signer state corresponding to the session ID with e and returns its output.*

Output Determination: *$\mathcal{A}$ submits a list of message-signature pairs (m_i, σ_i) for $i = 1..l$. The game outputs 1 if there is a pair $(m, \sigma = (\zeta, \zeta_1, \dots))$ in the list such that the signature is valid on the message m under the public key pk and it holds that there exists no closed signing session sid such that $\text{dlog}_z \zeta = \text{dlog}_{z_1, \text{sid}} \zeta_1$.*

We define the advantage of the adversary in the game RB as

$$\mathbf{Adv}_{\text{ACL}}^{\text{RB}}(\mathcal{A}) := \Pr[\text{RB}_{\text{ACL}}^{\mathcal{A}} = 1].$$

Lemma 15 (Restrictive Blinding). *In the ROM and AGM, if DLP is (t, ε) -hard in $\mathbb{G}$, it holds that for any adversary running in time at most t , the advantage is at most*

$$\mathbf{Adv}_{\text{ACL}}^{\text{RB}}(\mathcal{A}) \leq 13 \cdot \varepsilon + \frac{4 \cdot Q_C + 31}{p} + \frac{1}{\nu}.$$

where Q_C is the number of registration queries made by the adversary.

Proof. We use a similar proof technique to the one used in Theorem 4.1. in [37] (and full version [38]). We proceed in a series of games to gradually rule out bad event, reaching a game that we can reduce DLP if $\mathcal{A}$ outputs a valid signature such that there exists no closed signing session sid satisfying $\text{dlog}_z \zeta = \text{dlog}_{z_1, \text{sid}} \zeta_1$.

We start by ruling out the first abort condition corresponding to the representation of the registered commitments C .

Game $\mathbf{G}_0$. This is Game RB.

Let Q_C be the number of commitments $\mathcal{A}$ registers. We define $\mathbf{BAD}_{15}$ as the event in which $\mathcal{A}$ registers a commitment C_j for $j \in [Q_C]$ successfully using a proof $\pi = (M, s_0, \dots, s_\nu)$ while $\mathbf{T}_{\text{pp}}[(C, M)] = \perp$, where $\mathbf{T}_{\text{p}}$ is the table of the random oracle $\mathbf{H}_{\text{reg}}$ used in the Fiat-Shamir transformation of the protocol from construction 3. Intuitively, this corresponds to $\mathcal{A}$ registering a commitment C_j successfully without having made a hash query beforehand.

Game $\mathbf{G}_1$. This game is identical to Game $\mathbf{G}_0$, except that it aborts if Event $\mathbf{BAD}_{15}$ occurs.

Claim 27.

$$\Pr[\mathbf{BAD}_{15}] \leq \frac{Q_C}{p}.$$

Proof. Outputting a valid proof $\pi = (M, s_0, \dots, s_\nu)$ for a commitment C implies that the equation $C^c \cdot \prod_{i \in [\nu]} h_i^{s_i} = M$, where $c = \mathbf{H}_{\text{reg}}(C, M)$. Thus, outputting a valid proof without querying $\mathbf{H}_{\text{reg}}$ beforehand to acquire c is equivalent to guessing c . However, since $\mathbf{H}_{\text{reg}}$ is modeled as a random oracle, the value c is chosen uniformly at random, and the probability of $\mathcal{A}$ guessing this value correctly is at most $\frac{1}{p}$. Since $\mathcal{A}$ registers up to Q_C values, the probability that it successfully registers some commitment without having made a corresponding hash query is at most $\frac{Q_C}{p}$.

By this claim, it holds $\mathbf{Adv}_{\mathbf{G}_1}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_0}^{\mathcal{A}} - \frac{Q_C}{p}$.

As $\mathbf{BAD}_{15}$ does not occur, we assume in the following that for each valid commitment-proof tuple $(C, \pi = (M, s_0, \dots, s_\nu))$ the adversary outputs, it holds that $\mathbf{T}_{\text{p}} \neq \perp$.

Let $\mathbf{BAD}_{16}$ be the event in which $\mathcal{A}$ successfully registers a commitment C_j for $j \in [Q_C]$ satisfying the condition

$$\bigvee_{j \in [Q_C]} y_{[C_j]} + \sum_{\text{sid} \in Q_S} c_{\text{sid}} \cdot a_{\text{sid}[C_j]} \neq 0,$$

where Q_S is the number of open signing sessions at the time $\mathcal{A}$ registers C_j , and c_{sid} is the random chosen by the signer to compute a_{sid} . Intuitively, this condition means that $\mathcal{A}$ uses y or some a_{sid} (or both) in the representation of some commitment C_j such that they do not cancel each other out.

Game $\mathbf{G}_2$. This game is identical to $\mathbf{G}_1$, except that it aborts if event $\mathbf{BAD}_{16}$ occurs.

Claim 28.

$$\Pr[\mathbf{BAD}_{16}] \leq \text{Adv}_{\text{DLP}}^{\mathcal{R}_1} + \frac{1}{p}.$$

Proof. It follows directly from Claim E.1 in [37].

By the claim, we have $\text{Adv}_{\mathbf{G}_2}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_1}^{\mathcal{A}} - \text{Adv}_{\text{DLP}}^{\mathcal{R}_1} - \frac{1}{p}$.

By this claim, we assume that $\mathcal{A}$ wins $\mathbf{G}_2$ and $\mathbf{BAD}_{16}$ does not occur. However, $\mathcal{A}$ may still use a_{sid} or y components in the representation of C without triggering $\mathbf{BAD}_{16}$ in a way that prevents $\mathcal{R}$ from functioning properly. Specifically, if $\mathcal{A}$ uses the group element a_{sid} or $b_{2,\text{sid}}$ of the signing session sid in the representation of some commitment C , but it never closes sid , this may lead to an issue for $\mathcal{R}$, because it won't be able to compute $\text{dlog}_g C$ without knowing the opening of a_{sid} or b_{sid} , i.e. $(r_{\text{sid}}, c_{\text{sid}})$ and $(s_{2,\text{sid}}, d_{\text{sid}})$, respectively. Note that this does not cause an issue for the current game hop, because it is straight forward to know the opening of all group elements of any signing session sid since the signer oracles are simulated by the reduction. However, this is problematic later in the reduction from Game $\mathbf{OMMIM}$ to Game $\mathbf{OMUF}$, because $\mathcal{R}$ uses the challenger's prover oracles to simulate the signer oracles for $\mathcal{A}$. Consequently, the reduction will have access to the opening of a group element a_{sid} or b_{sid} iff the adversary makes $\mathbf{S2}$ query for sessions sid . Therefore, it is crucial to ensure that no a_{sid} or $b_{2,\text{sid}}$ is used in the representation of any commitment C if sid will not be closed. To capture this, we define $\mathbf{BAD}_{17}$ as the event that occurs if the equation

$$\bigvee_{j \in [Q_C]} (y_{[C_j]} + \sum_{\text{sid} \in \mathbf{I}_{op}} c_{\text{sid}} \cdot a_{\text{sid}[C_j]} = - \sum_{\text{sid} \in \mathbf{I}_{cl}} c_{\text{sid}} \cdot a_{\text{sid}[C_j]}).$$

Game $\mathbf{G}_3$. This game is identical to $\mathbf{G}_2$, except that it aborts if event $\mathbf{BAD}_{17}$ occurs.

Claim 29.

$$\Pr[\mathbf{BAD}_{17}] \leq \frac{Q_C}{p}.$$

Proof. Identical to Claim E.2 in [37].

Consequently, we have $\mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_2}^{\mathcal{A}} - \frac{Q_C}{p}$.

Next, we rule out the usage of any h or $b_{2,\text{sid}}$ component in the representation of C_j for $\text{sid} \in Q_S$ and $j \in Q_C$. We define Event $\mathbf{BAD}_{18}$ as the event that occurs if

$$\bigvee_{j \in [Q_C]} (h_{[C_j]} + \sum_{\text{sid} \in Q_S} s_{2,\text{sid}} \cdot b_{2,\text{sid}[C_j]}) \neq 0$$

holds, where C_j are the commitments registered by $\mathcal{A}$. This condition means that $\mathcal{A}$ uses h or some $b_{2,\text{sid}}$ (or both) in the representation of some commitment C_j such that they do not cancel each other out.

Game $\mathbf{G}_4$. This game is identical to $\mathbf{G}_3$, except that it aborts if event $\mathbf{BAD}_{18}$ occurs.

Claim 30.

$$\Pr[\mathbf{BAD}_{18}] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_2} + \frac{1}{p}.$$

Proof. Identical to Claim E.3 in [37].

Per this claim, we have $\mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_3}^{\mathcal{A}} - \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_2} - \frac{1}{p}$.

Next, we prevent $\mathcal{A}$ from using $b_{2,\text{sid}}$ from an open signing session sid in the representation of some C_j for $j \in [Q_C]$ without triggering $\mathbf{BAD}_{18}$.

Define $\mathbf{BAD}_{19}$ as the event that occurs if

$$\bigvee_{j \in [Q_C]} (h_{[C_j]} + \sum_{\text{sid} \in \mathcal{I}_{cl}} s_{2,\text{sid}} \cdot b_{2,\text{sid}[C_j]} = - \sum_{\text{sid} \in \mathcal{I}_{op}} s_{2,\text{sid}} \cdot b_{2,\text{sid}[C_j]}).$$

This condition means that $\mathcal{A}$ uses h or some $b_{2,\text{sid}}$ (or both) in the representation of some commitment C_j such that they do not cancel each other out.

Game $\mathbf{G}_5$. This game is identical to $\mathbf{G}_4$, except that it aborts if event $\mathbf{BAD}_{19}$ occurs.

Claim 31.

$$\Pr[\mathbf{BAD}_{19}] \leq \frac{Q_C}{p}.$$

Proof. The same as Claim E.4 in [37].

By the claim, it holds $\mathbf{Adv}_{\mathbf{G}_5}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_4}^{\mathcal{A}} - \frac{Q_C}{p}$.

Let Event $\mathbf{BAD}_{20}$ be the event that occurs if

$$\bigvee_{j \in [Q_C]} (z_{[C_j]} + \sum_{\text{sid} \in Q_S} d_{\text{sid}} \cdot b_{2,\text{sid}[C_j]} \neq 0).$$

holds. This condition is equivalent to $\mathcal{A}$ using z or some $b_{2,\text{sid}}$ (or both) in the representation of some commitment C_j without canceling each other out.

Game $\mathbf{G}_6$. This game is identical to $\mathbf{G}_5$, except that it aborts if event $\mathbf{BAD}_{20}$ occurs.

Claim 32.

$$\Pr[\mathbf{BAD}_{20}] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_3} + \frac{1}{p}.$$

Proof. The same as Claim E.5 in [37].

It follows that $\mathbf{Adv}_{\mathbf{G}_6}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_5}^{\mathcal{A}} - \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_3} - \frac{1}{p}$.

Similar to what we did in Games $\mathbf{G}_3$ and $\mathbf{G}_5$, we rule out using $b_{2,\text{sid}}$ from a signing session sid that is never closed in the representation of some commitment C_j without triggering $\mathbf{BAD}_{20}$.

Let Event $\mathbf{BAD}_{21}$ be the event that occurs if

$$\bigvee_{j \in [Q_C]} z_{[C_j]} + \sum_{\text{sid} \in \mathcal{I}_{op}} d_{\text{sid}} \cdot b_{2,\text{sid}[C_j]} = - \sum_{\text{sid} \in \mathcal{I}_{cl}} d_{\text{sid}} \cdot b_{2,\text{sid}[C_j]}.$$

holds.

Game $\mathbf{G}_7$. This game is identical to $\mathbf{G}_6$, except that it aborts if event $\mathbf{BAD}_{21}$ occurs.

Claim 33.

$$\Pr[\mathbf{BAD}_{21}] \leq \frac{Q_C}{p}.$$

Proof. Identical to Claim E.6 in [37].

It follows that $\mathbf{Adv}_{\mathbf{G}_7}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_6}^{\mathcal{A}} - \frac{Q_C}{p}$.

Let open_j be a vector of all signing sessions identifiers that are still open at the time $\mathcal{A}$ registers a commitment C_j for $j \in [Q_C]$, ordered by the time the sessions were opened. Let Event $\mathbf{BAD}_{22}$ be the event that occurs if

$$\bigvee_{j \in [Q_C]} \sum_{\text{sid} \in \text{open}_j} u_{3,\text{sid}} \cdot b_{1,\text{sid}[C_j]} \neq 0,$$

holds, where $u_{3,\text{sid}} := \text{dlog}_g b_{1,\text{sid}}$.

Claim 34.

$$\Pr[\mathbf{BAD}_{22}] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_4} + \frac{3}{p} + \frac{1}{\nu}.$$

Proof. Identical to Claim E.7 in [37].

By this claim, it holds that $\mathbf{Adv}_{\mathbf{G}_7}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_6}^{\mathcal{A}} - \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_4} - \frac{3}{p} - \frac{1}{\nu}$.

Corollary 3. *Given a representation of a successfully registered commitment C , $\mathcal{R}$ can efficiently compute $\text{dlog}_g C$.*

Proof. If $\mathcal{A}$ wins $\mathbf{G}_7$, non of the abort conditions of the previous games (i.e., $\mathbf{BAD}_1 - \mathbf{BAD}_{22}$) was triggered, except with negligible probability. In this case, the representations of all registered commitments C do not contain any group elements with unknown discrete logarithms to $\mathcal{R}$, such as the group elements of the challenger's public key or the group elements $\mathcal{A}$ receiver via $\mathcal{S}_1$ queries. Thus, $\mathcal{R}$ can efficiently compute $\text{dlog}_g C$.

Next, we address the second abort condition, which relates to the representation of the group element ζ .

Let $\mathbf{BAD}_{23}$ be the event in which $\mathcal{A}$ outputs a valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ for a message m , while $\text{T}_3[(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta, m)] = \perp$. This corresponds to $\mathcal{A}$ outputting a valid signature without making a corresponding hash query to H_3 beforehand.

Game $\mathbf{G}_8$. This game is the same as $\mathbf{G}_7$ except that it aborts if Event $\mathbf{BAD}_{23}$ occurs.

Claim 35.

$$\Pr[\mathbf{BAD}_{23}] \leq \text{Adv}_{\mathbf{G}_7}^{\mathcal{A}} + \frac{1}{p}.$$

Proof. A valid signature must satisfy the verification equation $\omega + \delta = \text{H}_3(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta, m)$. However, outputting a valid signature without querying H_3 beforehand is equivalent to guessing the output of H_3 , i.e. $\omega + \delta$, on input $(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta, m)$. As H_3 is modeled as a random oracle, its output is chosen uniformly at random from $\mathbb{Z}_p$ and the probability that $\mathcal{A}$ succeeds in guessing its output is at most $\frac{1}{p}$.

By the claim, it holds $\text{Adv}_{\mathbf{G}_8}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_7}^{\mathcal{A}} - \frac{1}{p}$.

Define Event $\mathbf{BAD}_{24}$ as the event that occurs if $\mathcal{A}$ outputs a valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ for a message m with a corresponding H_3 hash query $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$, and it holds $z^{z[\zeta]} \neq \zeta$. That is, the event occurs if the representation of ζ contains any group element other than z .

Game $\mathbf{G}_9$. This game is identical to $\mathbf{G}_8$, except that it aborts if $\mathbf{BAD}_{24}$ occurs.

Claim 36.

$$\Pr[\mathbf{BAD}_{24}] \leq \text{Adv}_{\text{DLP}}^{\mathcal{R}_5} + 2 \cdot \text{Adv}_{\text{DLP}}^{\mathcal{R}_6} + \frac{6}{p}.$$

Proof. Identical to Claim E.11 in [37].

By this claim, we have $\text{Adv}_{\mathbf{G}_9}^{\mathcal{A}} \geq \text{Adv}_{\mathbf{G}_8}^{\mathcal{A}} - \text{Adv}_{\text{DLP}}^{\mathcal{R}_5} - 2 \cdot \text{Adv}_{\text{DLP}}^{\mathcal{R}_6} - \frac{6}{p}$, and therefore, we assume $\mathcal{A}$ wins $\mathbf{G}_9$ and ζ is computed honestly, i.e. its representation contains only a z component.

Corollary 4. *For any valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ with a corresponding H_3 hash query $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$, $\mathcal{R}$ can use the representation of ζ submitted along the hash query to efficiently compute $\text{dlog}_z \zeta$.*

Proof. If $\mathcal{A}$ wins $\mathbf{G}_9$, the abort conditions $\mathbf{BAD}_9$ and $\mathbf{BAD}_{24}$ cannot be triggered, except with probability $\frac{7}{p} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_5} + 2 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_6}$. In this case, the representation of the group element ζ of every hash query corresponding to some valid signature only consists solely of a z component, i.e. $\zeta = g^{z[\zeta]}$. Consequently, $\mathcal{R}$ can efficiently compute $\text{dlog}_z \zeta$, except with negligible probability.

Let Event $\mathbf{BAD}_{25}$ be the event that occurs if $\mathcal{A}$ outputs a valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ for a message m with a corresponding $\mathbf{H}_3$ hash query $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$ satisfying $\text{dlog}_z z_{1,\text{sid}} \neq \text{dlog}_\zeta \zeta_1$ for all signing session sid .

Game $\mathbf{G}_{10}$. This game is identical to $\mathbf{G}_9$, except that it aborts if $\mathbf{BAD}_{25}$ occurs.

Claim 37.

$$\Pr[\mathbf{BAD}_{25}] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_7} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_8} + 4 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_9} + \frac{18}{p}.$$

Proof. The same as Claim E.11 in [37].

By this claim, we have $\mathbf{Adv}_{\mathbf{G}_{10}}^{\mathcal{A}} \geq \mathbf{Adv}_{\mathbf{G}_9}^{\mathcal{A}} - \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_7} - \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_8} - 4 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_9} - \frac{18}{p}$. Therefore, It must hold that each valid signature output by $\mathcal{A}$ is linked to some signing session sid by satisfying $\text{dlog}_z z_{1,\text{sid}} = \text{dlog}_\zeta \zeta_1$.

Corollary 5. *For any valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ with a corresponding $\mathbf{H}_3$ hash query $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$, let $\gamma := \text{dlog}_z \zeta$. Then there exists a signing session sid such that $\zeta_1^{1/\gamma} = z_{1,\text{sid}}$, hence $\text{dlog}_g \zeta_1^{1/\gamma} = \text{rnd}_{\text{sid}}$.*

Proof. If $\mathcal{A}$ wins $\mathbf{G}_{10}$, the abort condition $\mathbf{BAD}_{25}$ was not triggered, except with negligible probability. It follows that any valid signature satisfies $\text{dlog}_z z_{1,\text{sid}} = \text{dlog}_\zeta \zeta_1$ for some signing session sid .

Define Event $\mathbf{BAD}_{25}'$ as the event that occurs if $\mathcal{A}$ outputs a valid signature $(\zeta, \zeta_1, \rho, \omega, \sigma_1, \sigma_2, \mu, \delta)$ with a corresponding $\mathbf{H}_3$ hash query $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$ such that for all signing sessions sid it holds $\text{dlog}_z \zeta = \text{dlog}_{z_{\text{sid}}} \zeta_1$.

Corollary 6.

$$\Pr[\mathbf{BAD}_{25}'] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_7} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_8} + 4 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_9} + \frac{18}{p}.$$

Proof. We know that $\Pr[\mathbf{BAD}_{25}] \leq \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_7} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_8} + 4 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}_9} + \frac{18}{p}$, thus, it suffices to show that $\mathbf{BAD}_{25} \Leftrightarrow \mathbf{BAD}_{25}'$. Let $\gamma := \text{dlog}_z \zeta$. It follows that $\text{dlog}_z z_{1,\text{sid}} = \text{dlog}_z \gamma \zeta_1 =: v_0$, hence $\zeta_1 = z^{v_0 \cdot \gamma} = z_{1,\text{sid}}^\gamma$. It follows that $\text{dlog}_z z_{1,\text{sid}} = \text{dlog}_\zeta \zeta_1 \Leftrightarrow \text{dlog}_z \zeta = \text{dlog}_{z_{1,\text{sid}}} \zeta_1$.

Due to Corollary 6, we have

$$\begin{aligned} \mathbf{Adv}_{\text{ACL}}^{\text{RB}}(\mathcal{A}) &\leq \frac{4 \cdot Q_C + 31}{p} + \frac{1}{\nu} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}1} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}2} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}3} \\ &+ \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}4} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}5} + 2 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}6} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}7} + \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}8} + 4 \cdot \mathbf{Adv}_{\text{DLP}}^{\mathcal{R}9}, \end{aligned}$$

hence

$$\mathbf{Adv}_{\text{ACL}}^{\text{RB}}(\mathcal{A}) \leq 13 \cdot \varepsilon + \frac{4 \cdot Q_C + 31}{p} + \frac{1}{\nu}.$$

Definition 28 (2-to-1 tagging game). [Game ℓ -2-to-1-Tagging (ℓ -RT)]
For a positive integer $\ell \in \mathbb{Z}^+$, we define the game ℓ -RT for the identification scheme $\text{ID}_{\text{ACL}} := (\mathcal{G}, \mathcal{P}, \mathcal{V})$ and an adversary $\mathcal{A}$ as follows.

Initialization. The challenger samples a key pair $(\text{pk}, \text{sk}) \leftarrow_{\$} \mathcal{G}(\text{pp}_{\mathbb{G}})$. It outputs pk to the adversary $\mathcal{A}$.

Online Phase. The adversary gets to interact with prover oracles $\mathcal{P}_1$ and $\mathcal{P}_2$ and verifier oracles $\mathcal{V}_1, \mathcal{V}_2$.

Output Determination. The game outputs 1 there are two accepting sessions i_1, i_2 with the verifier, such that there exists a $\mathcal{P}_2$ invocation j , it holds that $\zeta_{i_1, i} = z_{1, j} = \zeta_{i_2, i}$. We call these two verifier sessions the special sessions.

We define the advantage of an adversary $\mathcal{A}$ in winning the game for the ACL identification scheme ID_{ACL} as

$$\mathbf{Adv}^{\ell\text{-TTO}}_{\text{ID}_{\text{ACL}}}(\mathcal{A}) := \Pr[\ell\text{-TTO}^{\mathcal{A}}_{\text{ID}_{\text{ACL}}} = 1].$$

Lemma 20 (2-to-1 tagging (adapted from Lemma 3.5 in [37])). There exists an adversary $\mathcal{R}$ against the discrete logarithm problem such that for any algebraic adversary $\mathcal{A}$ it holds that

$$\mathbf{Adv}^{\text{DLP}}(\mathcal{R}) \geq \frac{1}{2} \mathbf{Adv}^{\ell\text{-RT}}_{\text{ID}_{\text{ACL}}}(\mathcal{A}) - \frac{1}{p}.$$

Proof. The proof is identical to the proof of Lemma 3.5 in [37] as their proof only uses the fact that there are two special sessions with the same tag.

Definition 29 (2-to-1 Game). We describe the 2-to-1 game ℓ -TTO for an adversary $\mathcal{A}$:

Setup: The challenger samples a key pair (pk, sk) according to $\mathcal{G}(\text{pp}_{\mathbb{G}})$. This uses the random oracle H_1 which allows the challenger to keep track of the discrete logarithms of its outputs. It outputs pk to the adversary.

Online Phase: The adversary gets access to the following oracles

Oracle H_1 : lazily samples $w \leftarrow_{\$} \mathbb{Z}_p$ and returns g^w . Keeps a list of the hash inputs, w , and the output g^w .

Oracle H_{pp} : lazily samples $v \leftarrow_{\$} \mathbb{Z}_p$ and returns g^v . Keeps a list of the hash inputs v , and the output g^v .

Oracle H_3 lazily samples $\varepsilon \leftarrow_{\$} \mathbb{Z}_p$ and returns ε

$\mathcal{S}_1$: runs the signing procedure $\mathcal{S}_1$ of ACL and returns its output along with a fresh session ID. Keeps a list of the commitment C and the

$\mathcal{S}_2$: takes as input a session ID and a challenge e . If the session ID has never been used before in a $\mathcal{S}_2$ query, it runs the $\mathcal{S}_2$ procedure of ACL using the internal signer state corresponding to the session ID with e and returns its output.

Output Determination: The adversary submits a list of message-signature pairs (m_i, σ_i) for $i = 1..l$. The game outputs 1 if there are two messages m_1, m_2 and two signatures $(\zeta^{(1)}, \zeta_1^{(1)}, \rho^{(1)}, \omega^{(1)}, \sigma_1^{(1)}, \sigma_2^{(1)}, \mu^{(1)}, \delta^{(1)})$ and $(\zeta^{(2)}, \zeta_1^{(2)}, \rho^{(2)}, \omega^{(2)}, \sigma_1^{(2)}, \sigma_2^{(2)}, \mu^{(2)}, \delta^{(2)})$ in the list such that the signatures are valid on the respective messages under the public key $\mathbf{pk}$ and it holds that there exists a single closed signing session such that $\text{dlog}_z \zeta^{(1)} = \text{dlog}_{C \cdot g^{\text{rnd}}} \zeta_1^{(1)}$ and $\text{dlog}_z \zeta^{(2)} = \text{dlog}_{C \cdot g^{\text{rnd}}} \zeta_1^{(2)}$.

We define the advantage of the adversary in the game ℓ -TTO as

$$\text{Adv}^{\ell\text{-TTO}}_{\text{ACL}}(\mathcal{A}) := \Pr[\ell\text{-TTO}_{\text{ACL}}^{\mathcal{A}} = 1].$$

Lemma 16 (2-to-1 blinding). In the ROM and AGM, if DLP is (t, ε) -hard in $\mathbb{G}$, it holds that for any adversary running in time at most t , the advantage is at most

$$\text{Adv}^{\ell\text{-TTO}}_{\text{ACL}}(\mathcal{A}) \leq 7 \cdot \varepsilon + \frac{20}{p}.$$

Proof. Assume there is an algebraic adversary $\mathcal{A}$ that runs and wins experiment ℓ -TTO. It follows that $\mathcal{A}$ provides two valid signatures $(\zeta^{(1)}, \zeta_1^{(1)}, \rho^{(1)}, \omega^{(1)}, \sigma_1^{(1)}, \sigma_2^{(1)}, \mu^{(1)}, \delta^{(1)})$ and $(\zeta^{(2)}, \zeta_1^{(2)}, \rho^{(2)}, \omega^{(2)}, \sigma_1^{(2)}, \sigma_2^{(2)}, \mu^{(2)}, \delta^{(2)})$ satisfying the equalities $\text{dlog}_z \zeta^{(1)} = \text{dlog}_{C \cdot g^{\text{rnd}}} \zeta_1^{(1)}$ and $\text{dlog}_z \zeta^{(2)} = \text{dlog}_{C \cdot g^{\text{rnd}}} \zeta_1^{(2)}$. Using the same game hop from the proof of Lemma 15 ($\mathbf{G}_1$ - $\mathbf{G}_{10}$), we rule out the abort events of these Games. Consequently, Corollaries 3, 4, and 5 hold, and $\mathcal{R}_0$ can efficiently compute $\text{dlog}_g C$ for each registered commitment C , $\text{dlog}_z \zeta$, and $\text{dlog}_{z1, \text{sid}} \zeta_1$ of both hash queries linked to the signatures $\mathcal{A}$ outputs, for some signing sessions sid .

When $\mathcal{R}$ is executed with game ℓ -RT, it takes $\mathbf{pk} = (h, y, z)$ as input and gets access to the oracles $\mathcal{P}_1, \mathcal{P}_2, \mathcal{V}_1$, and $\mathcal{V}_2$. Then, it invokes $\mathcal{A}$ with input $\mathbf{pk}$ and simulates Game ℓ -TTO for $\mathcal{A}$ by providing access to the following oracles:

- $\mathcal{RG}_5(C, \pi)$. If π is a valid proof of knowledge of an opening for C and $C \notin \mathbf{registered}$, store C in **registered** and output 1, 0 otherwise.
- $\mathcal{S1}(C)$. Output $\perp$ if $C \notin \mathbf{registered}$. Query the challenger's oracle $(\text{sid}, a_{\text{sid}}, b_{1, \text{sid}}, b_{2, \text{sid}}, \text{rnd}_{\text{sid}}) \leftarrow_{\$} \mathcal{P}_1(\perp)$, store sid in **open**, compute $\text{rnd}'_{\text{sid}} := \text{rnd}_{\text{sid}} - \text{dlog}_g C$ (this is possible due to Corollary 3). Output $(\text{sid}, a_{\text{sid}}, b_{1, \text{sid}}, b_{2, \text{sid}}, \text{rnd}'_{\text{sid}})$.
- $\mathcal{S2}(\text{sid}, e_{\text{sid}})$. Output $\perp$ if $\text{sid} \notin \mathbf{open}$. Store sid in **closed**, query the challenger's oracle $(c_{\text{sid}}, d_{\text{sid}}, r_{\text{sid}}, s_{1, \text{sid}}, s_{2, \text{sid}}) \leftarrow \mathcal{P}_2(\text{sid}, e_{\text{sid}})$, and output $(c_{\text{sid}}, d_{\text{sid}}, r_{\text{sid}}, s_{1, \text{sid}}, s_{2, \text{sid}})$.

- $H_1(Q)$. Via lazy sampling.
- $H_3(Q)$. If $T_3[Q] \neq \perp$, output $T_3[Q]$. Otherwise, parse $(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$, compute $\gamma := \text{dlog}_z \zeta$ using the representation of ζ if possible (if this query is used later to generate a signature, this is possible due to Corollary 4), otherwise respond to the query with a random value. Compute $z'_1 := \zeta_1^{1/\gamma}$, and find a signing session sid such that $z_{1,\text{sid}} = z'_1$ and set $\text{rnd}^* = \text{rnd}_{\text{sid}}$ (if this query is used later to generate a signature, this is possible due to Corollary 5). If no such signing session exists, respond with a uniformly random value. Next, set $A := \alpha, B_1 := \beta_1^{1/\gamma}, B_2 := \beta_2^{1/\gamma}$, make a query $(\text{vid}, \varepsilon_{\text{vid}}) \leftarrow_{\S} \mathcal{V}_1(\mathcal{A}, B_1, B_2, \text{rnd}^*)$. Finally, set $T_V[Q] := \text{vid}, T_3[Q] := \varepsilon$, and output ε .
- $H_{\text{pp}}(Q)$. Via lazy sampling.

When $\mathcal{A}$ outputs a signature $(\zeta, \zeta_1, \omega, \rho, \sigma_1, \sigma_2, \delta, \mu)$ satisfying the condition $\text{dlog}_z \zeta \neq \text{dlog}_{z_{\text{sid}} 1} \zeta_1$ for all signing sessions sid , $\mathcal{R}$ retrieves $\text{vid} \leftarrow T_3[(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta)]$ for $\zeta_2 := \zeta/\zeta_1$. If $\text{vid} = \perp$, $\mathcal{R}$ aborts and outputs $\perp$. This abort event does not occur because we know from game $\mathbf{G}_8$ that each signature has a corresponding hash query, and the responses of such queries were generated via $\mathcal{V}$ queries., hence each query is linked to a valid verifier session vid . Otherwise, it closes the session vid by sending $(\omega, \delta, \rho, \sigma_1/\gamma, \sigma_2/\gamma)$.

Analysis. First, we note that $\mathcal{R}$'s simulation for the signer's oracles is perfectly indistinguishable from an honest run of the protocol. To see this, note that the only difference between the prover's oracles of the challenger and the honest signer in ACL is the way z_1 is computed. More specifically, $z_1 = g^{\text{rnd}}$ in ID_{ACL} while $z_1 = g^{\text{rnd}} \cdot C$ in ACL . To avoid this inconsistency, $\mathcal{R}$ forwards an updated $\text{rnd}' = \text{rnd} - \text{dlog}_g C$ to $\mathcal{A}$ ensuring that $g^{\text{rnd}'} \cdot C = g^{\text{rnd}} \cdot C^{-1} \cdot C = g^{\text{rnd}}$. The resulting z_1 (if computed honestly) at the adversary side would be identical to z_1 at the prover side, hence forwarding the rest of the values from the prover to $\mathcal{A}$ would preserve the correctness and ensure a perfect simulation.

Second, we note that, if the reduction $\mathcal{R}$ does not abort and $\mathcal{A}$ outputs a valid signature allowing it to win the RB lemma, $\mathcal{R}$ would use this signature to close a verifier session successfully and satisfying the condition required to win the ℓ -RT game. Once $\mathcal{A}$ outputs a valid signature $(\zeta, \zeta_1, \omega, \rho, \sigma_1, \sigma_2, \delta, \mu)$ satisfying the condition $\text{dlog}_z \zeta \neq \text{dlog}_{z_{\text{sid}} 1} \zeta_1$ for all signing sessions sid , $\mathcal{R}$ searches for the verifier session vid that was opened to generate the challenge ε for this signature by searching $T_V[(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta, m)]$. Then, it sets $\omega' := \omega, \delta' := \delta, \rho' := \rho, \sigma'_1 := \sigma/\gamma, \sigma'_2 := \sigma_2/\gamma$ and closes the session vid by sending $(\text{vid}, \omega', \rho', \delta', \sigma'_1, \sigma'_2)$ in a $\mathcal{V}_2$ query. The verifier outputs 1 iff the values $(A, B_1, B_2, \text{rnd}^*)$ and $(\omega', \rho', \delta', \sigma'_1, \sigma'_2)$ satisfy the equalities $\varepsilon = \omega + \delta, \alpha' = g^{\rho'} \cdot y^{\omega'}, B_1 = g^{\sigma'_1} \cdot z_1^{\delta'}, B_2 = h^{\sigma'_2} \cdot z_2^{\delta'}$, where $z_1 = g^{\text{rnd}^*}$ and $z_2 := z_1/z$. Indeed, these equalities hold because

- $\omega' + \delta' = \varepsilon$ follows directly from the validity of the signature since $\omega = \omega', \delta = \delta'$, and $\varepsilon = H_3(\zeta, \zeta_1, \alpha, \beta_1, \beta_2, \eta, m)$,
- $\alpha' = g^{\rho'} \cdot y^{\omega'}$ follows directly from the validity of the signature since $\alpha = \alpha', \rho = \rho'$, and $\omega = \omega'$.

- Since the signature is valid, it holds that $\beta_1 = g^{\sigma_1} \cdot \zeta_1^\delta \Rightarrow \beta_1^{1/\gamma} = (g^{\sigma_1} \cdot \zeta_1^\delta)^{1/\gamma} \Rightarrow \beta_1^{1/\gamma} = g^{\sigma_1/\gamma} \cdot g^{\text{rnd}^* \cdot \delta} \Rightarrow B_1 = g^{\sigma_1'} \cdot z_1^{\delta'}$.
- Per the validity of the signature, we have $\beta_2 = h^{\sigma_2} \cdot \zeta_2^\delta \Rightarrow \beta_2^{1/\gamma} = (h^{\sigma_2} \cdot \zeta_2^\delta)^{1/\gamma} \Rightarrow \beta_2^{1/\gamma} = h^{\sigma_2/\gamma} \cdot (z/z_1)^\delta \Rightarrow B_2 = h^{\sigma_2'} \cdot z_2^{\delta'}$.

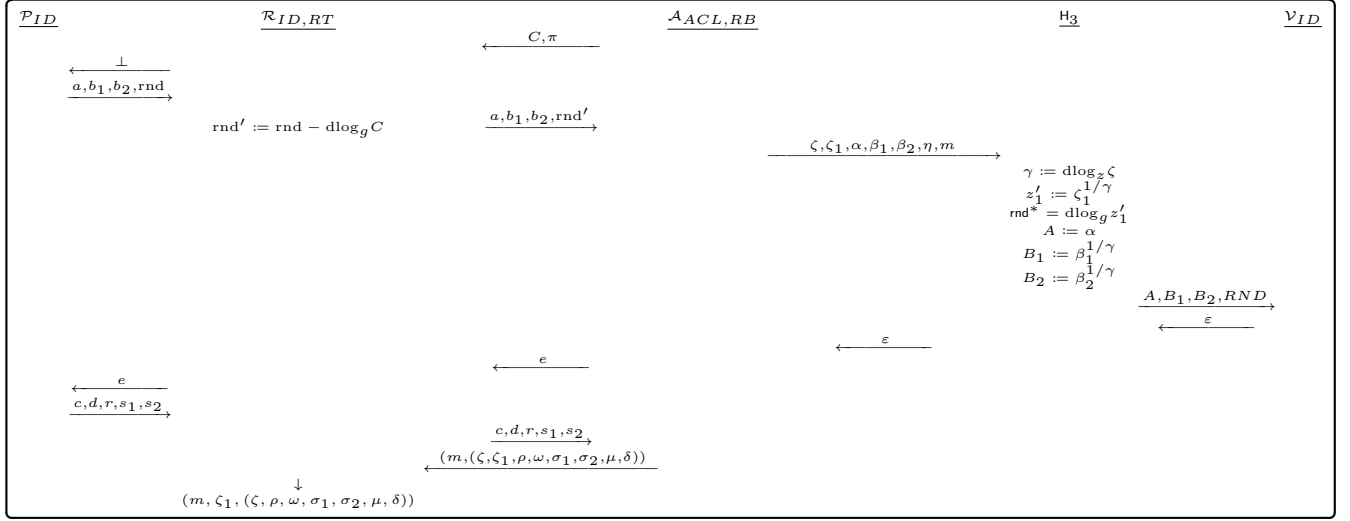


Fig. 5: A reduction from RB of ACL to the RT of ID_{ACL}.

I Deferred Figures

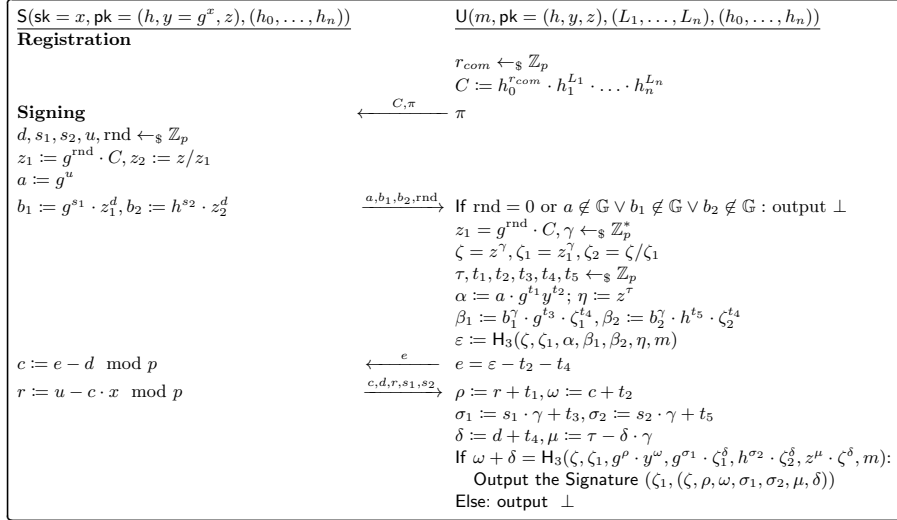


Fig. 6: The ACL scheme [4] depicted as an interactive protocol. The check $\omega + \delta = H_3(\zeta, \zeta_1, g^\rho \cdot y^\omega, g^{\sigma_1} \cdot \zeta_1^\delta, h^{\sigma_2} \cdot \zeta_2^\delta, z^\mu \cdot \zeta^\delta, m)$ also constitutes the verification equation for a resulting signature.

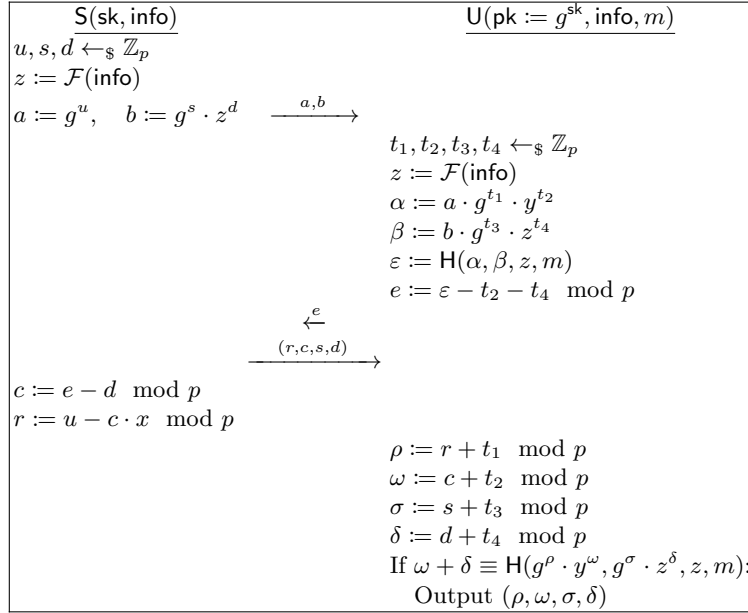


Fig. 7: The Abe-Okamoto (partially) blind signature scheme [2].